

Babet Manual

Contents

Babet — User manual	10
Getting started	10
Find a feature	11
Module reference	12
Structure of a module page	13
PDF generation	13
Getting started	14
Download	14
Build from source	14
Three ways to run a Lua script	14
1. Folder mode	14
2. Embedded mode (single-file executable)	14
3. Embedded via PATH (folder + auto-detect)	14
Command-line invocation	15
First script	15
Where to look next	15
Security model	16
What the hardening <i>does</i> protect against	16
What it does <i>not</i> protect against	17
Recommended pattern : least privilege	17
Archive — secure ZIP and TAR operations with gzip, xz, bzip2, or zstd	17
Scope	17
Contents	18
babet.archive.create	18
Output format	18
Inspection/extraction options and anti-bomb limits	21
babet.archive.list	22
babet.archive.extract	23
babet.archive.extractFile	24
Path rules	25
Destination protection	25
Atomic publication and cleanup	26
Permissions	26
Supported entry types	27
Errors and known limitations	27
argparse — command-line arguments	28
Module contents	28
Loading and metadata	29
API	29
Declaring flags and options	29
Declaring positionals	30

The opts table	30
Common fields	30
Value-only fields	30
Destination	31
Argument source	31
Global arg table	31
Explicit array	31
Recognized forms	31
Defaults, choices, and conversion	32
parse results	32
Normal success	32
Built-in help	32
User-input error	33
Programming error	33
Complete example	33
Text produced by get_usage	34
Current limitations	34
EXEC - external programs, capture, and streaming	34
Module contents	35
Essential conventions	35
No implicit shell	35
A non-zero exit is not a launch error	35
Data strings are binary-safe	36
API overview	36
Signature and arguments	36
cmd and program lookup	36
args and shell-free execution	37
Using a shell explicitly	37
Options	38
opts.cwd	38
opts.env	38
opts.stdin	39
opts.timeout	39
opts.max_output	40
Result table	40
Normal exit code	40
Signal termination	41
Timeout	41
Truncation	41
Complete examples	41
Basic capture	41
Arguments containing spaces	41
Handling a non-zero code	42
Capturing stderr	42
Sending text or binary stdin	42
Changing the environment	42
Changing the working directory	42
Bounding execution time	43
Bounding captured output	43
Building an application helper	43
Streaming processes with <code>babet.spawn</code>	44
Signature and options	44
Process methods	44
Reading stdout and stderr	45
Writing stdin	46
Waiting, terminating, and closing	46

Streaming loop example	46
Limits and deadlock prevention	47
Error contract	47
Raised Lua errors	47
Returned (nil, err) errors	47
Situations returning a table	48
Processes, signals, and security	48
Process group	48
Interaction with <code>babet.signal</code>	48
Memory	48
Design and limitations	48
Process pipelines	49
Shared command syntax	49
<code>babet.pipeline()</code> — complete execution	49
Signature	49
Options	50
Result	50
Timeout, large streams and truncation	51
<code>babet.spawnPipeline()</code> — streaming	51
Signature	51
Read methods	52
Writing and closing stdin	52
State and identifiers	53
Waiting and final status	53
Explicit termination	53
<code>close()</code> , GC and Lua <code><close></code>	53
Launch errors and reuse	54
FS — files, directories, paths, searching, copying, permissions, and checksums	54
Module contents	54
General conventions	55
Paths	55
Symbolic links	55
Result order	56
Return values	56
API overview	56
Check and inspect	56
Manipulate paths	56
Act on the filesystem	56
List and search	57
Copy and move	57
Change attributes	57
Existence, type, and size	57
<code>babet.fileExists(path)</code>	57
<code>babet.isFile(path)</code> and <code>babet.isfile(path)</code>	58
<code>babet.isDir(path)</code> and <code>babet.isdir(path)</code>	58
<code>babet.fileSize(path)</code>	58
Current working directory	58
<code>babet.currentDir()</code>	58
<code>babet.chdir(path)</code>	58
Lexical path manipulation	59
<code>babet.getBaseName(path)</code>	59
<code>babet.getFilename(path)</code>	59
<code>babet.getExtension(path)</code>	59

babet.getPath(path)	59
babet.joinPath(...)	59
Create, remove, rename, and link	60
babet.touch(path)	60
babet.mkdir(path)	60
babet.remove(path)	61
babet.rmdir(path)	61
babet.rmdirAll(path)	61
babet.rename(source, destination)	61
babet.link(target, linkpath)	62
List, iterate, and search	62
babet.listFiles(path, recursive?)	62
babet.createFileIterator(path, recursive?)	63
babet.find(path, opts?)	64
Copy and move	66
babet.copy(source, destination)	66
babet.copyTree(source, destination, continue_on_error?)	66
babet.moveTree(source, destination)	67
Unix permissions and attributes	68
babet.getMode(path) and babet.setMode(path, mode)	68
babet.getAttributes(path)	68
babet.setAttributes(path, uid, gid, mode?)	69
babet.symlinkAttr(path, uid, gid) and babet.symlinkattr(...)	69
File checksums	69
In-memory CRC-32	70
babet.crc32(data)	70
Error contract	71
Raised Lua errors	71
Returned errors	71
Partial operations	71
Design decisions and limitations	71
HTTP — synchronous web requests, headers, bodies, TLS, and limits	71
Module contents	72
Core conventions	72
Synchronous client	72
HTTP status is not a transport error	72
Binary bodies	73
Unknown option fields	73
API overview	73
babet.http.request(opts)	73
babet.http.get(url, opts?)	74
babet.http.download(url, destination, opts?)	74
babet.http.post call forms	75
URLs, fragments, and query	75
HTTP methods	76
Request headers	76
Request body and Content-Type	77
opts.timeout	77
HTTPS, verification, and CA	77
follow_redirects	78
max_body_size	78
max_file_size and file safety	78
Response tables	79
In-memory response	79
Download result	79
Complete examples	79

GET JSON	79
Authenticated JSON POST	80
Binary PUT	80
Download a release artifact	80
Separate network and HTTP failures	81
Parallel requests with WORKERS	81
Error contract	81
Security and limitations	81
babel.inotify — filesystem watching	82
Module contents	82
API	82
Creating a watcher	82
Adding a watch	83
onlydir option	83
Reading events	83
Queue overflow	84
Signal interruption	84
Removing and closing	84
Example	84
Error contract	85
Limits	85
babel.json — JSON encoding and decoding	85
Module contents	85
API	86
Type mapping	86
Lua to JSON	86
JSON to Lua	86
null	87
Empty object, empty array, and as_array	87
encode options	87
Strings, UTF-8, and NUL bytes	88
Numbers	88
Object order and duplicate keys	88
Error contract	88
Limits	89
logging — leveled logging	89
Module contents	89
Emitting a message	89
No-propagation guarantee	90
Output format	90
Threshold	90
Destination	90
ANSI colors	91
Module state	91
Error contract	91
Limitations	91
SIGNAL — graceful shutdown, reloads, and POSIX signals	92
Module contents	92
Essential conventions	92
Success returns one value	92
Names are strict	93
A callback receives no arguments	93
API overview	93

Supported signals	93
handle(name, fn) — install or replace	94
handle(name, nil) — remove a callback	94
ignore(name) — ignore	95
default(name) — restore the OS default	95
Callback execution	95
Main thread and safe context	95
Dispatch order	95
Occurrence coalescing	96
Callback errors	96
Interaction with blocking calls	96
Interaction with debug.sethook	97
Complete examples	97
Graceful shutdown on TERM and INT	97
Reload configuration with HUP	98
Interrupt a socket wait	98
Temporarily ignore PIPE	98
Observe callback errors	99
Error contract	99
Raised Lua errors	99
Returned system error	99
Workers, concurrency, and limitations	99
Features not provided	100
SOCKET — TCP clients and servers, binary streams, lines, and timeouts	100
Module contents	101
Core conventions	101
Synchronous TCP	101
Binary-safe data	101
Timeout rules	101
Return values	102
API overview	102
babet.socket.connect(host, port, timeout?)	102
babet.socket.listen(host, port, backlog?)	103
server:accept(timeout?)	103
sock:set_timeout(seconds)	103
sock:send(data)	104
sock:recv(count, timeout?)	104
sock:recv_line(timeout?)	104
sock:recv_all(timeout?, max_bytes?)	105
Mixing recv, recv_line, and recv_all	105
sock:peer() and sock:sockname()	105
sock:close() and garbage collection	105
Signals and interruption	106
Complete examples	106
Line-oriented echo server	106
Length-prefixed binary client	106
Response framed by EOF	107
Error contract	107
Security and limitations	107
babet.sqlite — embedded SQL database	107
Module contents	107
API	108
Opening and closing	108
Direct execution	108
Reusable prepared statements	109

Managed transactions	110
SQL parameters	111
Explicit BLOB values	111
SQL text and NUL bytes	112
Row reading and type mapping	112
Lifetime	112
Error contract	112
Complete examples	113
Prepared inserts and BLOB values	113
Reusable query and managed transaction	113
Functions not exposed	114
babet strings — splitting strings	114
Module contents	114
API	114
Separator mode	114
Limiting the number of cuts	115
Byte mode	115
Binary strings and NUL bytes	116
Empty subject	116
Error contract	116
Limits and design choices	116
SYS - process, host, environment, executables, version, and Lua memory	117
Module contents	117
General conventions	117
Flat functions	117
Strict strings	118
Process-global state	118
API overview	118
Version constants	118
Identify the process and host	119
babet.pid()	119
babet.hostname()	119
babet.uname()	120
Locate an executable	120
PATH lookup	120
Direct path	121
Missing or non-executable program	121
Environment variables	121
babet.env(name)	121
babet.setenv(name, value)	122
Empty value versus missing variable	122
Interaction with workers	123
Lua VM memory	123
babet.getMemoryUsage()	123
babet.getDetailedMemoryUsage()	123
What is and is not measured	124
Error contract	124
Design decisions and limits	124
babet tables — merging and copying Lua tables	125
Module contents	125
API	125
babet.mergeTables(t1, t2, ...)	125
Positive integer keys: concatenation	126
Every other key: last writer wins	126

Shallow merge	126
<code>babel.deepCopyTable(t)</code>	126
Values and shared graph structure	126
Keys are not copied	127
Metatables and raw traversal	127
Maximum depth	127
Error contract	127
Limits and design choices	128
Not in v1	128
babel.time - clocks, sleep, dates and durations	128
Module contents	128
API	128
<code>monotonic()</code> and <code>now()</code>	129
<code>sleep(amount [, unit])</code>	129
<code>babel.time.iso([ts])</code>	130
<code>babel.time.parse_iso(text)</code>	130
<code>babel.time.parse_duration(text)</code>	131
<code>babel.time.format_duration(seconds)</code>	131
Error contract summary	131
Limits and design choices	132
TLS — direct encryption, STARTTLS, certificates, and SNI	132
Module contents	132
Core conventions	132
Verification is on by default	132
Same stream API after handshake	133
SNI is not certificate verification	133
API overview	133
TLS options	133
<code>babel.socket.connect_tls(host, port, opts?)</code>	134
<code>sock:starttls(opts?)</code>	134
Verification, hostname, and SNI	135
Certificate authorities	135
TLS versions	135
Timeouts	135
Socket state after failure	135
Complete examples	136
Raw HTTPS request	136
IP connection with DNS identity	136
Simplified SMTP STARTTLS	136
Controlled self-signed test	136
Error contract	137
Security and limitations	137
babel.toml - TOML decoding	137
Module contents	137
API	137
Type mapping	138
Integers and floats	138
Strings, UTF-8, and keys	138
Arrays and tables	139
Empty-container ambiguity	139
Dates and times	139
Ordering and discarded information	140
Error contract	140
Limits	140

Not exposed in v1	141
USER - look up system accounts by name or UID	141
Module contents	141
General conventions	141
Dedicated subtable	141
Name or UID, without implicit conversion	141
Fields are always present	142
API overview	142
Returned user table	142
Look up an account with get	143
By name	143
By UID	143
Missing account	143
Test existence with exists	144
By name	144
By UID	144
When to prefer get	144
Argument validation	145
NSS, workers, and security	145
Account sources	145
Calls from workers	145
Security-sensitive decisions	146
Error contract	146
Design decisions and limits	146
WORKERS — OS threads, isolated Lua states, and message queues	147
Module contents	147
Essential conventions	148
Each worker owns a separate Lua state	148
Results use a pcall-like convention	148
The final result can be consumed once	148
API overview	148
Parent side	148
Worker side	149
spawn(code, args?, opts?)	149
code	149
args and worker.args	150
Queue capacities	150
When setenv and chdir become locked	150
Transferable values	151
Scalars and strings	151
List and object tables	151
Rejected values	152
Copies, identity, and depth	152
Final worker result	152
join() — wait and consume	153
poll() — check and consume	153
Do not call join() after completed poll()	153
Parent-to-worker messages	154
Worker-to-parent messages	154
worker.send and worker.recv	155
Timeouts and failure reasons	155
close() and queue lifecycle	156
Module loading and worker environment	156
Complete examples	156
Simple calculation with join	156

Structured arguments	157
Non-blocking progress with poll	157
Persistent request/response worker	157
Sending and receiving nil	158
Close, then drain the outbox	158
Backpressure and timeout	159
Cooperative cancellation	159
Correct bounded pool	160
Deadlocks to avoid	161
Worker waits for inbox while parent calls join()	161
Worker blocks on a full outbox	161
Forgetting that poll() consumes	161
Potentially blocking GC	161
Error contract	162
Lua errors raised for invalid use	162
Failures returned by spawn	162
Final worker errors	162
Message serialization	162
Garbage collection and destruction	162
Signals, environment, and concurrency	163
Signals	163
Environment and cwd	163
External resources	163
Cost	163
Features not provided	163
Cookbook	164
Watch a directory and email new files	164
Ensure a service user exists, then drop to it	164
Fetch many URLs in parallel	165
Graceful shutdown of a long-running script	165
Read a TOML config, validate, persist to SQLite	166
Cache with a human-readable TTL, expiry as ISO timestamps	166
Pretty-print a Lua value	167
Hello there	167

Babet — User manual

Babet, n.m. — a regional word from south-eastern France (Lyonnais, Forez, Dauphiné, Savoie, and neighbouring Swiss Romandy) meaning a pine cone. Small, light, full of seeds, and able to start a fire—like this binary.

Babet is a standalone Lua binary for Linux scripting and automation. This manual is organised by need and by module: the tables below describe both the technical chapter name and the features it contains.

Documentation for the stable and audited **2.6.0** release.

Getting started

- [Getting started](#) — installation, first script, running a file or directory, creating an embedded executable, and using Babet through PATH.
- [Security](#) — threat model, actual protections, limitations, and least-privilege rules.
- [Cookbook](#) — complete recipes combining several modules.
- [Changelog](#) — complete 2.6.0 release notes and the history of earlier releases, migration, validation, and known limitations.

Find a feature

I need to...	Chapter	What it contains
create, remove, list, search, copy, or move files	FS – filesystem	files, directories, paths, symlinks, <code>listFiles</code> , <code>find</code> , <code>copyTree</code> , permissions, and checksums
parse script arguments	Argparse – command line	flags, options, positional arguments, defaults, choices, and conversion
run an external program	Exec – processes	shell-free arguments, complete capture with <code>exec</code> , streaming with <code>spawn</code> , environment, <code>cwd</code> , and process control
chain several commands	Process pipelines	complete capture or streaming, separate <code>stderr</code> streams, per-stage statuses, and process-group cleanup
create or inspect/extract ZIP or TAR archives	Archive – secure multi-format archives	deterministic ZIP and plain/gzip/xz/bzip2/zstd TAR creation, listing, extraction, anti-bomb limits, special-type refusal, and atomic file publication
inspect the process, environment, machine, or Lua VM memory	SYS – process and machine	runtime version, PID, <code>hostname</code> , <code>uname</code> , <code>PATH</code> , <code>env</code> , <code>setenv</code> , and Lua-state memory
send an HTTP request	HTTP – web client	GET/POST and other methods, query, validated headers, binary bodies, atomic downloads, redirects, TLS, timeout, and size limits
open a TCP connection	Socket – TCP	client, server, <code>accept</code> , binary streams, lines, EOF reads, timeouts, buffers, and limits
encrypt TCP or perform STARTTLS	TLS – secure sockets	direct TLS, STARTTLS, verification, CA, <code>hostname</code> , SNI, versions, timeout, and fail-closed state
store SQL data locally	SQLite – embedded database	opening, execution, prepared statements, explicit BLOBs, and assisted transactions
encode or decode JSON	JSON – structured data	scalars, arrays, objects, null, empty arrays, pretty printing, and errors
read TOML configuration	TOML – configuration	decoding, TOML types, arrays, sections, dates, and parse errors
watch a directory	Inotify – file events	watches, events, timeout reads, moves, cookies, and closing
handle Unix signals	SIGNAL – clean shutdown and reload	TERM/INT/HUP/USR1/USR2/PIPE, deferred callbacks, fixed order, coalescing, interruptible calls, and workers

I need to...	Chapter	What it contains
parallelise Lua work	WORKERS – OS threads and messages	isolated Lua states, spawn, join, poll, inbox/outbox, timeouts, closing, serialisation, and deadlocks
obtain or format time	Time – clocks and durations	realtime and monotonic clocks, sleep, ISO-8601, parsing, and duration formatting
look up a system account by name or UID	USER – system accounts	NSS, get, exists, UID, primary GID, GECOS, home, shell, and resolution errors
split or transform strings	Strings	split, separators, limits, character mode, and binary strings
copy, merge, or mark tables	Tables	mergeTables, deepCopyTable, cycles, and shared structures
produce logs	Logging	levels, threshold, destination, colours, and sink-error behaviour

Module reference

Each module has a standalone page under `modules/`. The expanded names below define each chapter’s scope before you open it.

Module	Detailed scope
Archive – secure ZIP, TAR, gzip, xz, bzip2, and zstd operations	Create deterministic ZIP, TAR, gzip TAR, xz TAR, bzip2 TAR, or zstd TAR archives and inspect/extract them with bounded resources, confined paths, explicit special-type refusal, and atomic per-file publication.
Argparse – command-line arguments	Declare flags, options, and positional values; generate help; validate choices; convert values.
Exec – external programs and processes	Run without a shell using exec, or progressively drive stdin/stdout/stderr using spawn.
Process pipelines	Connect commands without a shell, using complete capture or streaming and per-stage statuses.
FS – files, directories, paths, and attributes	Existence and types, creation/removal, paths, listings, search, iterators, tree copy/move, symlinks, Unix modes, and checksums.
HTTP – web requests	URLs, methods, query, validated headers, binary bodies, responses, atomic file downloads, redirects, TLS verification, timeout, and limits.
Inotify – filesystem monitoring	Add/remove watches, read events, handle timeouts, moves, cookies, and closing.
JSON – encoding and decoding	Lua/JSON types, null, empty arrays, array marking, indentation, UTF-8, cycles, and limits.
Logging	Levels, filtering, destination, colours, and variadic calls.
SIGNAL – POSIX signals and clean shutdown	Install, replace, or remove callbacks; ignore/restore; dispatch order, coalescing, interruptions, and multithread restrictions.

Module	Detailed scope
Socket – TCP client and server	Connect, listen, accept, binary streams, block/line/EOF reads, shared buffers, addresses, closing, and timeouts.
SQLite – embedded database	Connections, WAL/timeout, direct SQL, reusable prepared statements, explicit BLOBs, iteration, and assisted transactions.
Strings	Splitting, character mode, separators, split limits, and binary content.
SYS – process, machine, environment, and Lua memory	Version constants, PID, hostname, uname, executable lookup, environment reads/writes, worker interaction, and Lua-state memory.
Tables	Deterministic merging, list/map keys, deep copy, cycles, and shared subtables.
Time – clocks, ISO, and durations	Realtime and monotonic clocks, sleep, ISO-8601 formatting/parsing, and duration parsing/rendering.
TLS – encrypted connections	Direct TLS, STARTTLS, verification, CA, hostname, SNI, versions, deadlines, and fail-closed behaviour.
TOML – configuration files	TOML decoding, scalars, arrays, tables, arrays of tables, dates/times, and diagnostics.
USER – NSS system users	Lookup by name or UID, existence, absence vs NSS errors, passwd fields, workers, and security limits.
WORKERS – OS threads and message queues	Isolated Lua states, JSON transport, consumable results, poll/join, inbox/outbox, timeouts, closing, GC, and deadlock traps.

Structure of a module page

Detailed pages are progressively aligned on this structure:

1. **Scope** — what the module covers and deliberately excludes;
2. **Internal contents** — direct access to each function group;
3. **API overview** — signatures and results;
4. **Detailed behaviour** — defaults, symlinks, recursion, limits, and side effects;
5. **Examples** — one example per important mode or option;
6. **Error contract** — raised errors and returned errors;
7. **Design decisions and limitations** — intentional choices and omissions.

The goal is to let a function be used without reading its source code or having to guess default values.

PDF generation

The whole manual can be exported as one PDF:

```
cd docs
./build_doc.sh
```

English chapter order is defined in `manual_order_en.txt`.

Getting started

Download

Prebuilt binaries for x86_64, aarch64 (RPi4), and armv6l (RPi0) are available on the [releases page](#).

Build from source

Babet vendors all its dependencies (Lua, OpenSSL, SQLite, miniz, libarchive, zlib, XZ Utils/liblzma, bzip2/libbz2, Zstandard/libzstd, RE2, Abseil, nlohmann/json, cpp-httplib, tomlplusplus), so the only prerequisites on your system are :

- a C++23 compiler (recent GCC or Clang)
- CMake (≥ 3.22)
- wget, unzip, and xz

```
git clone https://github.com/Chipsterjulien/babet.git
cd babet
./build_local.sh          # downloads deps and compiles
                           # (~5 min on first run, faster afterwards)
./run_tests.sh           # offline harness – should finish with 0 FAIL
```

The build script downloads each dependency from a pinned upstream source, verifies its SHA256, then compiles it statically. Several dependencies have an explicit mirror or fallback URL. When a source has no mirror, the script prints the exact archive name to place manually in downloads/; the same SHA256 check still applies before extraction.

The resulting binary is at test/babet.

Three ways to run a Lua script

Babet supports three launch modes :

1. Folder mode

Point the binary at a directory containing main.lua :

```
echo 'print("hello from Babet")' > main.lua
./test/babet .
```

require("X") in main.lua looks for X.lua in the same directory.

2. Embedded mode (single-file executable)

Package a script and its modules into a self-contained binary :

```
./test/babet --create-exe . myapp
./myapp      # runs main.lua from the embedded ZIP
```

Version-control metadata directories .git, .svn, and .hg are automatically excluded at any depth.

Internally, Babet appends a ZIP to its own binary, and reads main.lua (plus any required module) from that ZIP at runtime. The resulting myapp is fully self-contained — no Babet install needed on the target machine, just glibc.

3. Embedded via PATH (folder + auto-detect)

If Babet is on \$PATH and you chmod +x main.lua after adding a shebang line :

```
#!/usr/bin/env babet
print("hello")
```

```
chmod +x main.lua
./main.lua
```

Babet uses the script's own directory as the folder, so `require("helpers")` finds `helpers.lua` next to `main.lua`.

Command-line invocation

In addition to the three launch modes above, Babet accepts a few standard flags :

Flag	Effect
-h, --help	Print the help text and exit 0.
-V, --version	Print babet <version> and exit 0.
-c <dir> <out>, --create-exe <dir> <out>	Create a self-contained executable named <out> by embedding <dir> (which must contain <code>main.lua</code>).

Any other argument starting with - is treated as an **unknown option** : Babet prints `Unknown option: ...` + a hint to use `--help`, and exits 1 instead of trying to interpret it as a directory name. Folders whose name legitimately starts with - can still be passed via `./-dirname` (POSIX convention).

```
babet --version    # babet 2.6.0
babet --help      # full usage
babet --bogus     # Unknown option: --bogus
                  # Try 'babet --help' for more information.
```

The same version is also exposed to scripts as `babet.VERSION` (plus `babet.VERSION_MAJOR` / `VERSION_MINOR` / `VERSION_PATCH` as integers) — see [sys](#).

First script

Once the binary is built, try this :

```
-- main.lua
print("Babet says hi")
print("PID:", babet.pid())
print("Host:", babet.hostname())

local r, err = babet.http.get("https://example.com/")
if r then
    print("Status:", r.status)
else
    print("HTTP failed:", err)
end
```

Run it with `./test/babet .` (or whichever mode you prefer).

Where to look next

- Each module has its own page under `modules/`.
- The [user](#) module is a small, complete example of the documentation pattern used throughout — read it as the canonical reference.
- For date and duration utilities (ISO 8601 timestamps, human durations like "5m" or "2h30m"), see [time](#) — the `babet.time.*` sub-table covers parsing and formatting.
- See [security.md](#) before exposing Babet scripts to anything that might receive untrusted input.

Security model

Babet runs trusted Lua scripts. It is not a sandbox.

Scripts execute with the full Lua standard library (`luaL_openlibs`), and Babet additionally exposes filesystem and process primitives such as `exec`, `remove`, `rmdirAll` and `chdir`. A script therefore has the same privileges as the process running it : it can read, modify or delete files and run arbitrary commands. This is by design — like `make`, a shell script, or any build tool, Babet is meant to run code you control.

Only run `main.lua` and required modules that you trust. Do **not** use Babet to execute Lua from untrusted sources ; it provides no isolation against hostile code, and none is intended. If you need to run untrusted scripts, use a purpose-built Lua sandbox instead.

What the hardening *does* protect against

The hardening work in Babet protects *legitimate* use against accidents and supply-chain tampering :

- **copyTree / moveTree confinement** : the destination root is opened once, then every create, copy and move is performed relative to that descriptor with `openat/mkdirat/renameat` and `O_NOFOLLOW/AT_SYMLINK_NOFOLLOW`. A pre-existing symlink, or one swapped during the operation, cannot redirect a write to an outside target. The `is_within()` guards additionally reject a destination resolving inside the source.
- **Confined ZIP creation and extraction**: `babet.archive` refuses symlinked creation-source and output/extraction-destination paths, unsupported source objects, non-UTF-8 source names, outputs inside the source tree, absolute archive-entry paths, `./.` components, backslashes, duplicates, ZIP symlinks, and special filesystem types. Input archives are opened once and must resolve to a regular file, excluding blocking FIFO sources. It walks the destination through descriptors with `O_NOFOLLOW`; each file is decompressed into a same-directory temporary, checked against anti-bomb limits, then published atomically.
- **Bounded process-group cleanup** in `babet.exec` also covers the `chdir/exec` launch phase, internal polling failures, and children that close all pipes while continuing to run. On timeout, `TERM` then `KILL` target the whole group; these error paths never end in an unbounded final `waitpid`.
- **Output limits** on `babet.exec` bound the amount of `stdout` and `stderr` captured into memory (default 10 MiB each, configurable), so a runaway subprocess can't OOM the Babet process.
- **Pinned local metadata updates**: `setAttributes` resolves its target once and keeps `chown`, `chmod`, and `rollback` attached to the same inode. `touch` creates with `O_EXCL`, never truncates an existing file, and pins an existing target before changing its timestamp.
- **Anonymous and pinned executable staging**: `--create-exe` unlinks its temporary ZIP immediately after `mkstemp` and lets `miniz` and the merge step reopen the still-live inode through `/proc/self/fd`, not through a replaceable temp path. The final merge pins the output directory and publishes with `renameat()`, so changing a parent symlink cannot redirect the executable.
- **Dependency checksums** : every vendored dependency is SHA256-pinned in `build_local.sh`. An empty hash refuses to build. A mismatch deletes the downloaded file and exits with an error. This protects against a compromised upstream or Wayback Machine archive (which is the fallback source).
- **TLS verification** is on by default (`verify=true`). Disabling it requires an explicit per-call `verify=false` — there is no silent fallback. TLS sockets accept extra trust through `ca_cert` or `ca_path`; HTTP exposes `ca_cert`. OpenSSL's `SSL_CERT_FILE` / `SSL_CERT_DIR` remain available. SNI is sent independently from verification when a DNS hostname is known.
- **HTTP line validation** rejects CR/LF in URLs and header values, and header names must follow the HTTP token grammar. User-controlled values cannot inject an extra header or request line through those fields.
- **Network memory limits** bound `socket:recv_line`, `socket:recv_all`, and `http.max_body_size`. After a socket-read timeout, already consumed bytes stay in one shared pending buffer so switching receive methods cannot lose or reorder the stream.

- **Bounded filename matching** provides a non-recursive safe glob engine and RE2-backed regular expressions for `babel.find`. Regex patterns are capped at 4096 bytes, compiled with a 1 MiB memory budget, and cannot trigger catastrophic backtracking. RE2-unsupported constructs fail before traversal.

What it does *not* protect against

- A malicious script. Babet has no sandbox. `babel.exec` does not invoke a shell, but allowing untrusted input to choose the executable or its arguments can still permit arbitrary program execution or option injection. Passing untrusted text to `load` gives arbitrary Lua code execution.
- A determined attacker who has obtained code execution on the machine running Babet. The hardening makes accidental mistakes loud, not adversarial attacks impossible.
- Long-tail OS-level issues (kernel exploits, container escapes, privilege escalation). Babet is an ordinary userland binary.

Recommended pattern : least privilege

When running Babet as a service, treat it like any other unprivileged process :

As root, create a dedicated service user with no shell.

```
useradd --system \
        --home-dir /var/lib/myapp \
        --create-home \
        --shell /usr/sbin/nologin \
        --comment "myapp Babet service" \
        myapp
```

*# Use babel.user.exists("myapp") in your install script to
verify the user is provisioned before launching the daemon.*

Combine with `systemd` unit file options like `User=myapp`, `PrivateTmp=yes`, `ProtectSystem=strict`, `NoNewPrivileges=yes`, and `CapabilityBoundingSet=(empty unless you genuinely need a capability)`. The kernel does much more than Babet can to contain a misbehaving script ; let it.

Archive — secure ZIP and TAR operations with gzip, xz, bzip2, or zstd

Scope

The `babel.archive` submodule creates, inspects, and extracts **ZIP**, **TAR**, and **gzip-**, **xz-**, **bzip2-**, or **zstd-compressed TAR** archives without calling an external command:

```
babel.archive.create(source, archive [, opts])
babel.archive.list(archive [, opts])
babel.archive.extract(archive, destination [, opts])
babel.archive.extractFile(archive, entry, destination [, opts])
```

ZIP creation, inspection, and extraction continue to use `miniz`. TAR creation, inspection, and extraction use the statically linked `libarchive` backend; gzip filtering is provided by the statically linked `zlib` backend, xz filtering uses the statically linked `XZ Utils/liblzma` backend, bzip2 filtering uses the statically linked `bzip2/libbz2` backend, and zstd filtering uses the statically linked `Zstandard/libzstd` backend. Readers detect both the container and compression from the contents. For `create()`, `.tar` selects plain TAR, `.tar.gz` and `.tgz` select gzip-compressed TAR, `.tar.xz` or `.txz` select xz-compressed TAR, `.tar.bz2`, `.tbz2`, or `.tbz` select bzip2-compressed TAR, and `.tar.zst`, `.tar.zstd`, or `.tzst` select zstd-compressed TAR. `opts.format` can override the extension. Standalone compressed streams are not enabled.

All four functions use the usual result contract:

```
local result, err = babet.archive.list("backup.zip")
if not result then
    io.stderr:write(err, "\n")
end
```

They return (result, nil) on success and (nil, message) on failure. Wrong arity or a mandatory argument that is not a string raises a Lua error. An invalid option table returns (nil, message).

Contents

- [babet.archive.create](#)
- [Inspection/extraction options and anti-bomb limits](#)
- [babet.archive.list](#)
- [babet.archive.extract](#)
- [babet.archive.extractFile](#)
- [Path rules](#)
- [Destination protection](#)
- [Atomic publication and cleanup](#)
- [Permissions](#)
- [Supported entry types](#)
- [Errors and known limitations](#)

babet.archive.create

```
local result, err = babet.archive.create(
    source,
    archive
    [, opts]
)
```

source must be an existing directory. Babet archives its contents, not the source directory name itself. Traversal is descriptor-based and does not follow symlinks. A symlink anywhere in the source path or tree, or a FIFO, socket, device, or other unsupported object, makes the operation fail before the archive is published. Entry names produced by create() must be valid UTF-8. Linux permits arbitrary bytes in filenames, but Babet rejects names that cannot be represented safely and consistently in both supported output formats.

Output format

Without opts.format, create() uses these rules:

- a destination ending in .tar (case-insensitive) creates an uncompressed POSIX pax TAR archive;
- .tar.gz and .tgz (case-insensitive) create a gzip-compressed POSIX pax TAR archive;
- .tar.xz and .txz (case-insensitive) create an xz-compressed POSIX pax TAR archive;
- .tar.bz2, .tbz2, and .tbz (case-insensitive) create a bzip2-compressed POSIX pax TAR archive;
- .tar.zst, .tar.zstd, and .tzst (case-insensitive) create a zstd-compressed POSIX pax TAR archive;
- every other destination keeps the pre-2.6 historical behaviour and creates a ZIP archive, regardless of its extension.

opts.format = "zip", "tar", "tar.gz", "tar.xz", "tar.bz2", or "tar.zst" explicitly selects a backend. An explicit format always overrides the destination suffix. This means, for example, that format = "zip" may create a ZIP named backup.tar.gz, while format = "tar" may create an uncompressed TAR named backup.tgz or backup.txz, backup.tbz2, or backup.tzst. Format selection is based on the explicit option or content, not trust in the suffix.

Creation options:

Option	Default	Behaviour
format	inferred	strict string: "zip", "tar", "tar.gz", "tar.xz", "tar.bz2", or "tar.zst"
compression_level	6	ZIP/gzip/xz: 0 through 9; bzip2: 1 through 9; zstd: 0 through 19
overwrite	false	atomically replaces an existing regular archive
deterministic	true	stable entry order and fixed format-specific timestamps
include_directories	true	emits explicit directory entries, including empty directories
max_entries	10000	maximum output entries; hard ceiling 100000
max_file_size	256 * 1024 * 1024	maximum source-file size; hard ceiling 8 GiB
max_total_size	1024 * 1024 * 1024	maximum source bytes; hard ceiling 64 GiB

For plain TAR, an explicitly supplied `compression_level` is rejected. For ZIP, gzip-compressed TAR, and xz-compressed TAR, level 0 is valid and level 9 is the maximum. bzip2 accepts levels 1 through 9; level 0 is rejected because libbz2 block sizes start at 100 KiB. zstd accepts levels 0 through 19; Babet deliberately excludes the ultra levels 20 through 22.

All integer options require strict Lua integers. Boolean options require real Lua booleans. Unknown keys are rejected. `format` is case-sensitive and does not accept aliases.

The complete source tree is scanned before the output temporary file is created. Entry names are limited to 4096 bytes each and 64 MiB cumulatively. The scan also has fixed ceilings of 100000 filesystem nodes and 256 directory levels, independently of `include_directories`. Files are then reopened through the source directory descriptor with `O_NOFOLLOW`. Device, inode, size, modification time, and change time are checked before and after streaming; a file changed during creation causes a failure and the temporary archive is removed.

The destination parent must already exist and may not contain symlink or `..` components. The destination itself must be absent unless `overwrite = true`, and it may never be a symlink or directory. An archive cannot be written inside the source tree, avoiding accidental self-inclusion.

The result includes the selected format:

```
{
  files = 12,
  directories = 3,
  bytes = 987654,
  path = "backup.tar",
  format = "tar",
  compression = "none",
  compression_level = nil,
  deterministic = true,
}
```

For ZIP and gzip-, xz-, bzip2-, or zstd-compressed TAR, `compression_level` is the effective integer level. For plain TAR it is `nil`. Archive-level compression is "gzip", "xz", "bzip2", or "zstd" for

compressed TAR and "none" otherwise. ZIP entry compression remains reported through the ZIP-specific per-entry metadata returned by `list()`.

With the default deterministic mode, identical directory contents, format, libarchive/miniz/zlib/liblzma/libbz2/lib versions, and options produce byte-for-byte identical archives:

- ZIP entries use the fixed DOS timestamp 1980-01-01 00:00:00;
- TAR entries use Unix epoch zero, UID/GID 0, empty owner/group names, mode 0644 for files, and mode 0755 for directories;
- gzip-compressed TAR additionally writes a gzip header timestamp of zero, so the outer stream does not inject wall-clock time;
- xz streams contain no wall-clock timestamp, and the selected LZMA2 preset is fixed by `compression_level`;
- bzip2 streams contain no wall-clock timestamp, and their block size is fixed by `compression_level`;
- zstd frames contain no wall-clock timestamp, enable their frame checksum, and use the compression level fixed by `compression_level`.

With `deterministic = false`, sorted entry order remains stable. ZIP uses each source modification time, which must fit the DOS range 1980 through 2107 in the local timezone. TAR preserves the source modification time with nanosecond precision when representable by the platform and is not limited to the ZIP DOS range.

Ownership, ACLs, extended attributes, and source permission bits are not copied. TAR uses libarchive's restricted POSIX pax writer: ordinary names use portable ustar headers where possible, while long paths or metadata that need extension records are represented with pax headers. ZIP64 output is enabled automatically when the preflight indicates that classic ZIP fields may be exceeded. Small ZIP archives remain classic ZIP files.

The preflight and file checks prevent symlink substitution and detect changes to every file that is actually archived. Creation is not a filesystem snapshot: a file added after its directory was scanned may be absent from the result; a previously enumerated file that is then removed or renamed normally makes its secure reopen fail. Concurrent directory renames are not reported as a transactional conflict.

When `include_directories = false`, empty directories cannot be represented and are therefore omitted. Parent directories needed by regular files are recreated implicitly during extraction.

Examples:

```
local zip, err = babet.archive.create("project", "project.zip", {
    compression_level = 9,
    overwrite = true,
})
assert(zip, err)

local tar
tar, err = babet.archive.create("project", "project.tar", {
    deterministic = true,
})
assert(tar, err)

local compressed
compressed, err = babet.archive.create("project", "project.tar.gz", {
    compression_level = 9,
})
assert(compressed, err)

local xz
xz, err = babet.archive.create("project", "project.tar.xz", {
```

```

    compression_level = 9,
})
assert(xz, err)

local bzip2
bzip2, err = babet.archive.create("project", "project.tar.bz2", {
    compression_level = 9,
})
assert(bzip2, err)

local zstd
zstd, err = babet.archive.create("project", "project.tar.zst", {
    compression_level = 19,
})
assert(zstd, err)

local named
named, err = babet.archive.create("project", "snapshot.data", {
    format = "tar.zst",
})
assert(named, err)

```

Inspection/extraction options and anti-bomb limits

The options shared by `list()`, `extract()`, and `extractFile()` are:

Option	Default	Accepted maximum	Effect
<code>max_entries</code>	10000	100000	maximum number of entries in the archive
<code>max_entry_size</code>	$256 * 1024 * 1024$	$8 * 1024^3$	maximum expanded size of one non-directory entry
<code>max_total_size</code>	$1024 * 1024 * 1024$	$64 * 1024^3$	maximum sum of expanded non-directory entry sizes
<code>max_compression_ratio</code>	1000	1000000000	maximum per-entry ratio for ZIP; maximum global expanded-file-bytes/archive-bytes ratio for gzip-, xz-, bzip2-, or zstd-compressed TAR; irrelevant for plain TAR

The three integer limits must be **strictly positive Lua integers**. `max_compression_ratio` must be finite and at least 1. NaN, infinity, booleans, and numeric strings are rejected.

Independently of the options, the cumulative size of all entry names is internally limited to 64 MiB. This fixed ceiling bounds metadata memory even when `max_entries` is explicitly raised. Allocations made by the miniz ZIP reader are additionally capped at 128 MiB per archive. The central directory

and its comments therefore cannot trigger an arbitrary allocation before per-entry limits are applied. TAR inspection is streaming: Babet reads every entry data stream without storing it in memory, both to reach the following header and to detect truncated data. For gzip TAR, Babet additionally performs a bounded streaming validation with zlib because libarchive itself does not verify the gzip trailer CRC/ISIZE; corrupt trailers and non-gzip trailing bytes are therefore rejected. Standard zero padding after a gzip member is accepted. For zstd TAR, Babet performs an independent bounded streaming validation with libzstd, requiring complete frames, accepting valid concatenated frames, and rejecting corruption, truncation, or foreign trailing bytes before publication.

For complete extraction, the output tree is also internally capped at 100000 unique directories, including implicit parents, and 64 MiB of cumulative normalized directory paths. A few extremely deep entries therefore cannot cause an unbounded explosion in mkdir operations or prefix-memory use.

These limits are applied while inspecting the **whole archive**, including with `list()` and `extractFile()`. Therefore `extractFile()` cannot bypass the global policy by selecting a small entry from an archive whose metadata announces excessive expansion.

For ZIP, a non-empty entry that announces a zero compressed size is always rejected and the ratio is checked per entry. For gzip-, xz-, bzip2-, or zstd-compressed TAR, Babet checks the sum of announced regular-file sizes against the complete compressed archive size after the stream has been consumed. Plain TAR has no compression ratio. Directories do not count towards `max_entry_size` or `max_total_size`, but they do count towards `max_entries`.

Example:

```
local info, err = babet.archive.list("upload.zip", {
    max_entries = 2000,
    max_entry_size = 64 * 1024 * 1024,
    max_total_size = 512 * 1024 * 1024,
    max_compression_ratio = 200,
})
```

Unknown keys are rejected. Creation and extraction-only options are not accepted by `list()`.

babet.archive.list

```
local info, err = babet.archive.list(archive [, opts])
```

`archive` is the path to a ZIP, plain TAR, gzip-compressed TAR, xz-compressed TAR, bzip2-compressed TAR, or zstd-compressed TAR file. The extension is not used for detection. The path may be a symlink to a regular file: Babet opens its target once and keeps that same descriptor for the whole scan. Directories, FIFOs, sockets, and devices are rejected before format parsing; non-blocking open notably prevents a FIFO with no writer from hanging `list()`.

For ZIP, Babet first uses the existing miniz reader. If the same pinned file is not a ZIP, Babet rewinds that descriptor and enables only libarchive's TAR reader plus the built-in none, gzip, xz, bzip2, and zstd filters. No external decompressor can be invoked by this path. Concatenated TAR archives are traversed completely; non-TAR trailing data, truncated payloads, and data attached to non-regular entries are rejected. The function applies the limits above and returns:

```
{
  format = "zip",           -- "zip" or "tar"
  compression = "none",    -- "none", "gzip", "xz", "bzip2", or "zstd" for TAR
  entries = {
    {
      name = "docs/readme.txt", -- raw entry name
      path = "docs/readme.txt", -- normalised extraction path
      type = "file",           -- see the supported entry types below
      size = 1234,
      compressed_size = 530,
    }
  }
}
```

```

        crc32 = 305419896,          -- unsigned 32-bit integer
        compression_method = 8,
        encrypted = false,
        supported = true,
        safe_path = true,
        extractable = true,
        reason = nil,
        unix_mode = 420,            -- 0644, or nil without a Unix mode
        sparse = false,
        link_target = nil,         -- TAR symlink/hard-link target
    },
},
count = 1,
total_size = 1234,
archive_size = 666,
zip64 = false,                   -- nil for TAR
}

```

path removes the trailing / from a directory entry. It is only meaningful as an extraction path when `safe_path == true`.

For ZIP, compression remains "none": ZIP compression is per entry rather than an outer stream. For TAR, compression is "none", "gzip", or "xz" according to the detected filter.

For ZIP, extractable means that the entry has an acceptable path and type, uses a compression method understood by miniz, and is not encrypted. For TAR, safe regular files and directories are extractable. Sparse files, symlinks, hard links, FIFOs, devices, and other special TAR types are inspected but rejected with an explicit reason. extractable never predicts destination conflicts or duplicates between entries.

ZIP-specific fields keep their previous exact values. For TAR, `compressed_size`, `crc32`, and `compression_method` are nil, because TAR has no per-entry compression or CRC metadata. `encrypted` is false, `supported` means that libarchive could parse the entry, and `sparse` reports sparse-file metadata. `link_target` contains the raw target announced for a TAR symlink or hard link when libarchive exposes one, and is nil otherwise. The TAR types reported by this lot are file, directory, symlink, hardlink, fifo, character_device, block_device, socket, and unsupported.

`unix_mode` contains only bits 0000 through 7777 announced by an archive created on a Unix host. It is nil when that information is unavailable. A mode being present does not mean it will be preserved: see [preserve_permissions](#).

ZIP names read from an existing archive are byte strings. TAR pathnames are the native strings returned by libarchive after its TAR header handling (including ustar prefixes, GNU long names, and pax path headers). Comparisons are exact and case-sensitive, and Babet performs no additional Unicode normalisation. This inspection tolerance does not weaken `create()`, which emits valid UTF-8 names for both ZIP and TAR.

babet.archive.extract

```

local result, err = babet.archive.extract(
    archive,
    destination
    [, opts]
)

```

Additional options:

Option	Default	Behaviour
overwrite	false	allows atomic replacement of an existing regular file
preserve_permissions	false	preserves ordinary Unix rwx bits when available

Both values must be real Lua booleans.

archive may be a ZIP, plain TAR, gzip-compressed TAR, xz-compressed TAR, bzip2-compressed TAR, or zstd-compressed TAR; the format and compression are detected from its contents, not its extension. Before writing any file, Babet:

1. inspects the whole archive and applies the limits;
2. rejects every non-extractable entry;
3. rejects duplicate output paths;
4. rejects conflicts where one path would need to be both a file and a directory;
5. checks existing destination objects.

For TAR, Babet then rereads the **same pinned file** from the beginning, compares every header with the plan produced by inspection, and forwards regular-file blocks to the secure destination layer. An entry added, removed, or changed in metadata between the two passes makes the operation fail. This check is not a snapshot: concurrent same-size data modification can still trigger a read error, but is not guaranteed to be distinguished when the headers remain identical.

The result is:

```
{
  files = 12,
  directories = 3, -- explicit directory entries in the archive
  bytes = 987654, -- expanded file bytes
  path = "restore",
}
```

directories does not count implicit parents created to reach a file.

Example:

```
local result, err = babet.archive.extract("backup.tar", "restore", {
  overwrite = false,
  preserve_permissions = true,
  max_total_size = 2 * 1024 * 1024 * 1024,
})

if not result then
  error(err)
end

print(result.files, result.bytes)
```

babet.archive.extractFile

```
local result, err = babet.archive.extractFile(
  archive,
  entry,
  destination
[, opts]
)
```

archive may be a ZIP, plain TAR, gzip-compressed TAR, xz-compressed TAR, bzip2-compressed TAR, or zstd-compressed TAR; detection uses the file contents, not the extension. ZIP stays on miniz. TAR uses the same pinned-source and two-pass libarchive reader as complete extraction.

entry is the exact **raw name** exposed by `list()`. Lookup is case-sensitive and fails when the name is absent or occurs more than once. For TAR, GNU long names and pax path records are matched after libarchive has decoded the TAR headers, exactly as they appear in `list().entries[i].name`.

Only a regular-file entry can be selected. Sparse TAR files, symlinks, hard links, directories, and special types are refused when selected. The entry path is not reproduced: the content is written exactly to the caller-supplied destination path.

```
local result, err = babet.archive.extractFile(  
    "package.tar",  
    "assets/logo.bin",  
    "cache/current-logo.bin",  
    { overwrite = true }  
)
```

Result:

```
{  
    bytes = 4096,  
    path = "cache/current-logo.bin",  
    entry = "assets/logo.bin",  
}
```

Anti-bomb limits are still computed over the whole archive. For TAR, the first pass consumes and validates every data stream. The second pass compares every header with the inspected plan and consumes the archive to its end before the selected temporary file is published. Unselected regular-file data is discarded, and an unrelated unsafe path, special type, or sparse file does not prevent selecting a safe regular file. Malformed or truncated data anywhere in the archive still makes the operation fail without publishing the destination.

Path rules

For complete extraction, every entry name must satisfy all of these rules:

- between 1 and 4096 bytes;
- no NUL byte;
- no \;
- no leading /;
- no drive prefix such as C: in the first component;
- no empty component (a//b);
- no . or .. component;
- a trailing / only for an actual directory entry.

Paths remain relative to the destination. Babet never converts a Windows name into a Linux path and never silently “cleans” traversal: the archive is rejected. With `extractFile()`, the same rules apply to the selected entry name; unselected unsafe names do not become destination paths.

Duplicates are detected after removing the trailing / from directory names. An archive containing two identical outputs, or node as a file followed by node/child, is rejected before extraction.

Destination protection

Security does not rely only on string comparison. Babet walks the destination component by component through directory descriptors and refuses to follow symlinks:

- a symlink in the destination root path is rejected;
- a symlinked parent below the destination is rejected;

- a symlink final target is rejected, even with `overwrite = true`;
- a file may only replace a regular file;
- a directory entry may only use an existing or newly created directory.

Existing directories are accepted when they are real directories and remain unchanged. Babet does not alter their permissions.

The destination path itself is trusted script input and may be absolute or relative. The `..` rules above apply to untrusted names stored inside the archive.

Atomic publication and cleanup

`create()` writes the complete ZIP, plain TAR, gzip-compressed TAR, or xz-compressed TAR to a unique mode-0600 temporary file in the destination directory. Its short internal name does not repeat the output basename, so a valid destination close to the `NAME_MAX` limit remains creatable. After the miniz ZIP central directory or libarchive TAR stream is finalized, Babet flushes and `fsyncs` the file, changes it to 0644, publishes it atomically, and `fsyncs` the parent directory. Without `overwrite` it uses a no-clobber hard link publication; with `overwrite` it uses same-directory rename. Every error before publication removes the temporary and leaves the previous destination unchanged. If the final parent-directory `fsync` itself fails, the new archive has already been atomically published but the call returns an error because crash durability could not be confirmed.

Each extracted file is progressively written to a unique temporary file in its final directory with mode 0600. Temporary names are selected so they can never collide with an output declared by the archive. Writes are bounded by the announced size. For ZIP, miniz validates decompression and CRC. For TAR, libarchive provides streaming blocks and Babet validates their order, bounds, and final announced size. For gzip-wrapped TAR, Babet additionally validates every gzip member's CRC and ISIZE with zlib before publication. The temporary file then receives its final permissions.

All files of a complete extraction are prepared before publication starts. A read, decompression, or CRC error therefore publishes no file. `extractFile()` likewise keeps its selected file temporary until the complete archive has passed the verified read. Temporary files are removed, as are newly created empty directories **below** the destination. Destination-root path components that had to be created may remain present.

Publication is atomic **per file**:

- without `overwrite`, Babet uses creation that fails if the target exists;
- with `overwrite`, the same-directory temporary atomically replaces the target regular file.

The complete operation is not a multi-file transaction. A very late system error during publication or final directory permission changes may happen after previous files have been published. Those files may remain, and an old file already replaced is not restored.

For extraction, atomic publication prevents a partially visible file but does not promise crash durability: extracted files and directories are not fully `fsynced`. `create()` follows the stronger file-and-parent synchronization sequence described above.

Permissions

Defaults:

files	0644
final directories	0755
temporary files	0600
archive directories during preparation	0700
created destination-path components	0755 (subject to <code>umask</code>)

With `preserve_permissions = true`, Babet preserves only the nine ordinary owner/group/other bits (0777) from Unix modes announced by the ZIP or TAR. `setuid`, `setgid`, and sticky bits are always stripped.

Permissions of a newly created directory are applied only after its contents have been extracted, so a restrictive final mode such as 0000 cannot block the current operation. Existing directories are never chmodded.

UIDs, GIDs, ACLs, extended attributes, and timestamps are not restored.

Supported entry types

Type	ZIP/TAR create()	ZIP/TAR list()	ZIP/TAR extract()	ZIP/TAR extractFile()
regular file	supported; ZIP/gzip/xz accept levels 0 through 9, bzip2 accepts 1 through 9, zstd accepts 0 through 19, plain TAR stores raw data	inspected in ZIP and plain/gzip/xz/bzip2/ TAR	supported in ZIP and plain/gzip/xz/bzip2/ TAR, except sparse TAR	supported in ZIP and plain/gzip/xz/bzip2/zstd TAR, except sparse TAR
directory	supported when include_directories = true	inspected in ZIP and TAR	supported in ZIP and TAR	cannot be selected
sparse TAR file	never created	identified in TAR	rejected	rejected when selected; ignored when unrelated
symlink	always rejected	identified in ZIP and TAR	always rejected	rejected
hard link	not created	identified in TAR	rejected	rejected when selected
FIFO, socket, device, or unknown filesystem type	always rejected	identified when the format exposes it	rejected	rejected
encrypted entry	not created	identified in ZIP	rejected	rejected
unknown ZIP compression method	not created	identified	rejected	rejected

A ZIP entry that resembles a hard link but is encoded as a regular file is treated as an independent regular file; no link is created.

Errors and known limitations

Common failures include:

- missing or unsafe source directory, source symlink, non-UTF-8 source name, unsupported source object, invalid creation format, ZIP-only option used for TAR, or out-of-range ZIP timestamp;
- output archive inside the source, unsafe destination parent, or destination symlink;
- missing, unreadable, non-regular, unsupported, or malformed archive file;
- truncated TAR data, non-TAR trailing data, a non-regular TAR entry carrying a payload, or a change detected between inspection and extraction;
- inconsistent metadata or invalid CRC;
- exceeded anti-bomb limit;

- absolute path, traversal, duplicate, or path conflict;
- encrypted entry, symlink, or unsupported type;
- existing destination without `overwrite = true`;
- symlink or unsafe type in the destination;
- insufficient disk space, permissions, or another I/O error.

The module is synchronous: a call returns only after creation, inspection, or extraction has completed. It does not expose progress, cancellation, or a timeout yet.

The archive file must not be modified concurrently. ZIP truncation or modification will normally cause a read, decompression, or CRC error. TAR is streamed through libarchive; concatenated TAR streams are scanned to their end, and truncated or non-TAR trailing data is rejected. Neither back-end provides a concurrent snapshot. The destination must not be actively reorganised by another process either. Traversal rejects symlinks and rechecks types before publication, but an actor that continuously renames directories can make the operation fail and turn temporary-file cleanup into a best-effort action.

Plain, gzip-, xz-, bzip2-, and zstd-compressed TAR creation is supported. Standalone compression streams remain rejected until their API and safety policy are defined and audited separately.

Multi-disk archives are not a supported target. ZIP64 is accepted when miniz can read it, while still being subject to the configured limits.

argparse — command-line arguments

argparse is a pure-Lua module bundled with Babet. It builds a declarative parser for flags, valued options, and positional arguments:

```
local argparse = require("argparse")
local parser = argparse("my-script", "Optional description")
```

The module never calls `print` or `os.exit`. Help and errors are returned to the script, which decides how to display them and which exit code to use.

Module contents

- [Loading and metadata](#)
- [API](#)
- [Declaring flags and options](#)
- [Declaring positionals](#)
- [The `opts` table](#)
 - [Common fields](#)
 - [Value-only fields](#)
 - [Destination](#)
- [Argument source](#)
 - [Global `arg` table](#)
 - [Explicit array](#)
- [Recognized forms](#)
- [Defaults, choices, and conversion](#)
- [parse results](#)
 - [Normal success](#)
 - [Built-in help](#)
 - [User-input error](#)
 - [Programming error](#)
- [Complete example](#)
- [Text produced by `get\_usage`](#)
- [Current limitations](#)

Loading and metadata

```
local argparse = require("argparse")
```

```
print(argparse._VERSION)      -- "babet argparse 1.1.0"
print(argparse._DESCRIPTION)  -- English description
```

`require("argparse")` returns a callable table. The constructor accepts zero, one, or two arguments:

```
argparse()                -- displayed program: "prog"
argparse("tool")          -- program name
argparse("tool", "Description") -- name and description
```

`prog` and `description` must be strings when supplied. An extra argument or a wrong type raises a Lua error: this is a programming error, not a command-line error.

API

Method	Result	Purpose
<code>parser:flag(spec [, opts])</code>	<code>parser</code>	Declare a flag with no value.
<code>parser:option(spec [, opts])</code>	<code>parser</code>	Declare an option that consumes a value.
<code>parser:argument(name [, opts])</code>	<code>parser</code>	Declare a positional argument.
<code>parser:parse([src])</code>	<code>res, err</code>	Parse <code>src</code> or the global arg table.
<code>parser:get_usage()</code>	string	Generate the help text.

Builder methods are chainable:

```
local parser = argparse("copy")
  :flag("-v --verbose")
  :option("-o --output")
  :argument("source")
```

Arities are strict. For example, `parser:option("-o", {}, "extra")` and `parser:get_usage(true)` raise a Lua error.

Declaring flags and options

`spec` is one string containing one or more whitespace-separated names:

```
parser:flag("-v")
parser:flag("--verbose")
parser:flag("-v --verbose")
parser:option("-o --output")
```

Each name must:

- start with one or two dashes;
- contain at least one character after the dashes;
- contain no `=`;
- not be duplicated in the same `spec` or `parser`.

The following names are reserved and cannot be declared:

- `-`: treated as a literal positional;
- `--`: ends option parsing;
- `-h` and `--help`: built-in help.

A triple-dash name such as `---verbose` is rejected as well.

All validation happens before the parser is modified. If a builder error is caught with `pcall`, no partial option or alias remains registered.

Declaring positionals

```
parser:argument("input")
parser:argument("output", { required = false })
```

The name must be a non-empty string with no whitespace and no leading dash. Positionals are assigned strictly from left to right.

A positional is required by default. It becomes optional when:

- `required = false` is specified; or
- a default is supplied without an explicit `required` field.

All required positionals must precede optional ones. The builder therefore rejects this ambiguous declaration:

```
parser
  :argument("optional", { required = false })
  :argument("required") -- Lua error
```

The opts table

`opts` must be a table or `nil`. Unknown fields and fields with a wrong type raise a Lua error immediately. This catches a typo such as `hlep` instead of `help`.

Common fields

Field	Type	Effect
<code>help</code>	string	Text appended by <code>get_usage()</code> .
<code>default</code>	any Lua value	Value used when the item is absent.
<code>dest</code>	non-empty string	Key used in the result table.
<code>required</code>	boolean	Require the item to be present.

These four fields are accepted by `flag`, `option`, and `argument`.

Value-only fields

Field	Type	Effect
<code>choices</code>	dense array of strings	Allow only selected raw values.
<code>convert</code>	function	Convert the raw string to a Lua value.

`choices` and `convert` are accepted by `option` and `argument`, but not by `flag`, since a flag consumes no value.

The builder copies the `choices` array. Mutating the original table later does not change the parser.

Destination

Without `dest`, the destination is:

1. the first long name without `--`, when present;
2. otherwise the first short name without its first `-`.

```
parser:option("-o --output") -- destination: "output"  
parser:flag("-v")           -- destination: "v"
```

A destination must be unique across the parser. `help` and `usage` are reserved for the built-in help result.

```
parser:flag("-q", { dest = "quiet" })
```

Argument source

Global arg table

With no argument, `parse()` reads the global arg table at indices 1, 2, and so on up to the first hole:

```
local result, err = parser:parse()
```

Index 0 and negative indices are ignored. `parse(nil)` is equivalent to `parse()`.

Every encountered value must be a string. If `arg` is not a table, the parser uses an empty argument list.

Explicit array

For tests or programmatically built input:

```
local result, err = parser:parse({ "-v", "input.txt" })
```

The table must be a dense array indexed from 1 to `n`, with no holes, no extra keys, and string values only. An invalid source is a parsing error and returns `(nil, err)`; it does not raise.

Recognized forms

A valued option accepts:

```
--long value  
--long=value  
-s value  
-s=value
```

An empty inline value is valid:

```
parser:parse({ "--output=" }) -- output == ""
```

Options remain active after a positional until `--` is encountered:

```
input.txt --verbose
```

`--` ends option parsing. Every following token becomes positional, even if it starts with `-`:

```
parser:parse({ "--", "-5" })
```

Before `--`, a token such as `-5` is interpreted as an option name. A negative number used as an option value needs no delimiter:

```
parser:parse({ "--count", "-5", "input.txt" })
```

The token following a valued option is always consumed literally, even when it looks like an option or equals `--`:

```
parser:parse({ "--output", "--verbose" })
-- output == "--verbose"; verbose is not enabled
```

A lone - is positional. Repeated options are accepted and the last value wins. A repeated flag simply remains true.

The following forms are not supported:

- combined short options, such as -abc for three flags;
- attached short values, such as -ofile.txt;
- variadic positionals.

A literal declared name -abc is still a normal option when the spec contains that exact name; it is never split.

Defaults, choices, and conversion

A user-supplied value is processed in this order:

1. check choices against the raw string;
2. call convert with that string.

```
parser:option("-n --count", {
    choices = { "1", "2", "3" },
    convert = tonumber,
})
```

Every choice is a string. If the raw value is absent from the array, parse() returns (nil, err) without calling the converter.

A converter may:

- return any non-nil value, including false, to succeed;
- return nil to fail with the generic invalid value message;
- return nil, "reason" to supply a reason;
- raise, which is caught and reported as conversion error.

default values are already prepared Lua values: they are neither checked by choices nor passed through convert. They keep their type and, for a table, their identity. An option with required = true remains required even when a default exists.

For a flag:

- when absent, it equals default if supplied, otherwise false;
- when present, it always equals true.

An absent option or positional with a nil default has no observable key in the result table, as expected for Lua tables.

parse results

Normal success

```
local result, err = parser:parse()
-- result: table
-- err:    nil
```

The table is indexed by declared destinations.

Built-in help

Before --, -h or --help stops parsing immediately and returns:

```
{
    help = true,
    usage = "...",
}, nil
```

Required options, missing positionals, and later tokens are not validated. After --, -h and --help are ordinary positionals.

Help is a success and is not merged into a normal result table. Destinations help and usage are reserved to avoid ambiguity in calling code.

User-input error

Command-line errors return (nil, message):

```
unknown option '--bad'
option '--output' requires a value
flag '-v' does not take a value
missing required option '--output'
missing required argument 'input'
argument 'mode': invalid choice 'other'
option '--count': invalid value
unexpected argument 'extra'
```

parse() catches errors raised by convert, so a user value never propagates the converter's exception.

Programming error

The constructor and builder raise a Lua error for:

- wrong arity;
- wrong type;
- an empty or unusable spec;
- a reserved name or destination;
- a duplicate name or destination;
- an unknown opts field;
- an invalid choices array;
- a required positional after an optional positional.

Wrong parse arity is also a programming error. In contrast, a malformed src table returns (nil, err).

Complete example

```
local argparse = require("argparse")

local parser = argparse("convert", "Convert a file.")
:flag("-v --verbose", { help = "Show more detail" })
:option("-o --output", { default = "out.txt" })
:option("-n --count", {
    choices = { "1", "2", "3" },
    convert = tonumber,
    default = 1,
})
:argument("input")

local args, err = parser:parse()
if args and args.help then
```

```

    print(args.usage)
    return
end
if not args then
    io.stderr:write("error: ", err, "\n")
    io.stderr:write(parser:get_usage(), "\n")
    os.exit(2)
end

print(args.input, args.output, args.count, args.verbose)

```

Text produced by `get_usage`

`get_usage()` returns a string and writes nothing. The section labels and the built-in help message are fixed in English: Usage, Arguments, Options, and show this help. There is no localization, column-width, or visual-group API yet.

The text lists positionals and declared help strings, but it does not automatically show defaults, choices, or the required status of options.

Current limitations

The module does not expose:

- subcommands;
- variadic positionals (`nargs`);
- mutually exclusive groups;
- repeatable options accumulated into an array;
- combined short options;
- help localization or customization;
- a way to disable built-in help.

EXEC - external programs, capture, and streaming

Babet provides two complementary APIs for starting an external program without an implicit shell:

- `babet.exec` optionally sends complete input, captures stdout and stderr in memory, then waits for completion;
- `babet.spawn` immediately returns a controllable process object so streams can be read and written progressively without automatic RAM accumulation.

EXEC covers:

- looking up a program through `PATH`;
- building `argv` safely, without shell parsing;
- selecting a working directory for the child process;
- merging a custom environment with Babet's environment;
- sending a binary string to standard input;
- capturing binary stdout and stderr;
- applying one timeout to launch, I/O, and final wait;
- enforcing a separate memory limit for each output stream;
- terminating the child's process group after a timeout.

It does not provide direct file redirection or an implicit command interpreter. To connect several programs without a shell, use `babet.pipeline()` or `babet.spawnPipeline()`.

Module contents

- Essential conventions
- API overview
- Signature and arguments
 - cmd and program lookup
 - args and shell-free execution
 - Using a shell explicitly
- Options
 - cwd
 - env
 - stdin
 - timeout
 - max_output
- Result table
 - Normal exit code
 - Signal termination
 - Timeout
 - Truncation
- Complete examples
 - Basic capture
 - Arguments containing spaces
 - Handling a non-zero code
 - Capturing stderr
 - Sending text or binary stdin
 - Changing the environment
 - Changing the working directory
 - Bounding execution time
 - Bounding captured output
 - Building an application helper
- Streaming processes with `babet.spawn`
 - Signature and options
 - Process methods
 - Reading stdout and stderr
 - Writing stdin
 - Waiting, terminating, and closing
 - Streaming loop example
 - Limits and deadlock prevention
- Error contract
- Processes, signals, and security
- Design and limitations

Essential conventions

No implicit shell

This call:

```
local result, err = babet.exec("printf", { "%s\n", "hello world" })
```

starts `printf` directly. Every `args` element becomes a separate `argv` slot. Spaces, `$`, `*`, `;`, `>`, quotes, and other shell characters are not interpreted.

This avoids most quoting mistakes and command-injection issues.

A non-zero exit is not a launch error

A program that starts successfully and exits with code 1, 2, or 127 still produces a result table. The caller decides which codes are acceptable.

```

local result, err = babet.exec("grep", { "needle", "data.txt" })
assert(result, err)

if result.code == 0 then
    print("match found")
elseif result.code == 1 then
    print("no match")
else
    error(result.stderr)
end

```

assert(babet.exec(...)) only checks that Babet could launch and monitor the program. It does not check result.code.

Data strings are binary-safe

opts.stdin, result.stdout, and result.stderr are binary-safe Lua strings. They may contain NUL bytes and do not need to be UTF-8 text.

Strings passed to POSIX APIs that require C strings - cmd, args elements, cwd, and environment names and values - cannot contain a NUL byte.

API overview

```

local result, err = babet.exec(cmd, args?, opts?)

```

Element	Type	Default	Purpose
cmd	strict string	required	program name or path
args	string sequence	no arguments	argv[1]..argv[n]
opts.cwd	string	inherited current directory	child working directory
opts.env	string-to-string table	inherited environment	variables to add or replace
opts.stdin	binary string	stdin closed immediately	data sent to the child
opts.timeout	finite number > 0	no limit	global budget in seconds
opts.max_output	integer 1..2 GiB	10 MiB	cap per captured stream

Main outcomes:

Situation	Return value
program launched and completed	(result, nil)
program launched then timed out	(result, nil) with timed_out = true
non-zero exit status	(result, nil) with code ~= 0
program not found or invalid cwd	(nil, err)
invalid args or opts	(nil, err)
missing or non-string cmd	raised Lua error

Signature and arguments

cmd and program lookup

cmd must be an actual Lua string.

```
local result, err = babet.exec("git", { "status", "--short" })
```

When cmd contains no /, Babet asks the C library to search for it through the Babet process's PATH.

```
local result = assert(babet.exec("python3", { "--version" }))
```

When cmd contains /, it is treated as a direct path.

```
local result = assert(babet.exec("/usr/bin/id", { "-u" }))
```

A relative path containing / is resolved in the child's directory after opts.cwd has been applied.

```
local result = assert(babet.exec("./tool", {}, {  
    cwd = "/opt/my-app/bin",  
}))
```

PATH and opts.env nuance The child may receive a replacement PATH through opts.env. However, the current Linux implementation uses execvp, whose initial lookup of cmd uses the Babet process's PATH, not opts.env.PATH.

```
local result, err = babet.exec("my-private-tool", {}, {  
    env = { PATH = "/opt/private/bin" },  
})
```

This does not guarantee that /opt/private/bin/my-private-tool will be found. Use a direct path:

```
local result, err = babet.exec("/opt/private/bin/my-private-tool", {}, {  
    env = { PATH = "/opt/private/bin" },  
})
```

The replacement PATH is still visible **inside** the child and to commands it starts later.

args and shell-free execution

args is optional. It must be a dense 1..n sequence of strings.

```
local result = assert(babet.exec("cp", {  
    "--",  
    "file with spaces.txt",  
    "destination/",  
}))
```

Babet reads only the Lua sequence part, from 1 through #args. For predictable behavior:

- use consecutive indexes starting at 1;
- do not leave holes;
- do not store metadata in the same table;
- use strings only.

-- Correct

```
local args = { "-c", "print('ok')" }
```

-- Invalid: numeric element

```
local result, err = babet.exec("lua", { "-e", 42 })
```

-- result == nil; err says all arguments must be strings

argv[0] is set to cmd automatically. The first args element becomes argv[1].

Using a shell explicitly

Operators such as |, >, &&, ;, variables such as \$NAME, and globs are only interpreted when you explicitly start a shell.

```
local result = assert(babet.exec("sh", {
    "-c",
    "printf '%s\n' \"\$HOME\" | sed 's/^#home=#'",
}))
```

Values embedded in shell source must then follow shell quoting rules. For untrusted data, pass values through args, stdin, or env instead of concatenating them into the -c script.

Options

The third argument is optional. When present, it must be a table. Unknown fields are currently ignored; a typo such as `timeot = 5` therefore does not raise an error. Use the exact names below.

opts.cwd

Sets the child's current directory without changing the Babet process's current directory.

```
local result, err = babet.exec("pwd", {}, {
    cwd = "/var/tmp",
})
assert(result, err)
print(result.stdout) -- /var/tmp
```

The child performs `chdir` before launching the program. It affects:

- relative paths used by the program;
- a relative cmd containing `/`;
- relative entries in the `PATH` used for lookup.

If the directory does not exist or is inaccessible, Babet returns `(nil, err)`. The parent current directory is unchanged.

opts.env

env is a table whose keys and values must be strings. It is **merged** with the Babet process's environment.

```
local result = assert(babet.exec("sh", {
    "-c", "printf '%s' \"\$APP_MODE\"",
}, {
    env = { APP_MODE = "production" },
}))

assert(result.stdout == "production")
```

Rules:

- a supplied key replaces the inherited variable of the same name;
- an omitted key remains inherited;
- `""` creates or keeps a defined-but-empty variable;
- there is no option to remove a variable completely;
- a key must be non-empty and cannot contain `=`;
- keys and values reject NUL bytes;
- non-string keys and values are rejected.

Defined-but-empty is different from absent:

```
local result = assert(babet.exec("sh", { "-c", [[
    if [ "${TOKEN+x}" = x ] && [ -z "$TOKEN" ]; then
        printf 'defined-empty'
    fi
]]
```

```

]] }, {
    env = { TOKEN = "" },
}))

```

```
assert(result.stdout == "defined-empty")
```

The environment is built in the parent before fork, avoiding setenv calls in a child of a multithreaded process.

opts.stdin

stdin is one string sent completely to the program's standard input.

```

local result = assert(babet.exec("sha256sum", { "-" }, {
    stdin = "hello world\n",
}))
print(result.stdout)

```

It is binary-safe:

```

local payload = "A\0B\0C"
local result = assert(babet.exec("cat", {}, { stdin = payload }))
assert(result.stdout == payload)

```

When stdin is absent or nil, Babet closes the input pipe immediately, so the child sees EOF. An empty string also produces EOF after the normal pipe setup.

Babet writes stdin while reading both output pipes, preventing deadlocks with a program that produces substantial output before consuming all its input.

opts.timeout

timeout is a finite, strictly positive number of seconds.

```

local result = assert(babet.exec("sleep", { "30" }, {
    timeout = 0.5,
}))

```

```
assert(result.timed_out)
```

The budget starts before launch preparation and covers:

1. environment preparation;
2. fork;
3. the child chdir;
4. the exec call;
5. stdin transfer;
6. stdout and stderr capture;
7. the final process wait.

Internal resolution is one millisecond. Values below 0.001 therefore act as an almost immediate deadline and should not be used for precise timing.

The value must fit within INT_MAX milliseconds, about 24.8 days. Zero, negative, NaN, infinite, and larger values return (nil, err).

After expiry once the child has launched:

1. Babet sends SIGTERM to the process group;
2. it allows up to 2 seconds for a clean exit;
3. it sends SIGKILL if needed;
4. it continues draining output for a bounded window.

opts.max_output

max_output limits retained bytes **for each stream**.

```
local result = assert(babet.exec("yes", {}, {
    timeout = 0.2,
    max_output = 4096,
}))
```

```
assert(#result.stdout == 4096)
assert(result.stdout_truncated)
```

Values:

- default: 10 * 1024 * 1024, or 10 MiB;
- minimum: 1 byte;
- maximum: 2 * 1024 * 1024 * 1024, or 2 GiB;
- type: integral Lua number.

The cap is independent: up to 10 MiB of stdout **and** 10 MiB of stderr by default.

After reaching the cap, Babet keeps reading and discarding excess bytes. This is necessary to avoid filling the pipe and blocking the child. The first bytes are retained; the final bytes are lost.

Result table

After a successful launch, the first return value is always a table with six fields:

```
{
    stdout = "...",
    stderr = "...",
    code = 0,
    timed_out = false,
    stdout_truncated = false,
    stderr_truncated = false,
}
```

Field	Type	Meaning
stdout	binary string	first retained bytes from standard output
stderr	binary string	first retained bytes from standard error
code	integer	normal code, signal convention, or -1 if unavailable
timed_out	boolean	Babet initiated termination after deadline expiry
stdout_truncated	boolean	stdout exceeded max_output
stderr_truncated	boolean	stderr exceeded max_output

Normal exit code

When the program calls `exit(n)` or returns from `main`, code is the POSIX status 0..255.

```
local result = assert(babet.exec("sh", { "-c", "exit 7" }))
assert(result.code == 7)
assert(not result.timed_out)
```

Signal termination

When the process is terminated by a signal, Babet uses:

```
code = 128 + signal_number
```

For example, SIGKILL is normally 9 on Linux, giving `code == 137`. This is a useful convention but there is no separate result field containing the signal.

Timeout

`timed_out == true` means Babet reached its deadline and started termination. `code` depends on how the child eventually stopped:

- an application-defined code if it handles SIGTERM and exits cleanly;
- often 143 for SIGTERM;
- often 137 after SIGKILL;
- exceptionally -1 if status could not be obtained within bounded cleanup.

Do not test only `code == 137` to detect a timeout; use `timed_out`.

Output produced before expiry is retained up to `max_output`.

Truncation

`stdout_truncated` and `stderr_truncated` only mean that the corresponding stream exceeded `max_output`.

```
if result.stdout_truncated then
    print("stdout is incomplete: increase max_output or redirect to a file")
end
```

In an extremely pathological kernel/process-group case, a descendant may keep a pipe open after SIGKILL and force Babet to abandon draining rather than block forever. This rare cause is not represented by the truncation flags, which are reserved for the memory cap.

Complete examples

Basic capture

```
local result, err = babet.exec("git", { "rev-parse", "HEAD" })
assert(result, err)

if result.code ~= 0 then
    error("git failed: " .. result.stderr)
end

local commit = result.stdout:gsub("%S+$", "")
print(commit)
```

Arguments containing spaces

```
local filename = "July 2026 report.txt"
local result = assert(babet.exec("printf", {
    "name=<%s>\n",
    filename,
}))

assert(result.stdout == "name=<July 2026 report.txt>\n")
```

No additional quoting is needed around filename: the Lua table already represents the argument boundary.

Handling a non-zero code

```
local result, err = babet.exec("diff", {
    "--brief",
    "config.old",
    "config.new",
})
assert(result, err)

if result.code == 0 then
    print("files are identical")
elseif result.code == 1 then
    print("files differ")
else
    error("diff failed: " .. result.stderr)
end
```

Capturing stderr

```
local result = assert(babet.exec("sh", {
    "-c", "printf out; printf err >&2",
}))

assert(result.stdout == "out")
assert(result.stderr == "err")
```

Sending text or binary stdin

```
local csv = "name,score\nAlice,18\nBob,15\n"
local result = assert(babet.exec("sort", {}, { stdin = csv }))
print(result.stdout)

local bytes = string.char(0x00, 0x01, 0xFE, 0xFF)
local result = assert(babet.exec("cat", {}, { stdin = bytes }))
assert(result.stdout == bytes)
```

Changing the environment

```
local result = assert(babet.exec("sh", { "-c", [[
    printf 'mode=%s lang=%s' "$APP_MODE" "$LANG"
]] }, {
    env = {
        APP_MODE = "test",
        LANG = "C",
    },
}))

print(result.stdout)
```

Only supplied keys are replaced. Other variables, including HOME, remain inherited.

Changing the working directory

```
local result = assert(babet.exec("find", {
    ".", "-maxdepth", "1", "-type", "f",
```

```
}, {
    cwd = "/var/log",
}))
```

```
print(result.stdout)
```

The main script's current directory is unchanged.

Bounding execution time

```
local result, err = babet.exec("sh", {
    "-c", "printf started; sleep 30",
}, {
    timeout = 1.0,
})
assert(result, err)

if result.timed_out then
    print("command exceeded its budget")
    print("available output:", result.stdout)
end
```

Bounding captured output

```
local result = assert(babet.exec("sh", {
    "-c", "yes log-line | head -n 100000",
}, {
    max_output = 64 * 1024,
}))

if result.stdout_truncated then
    print("only the first 64 KiB were retained")
end
```

For gigabytes, do not capture into memory. Explicitly start a shell and redirect to a carefully selected file:

```
local result = assert(babet.exec("sh", {
    "-c", "my-command > /var/tmp/my-command.log 2>&1",
}, {
    timeout = 600,
}))
```

Building an application helper

```
local function run_checked(cmd, args, opts)
    local result, err = babet.exec(cmd, args, opts)
    if not result then
        return nil, err
    end
    if result.timed_out then
        return nil, string.format("%s: timeout", cmd)
    end
    if result.code ~= 0 then
        local detail = result.stderr ~= "" and result.stderr or result.stdout
        return nil, string.format(
            "%s: exit code %d: %s",
            cmd,

```

```

        result.code,
        detail
    )
end
return result
end

local result, err = run_checked("git", { "status", "--short" }, {
    cwd = "/srv/project",
    timeout = 10,
})
assert(result, err)
print(result.stdout)

```

Streaming processes with `babot.spawn`

`babot.spawn` uses the same secure launch engine as `babot.exec`, but it does not capture streams automatically and does not wait for completion. It returns a userdata representing the child process:

```

local process, err = babot.spawn("yt-dlp", {
    "--newline",
    "https://example.invalid/video",
})
assert(process, err)

```

The process owns three non-blocking pipes for stdin, stdout, and stderr. The script decides when to read, write, wait, or terminate the process group. This API is intended for long-running programs, large outputs, and real-time progress display.

Signature and options

```

local process, err = babot.spawn(command, args?, opts?)

```

Element	Type	Default	Purpose
command	strict string	required	program name or path
args	dense string array	no arguments	argv[1]..argv[n]
opts.cwd	string	inherited directory	child working directory
opts.env	string-to-string table	inherited environment	variables to add/replace
opts.launch_timeout	finite number > 0	unlimited wait	bounds chdir + exec launch

Unlike `babot.exec`, unknown options are rejected. `spawn` does not accept `stdin`, `timeout`, or `max_output`: `stdin` is sent through `process:write()`, lifetime is controlled with `wait()/terminate()`, and outputs are read progressively.

`launch_timeout` only covers the launch phase before the new executable is established. It is not a total process-lifetime limit.

PATH lookup, paths containing `/`, environment merging, and shell-free execution follow the same rules as `exec`.

Process methods

Method	Return	Purpose
<code>process:read_stdout([max_bytes(data, nil) or (nil, reason) [, timeout]])</code>		read at most max_bytes from stdout
<code>process:read_stderr([max_bytes(data, nil) or (nil, reason) [, timeout]])</code>		read at most max_bytes from stderr
<code>process:write(data [, timeout])</code>	(bytes_written, nil) or (nil, reason)	write part of data to stdin
<code>process:close_stdin()</code>	(true, nil)	close stdin idempotently
<code>process:is_running()</code>	boolean or (nil, err)	check state without blocking
<code>process:pid()</code>	integer	return the assigned PID
<code>process:wait([timeout])</code>	(result, nil) or (nil, reason)	wait without killing on timeout
<code>process:terminate([grace_period])</code>	(result, nil) or (nil, err)	SIGTERM, then SIGKILL if needed
<code>process:kill()</code>	(result, nil) or (nil, err)	SIGKILL the process group
<code>process:close()</code>	(true, nil)	clean pipes and any active child

Short reasons "timeout", "closed", and "interrupted" are typed, like socket errors. Other diagnostics use a process: or spawn: prefix.

The table returned by wait, terminate, and kill contains:

```
{
  code = 0,          -- normal code, or 128 + signal
  exited = true,    -- normal exit/_exit termination
  signaled = false,  -- terminated by a signal
  signal = nil,      -- signal number when signaled == true
}
```

Calling wait() again after completion returns the same result because the status is cached after the first waitpid.

Reading stdout and stderr

Reads are binary-safe. By default they read at most 64 KiB and are non-blocking (timeout = 0). The maximum size per call is 16 MiB.

```
local chunk, err = process:read_stdout(64 * 1024, 0.25)

if chunk then
  io.write(chunk)
elseif err == "timeout" then
  -- nothing became available for 250 ms
elseif err == "closed" then
  -- final EOF on stdout
else
  error(err)
end
```

"closed" is EOF for that stream, not necessarily process completion. A child may close stdout and keep working.

Drain both stdout **and** stderr. Reading only one while the other fills can block the child when the kernel pipe becomes full.

Writing stdin

`write` accepts a binary Lua string, including NUL bytes. The pipe is non-blocking, so the call may write only part of the string even after it became writable. Use the returned byte count to resume at the next byte.

```
local function write_all(process, data)
    local offset = 1
    while offset <= #data do
        local written, err = process:write(data:sub(offset), 1)
        assert(written, err)
        offset = offset + written
    end
end
```

```
write_all(process, payload)
assert(process:close_stdin())
```

An empty string returns (0, nil). After `close_stdin`, `write` returns (nil, "closed"). Closing stdin is often required for tools such as `cat`, `sort`, or `ffmpeg` to know the input is complete.

Waiting, terminating, and closing

`wait()` without an argument waits for process completion. `wait(timeout)` returns (nil, "timeout") when the budget expires without sending a signal or invalidating the object.

```
local result, err = process:wait(0)
if not result and err == "timeout" then
    -- process is still running
end
```

`terminate(grace_period)` sends SIGTERM to the whole group. If the group does not exit before the grace period ends, Babet sends SIGKILL. The default grace period is two seconds. `kill()` sends SIGKILL directly.

`close()` is idempotent. If the child is still running, it is terminated and reaped with bounded waits, then all three pipes are closed. The garbage collector and Lua `__close` metamethod perform the same cleanup:

```
local process <close> = assert(babet.spawn("long-job"))
-- automatic cleanup when leaving the scope
```

Explicit close is still recommended because garbage-collection timing is not deterministic.

Streaming loop example

```
local process, err = babet.spawn("yt-dlp", {
    "--newline",
    "-f", "bestvideo+bestaudio/best",
    url,
})
assert(process, err)

local stdout_open, stderr_open = true, true

while stdout_open or stderr_open do
    if stdout_open then
        local data, read_err = process:read_stdout(64 * 1024, 0.1)
        if data then
            io.write(data)
        end
    end
end
```

```

        io.flush()
    elseif read_err == "closed" then
        stdout_open = false
    elseif read_err ~= "timeout" then
        error(read_err)
    end
end

if stderr_open then
    local data, read_err = process:read_stderr(64 * 1024, 0)
    if data then
        io.stderr:write(data)
        io.stderr:flush()
    elseif read_err == "closed" then
        stderr_open = false
    elseif read_err ~= "timeout" then
        error(read_err)
    end
end
end

local result, wait_err = process:wait(5)
assert(result, wait_err)
process:close()

if result.code ~= 0 then
    error("yt-dlp exited with code " .. result.code)
end

```

Limits and deadlock prevention

- `wait()` does not drain output. Calling it immediately on a very verbose child can block when its pipes fill. Drain both streams while it runs, or use `babey.exec` for small automatically captured output.
- Sending large input to a program that simultaneously emits large output requires alternating writes and reads. A single `write_all` before any read can fill pipes in both directions.
- A `babey.spawn()` object cannot be connected after launch to another process object. For a linear pipeline, use `babey.spawnPipeline()` directly.
- `stdout` and `stderr` remain separate. Babey does not merge them.
- `close()` and `terminate()` target the process group created at launch. A descendant that deliberately changes process group or session may escape, just as with `babey.exec`.

Error contract

Raised Lua errors

Only the first argument's signature raises:

```

babey.exec()    -- raises: missing command
babey.exec(42) -- raises: cmd must be a string

```

Use `pcall` only when an invalid signature can come from uncontrolled data.

Returned (nil, err) errors

The following validations return (nil, err) rather than raising:

- `args` is neither a table nor nil;

- an args element is not a string;
- opts is neither a table nor nil;
- cwd, stdin, timeout, max_output, or env has the wrong type;
- an environment key is empty, contains =, or contains NUL;
- an environment value contains NUL;
- timeout is zero, negative, non-finite, or too large;
- max_output is zero, negative, fractional, or above 2 GiB;
- program not found or not executable;
- invalid cwd;
- pipe, fork, poll, waitpid, or another internal system failure.

```
local result, err = babet.exec("echo", "hello")
assert(result == nil)
print(err) -- args must be a table
```

Situations returning a table

These are not API errors:

- non-zero program exit;
- output written to stderr;
- timeout during or after launch;
- output exceeding max_output.

Processes, signals, and security

Process group

The child creates its own process group. On timeout, Babet signals the group, not only the direct PID. This normally covers descendants started by a shell or by the program.

A descendant that deliberately calls `setuid` or changes process group can escape this strategy. `babet.exec` is not a sandbox and does not replace `systemd`, `cgroups`, `namespaces`, or resource limits.

Interaction with `babet.signal`

`babet.exec` is not signal-aware. A handled signal received while the call is running does not return `(nil, "interrupted")`. The C handler records a pending flag; the Lua callback can only be dispatched after control returns to the Lua VM, normally through `SIGNAL`'s instruction hook.

Use `opts.timeout` to bound a long-running command. For externally requested cancellation, design another protocol or run the command under a supervisor.

Memory

The script already holds `stdin` in memory, and `stdout/stderr` are captured in memory up to the selected limits. The defaults permit about 20 MiB of captured output in total, in addition to application and Lua-string allocations.

Design and limitations

- Linux/POSIX only: the implementation uses `fork`, `execvp`, `poll`, process groups, and signals.
- No implicit shell.
- One `exec` call still represents one command; native pipelines use `babet.pipeline()` or `babet.spawnPipeline()`.
- One complete `stdin` string, no streaming producer.
- Complete capture returned at the end, no line callback.
- No native `stdin/stdout/stderr` redirection to files or descriptors.

- No separate signal field in the result table.
- No full environment replacement or unset operation: env merges and an empty string remains a defined variable.
- Unknown opts fields are ignored.
- Initial PATH lookup currently follows the Babet process environment even when opts.env.PATH is replaced.
- babet.signal callbacks are not dispatched during the call.

See [Security](#) for the general security model.

Process pipelines

Babet provides two complementary APIs for connecting programs through POSIX pipes without invoking a shell:

- babet.pipeline() runs the whole pipeline and captures its streams;
- babet.spawnPipeline() immediately returns a streaming process object.

In both cases, each stage's stdout is connected directly to the next stage's stdin. The parent only owns the first stage's stdin, the last stage's stdout, and one separate stderr stream per stage.

Shared command syntax

```
local commands = {
  { "printf", { "orange\napple\norange\n" } },
  { "grep", { "orange" } },
  { "sort", { "-u" } },
}
```

commands must be a dense array containing 2 to 32 stages. Each stage has one of these forms:

```
{ command }
{ command, args }
{ command, args, opts }
```

To provide opts without arguments, explicitly use { command, {}, opts }; sparse arrays are rejected.

- command is a non-empty string without a NUL byte;
- args is a dense array of strings without NUL bytes;
- opts.cwd sets this stage's working directory;
- opts.env adds or replaces environment variables for this stage.

No string is parsed by a shell: globbing, redirection, variables, quoting and operators such as |, && or > have no special meaning.

Global cwd and env options provide defaults. A local cwd replaces the global one. Local environment entries are merged last and win on duplicate names. Every command is validated before the first fork().

As with exec() and spawn(), when a command contains no /, its initial lookup uses the Babet process's PATH, not a PATH replaced through opts.env or a stage-local environment. To launch a tool available only through that new PATH, use its explicit path or change Babet's PATH before launching.

babet.pipeline() — complete execution

Signature

```
result, err = babet.pipeline(commands [, opts])
```

Example:

```

local result, err = babet.pipeline({
  { "printf", { "orange\napple\norange\n" } },
  { "grep", { "orange" } },
  { "sort", { "-u" } },
}, {
  timeout = 10,
  max_output = 16 * 1024 * 1024,
})

assert(result, err)
print(result.stdout)

```

Options

Option	Type	Description
cwd	string	Default working directory for every stage.
env	string-to-string table	Variables merged with Babet's environment.
stdin	binary string	Data progressively written to the first stage, then stdin is closed.
timeout	finite positive number, at most 10 ¹²	Global timeout in seconds, including launch, I/O and final wait.
max_output	integer from 1 to 2 GiB	Separate limit for final stdout and each stderr; default: 10 MiB.

Unknown keys are rejected. stdin, stdout and stderr are binary-safe and may contain NUL bytes. Timeout resolution is one millisecond; values above 10¹² seconds are rejected.

Result

Once every stage has launched, a non-zero exit or signal is a process result, not an API error. Babet returns:

```

{
  stdout = "...",
  stderr = { "stage 1 stderr", "stage 2 stderr", ... },
  stages = {
    {
      launched = true,
      code = 0,
      exited = true,
      signaled = false,
      signal = nil,
    },
    -- ...
  },
  code = 0,
  all_succeeded = true,
  failed_index = nil,
  timed_out = false,
  stdout_truncated = false,
}

```

```
stderr_truncated = { false, false, ... },
}
```

code is the last stage's code, matching Unix pipeline convention. It is the normal exit status or 128 + signal_number. In the pathological case where a status remains unavailable after bounded timeout cleanup, it is -1 for that stage. all_succeeded is true only when every stage succeeded. failed_index identifies the first non-zero stage even when the last one succeeds.

A non-zero intermediate stage does not artificially stop the other programs. Normal POSIX pipe behaviour applies; an upstream producer may receive SIGPIPE after a downstream consumer closes its input.

Timeout, large streams and truncation

pipeline()'s timeout covers launch, stdin writes, stdout/stderr draining and the final wait. If it expires before every stage completes chdir and exec, the call returns nil, err with the message "pipeline: launch timed out" after rolling back stages already created. After a complete launch, expiry returns a result table with timed_out == true: Babet closes stdin, sends SIGTERM to every stage process group, then SIGKILL to survivors, and reaps direct children.

Descriptors are non-blocking and driven with poll(). The first stdin, final stdout and all stderr streams progress concurrently, preventing the usual large-volume deadlocks. Once max_output is reached, extra bytes are discarded but the stream continues to be drained, and the matching truncation flag is set.

babet.spawnPipeline() — streaming

Signature

```
pipeline, err = babet.spawnPipeline(commands [, opts])
```

Global options:

Option	Type	Description
cwd	string	Default working directory for every stage.
env	string-to-string table	Variables merged with Babet's environment.
launch_timeout	finite positive number	Deadline covering only creation and complete launch of all stages, at most INT_MAX milliseconds (about 24.8 days).

spawnPipeline() accepts no stdin, timeout, or max_output option: the script streams data and applies its own limits. Success returns a userdata and nil; validation or launch failure returns nil, err after cleaning up any stage that was already created.

```
local pipeline, err = babet.spawnPipeline({
  { "cat" },
  { "tr", { "a-z", "A-Z" } },
})
assert(pipeline, err)

local written, write_err = pipeline:write("hello\n")
assert(written, write_err)
assert(pipeline:close_stdin())
```

```

local chunks = {}
while true do
    local data, read_err = pipeline:read_stdout(64 * 1024, 1)
    if data then
        chunks[#chunks + 1] = data
    elseif read_err == "closed" then
        break
    elseif read_err ~= "timeout" then
        error(read_err)
    end
end

local status, wait_err = pipeline:wait(5)
assert(status, wait_err)
assert(status.code == 0)
print(table.concat(chunks))

```

Read methods

```

data, err = pipeline:read_stdout([max_bytes [, timeout]])
data, err = pipeline:read_stderr(stage [, max_bytes [, timeout]])

```

- max_bytes defaults to 64 KiB and must be between 1 and 16 MiB;
- timeout is in seconds, defaults to zero, and must be finite and non-negative;
- stage ranges from 1 to the number of stages;
- invalid arity, stage index, or max_bytes raises a Lua error, matching `babet.spawn()` methods;
- an invalid timeout returns `nil, err`;
- success returns a non-empty binary string and `nil`;
- no immediately available data returns `nil, "timeout"`;
- EOF, or reading an already closed stream, returns `nil, "closed"`.

Each call reads at most one chunk. There is no hidden capture: the script must retain, limit, or discard received data itself. Stderr streams remain independent, so every stream capable of filling its pipe must be drained.

Writing and closing stdin

```

count, err = pipeline:write(data [, timeout])
ok, err = pipeline:close_stdin()

```

`write()` accepts a binary string and may write only part of it; `count` is the number of bytes actually transmitted. An empty string returns `0, nil`. A timeout leaves the object reusable. Stdin closed by the script or downstream process returns `nil, "closed"` without delivering `SIGPIPE` to Babet itself.

`close_stdin()` closes the first stage's input and is idempotent. Call it after the last byte, otherwise a program waiting for EOF may never terminate.

Complete-write pattern:

```

local offset = 1
while offset <= #data do
    local count, err = pipeline:write(data:sub(offset), 1)
    if count then
        offset = offset + count
    elseif err ~= "timeout" then
        error(err)
    end
end
-- Drain stdout and stderr during large transfers as well.

```

```
end
assert(pipeline:close_stdin())
```

State and identifiers

```
running = pipeline:is_running([stage])
pids = pipeline:pids()
```

Without an index, `is_running()` is true while at least one direct stage has not been reaped. With an index, it describes only that stage. It refreshes statuses through `waitpid(..., WNOHANG)` without consuming the final result: `wait()` remains available afterwards.

`pids()` returns direct PIDs in stage order. They are diagnostic values; scripts must not assume that a PID remains valid after the pipeline exits.

Waiting and final status

```
status, err = pipeline:wait([timeout])
```

Without a timeout, `wait()` waits for every direct child. With a finite non-negative timeout, `nil`, "timeout" leaves the pipeline intact and reusable. Once every status is known, subsequent calls are idempotent and return the same result:

```
{
  stages = {
    { launched = true, code = 0, exited = true,
      signaled = false, signal = nil },
    -- ...
  },
  code = 0,
  all_succeeded = true,
  failed_index = nil,
}
```

`wait()` **does not drain** stdout or stderr. A pipeline that fills a pipe may therefore block before exiting. Streams must be read while it runs, exactly as with `babet.spawn()`.

Explicit termination

```
status, err = pipeline:terminate([grace_period])
status, err = pipeline:kill()
```

`terminate()` sends `SIGTERM` to every process group, gives processes `grace_period` seconds—2 seconds by default—to exit, then sends `SIGKILL` to survivors. The grace period must fit within `INT_MAX` milliseconds, about 24.8 days. `kill()` sends `SIGKILL` immediately. Both close stdin and return the same status table as `wait()` after direct children are reaped.

Each stage owns its own process group. Before reaping the direct process, Babet removes descendants that still remain in that group, including normal completion where a command exits without waiting for a background child. The leader PID therefore remains reserved until signalling and cannot be recycled for an unrelated process.

For `terminate()`, exited leaders deliberately remain unreaped for the whole grace period: descendants retain that time to react to `SIGTERM`, then the group receives `SIGKILL` before final reaping. A stage that deliberately creates a new session or process group escapes this POSIX guarantee.

close(), GC and Lua <close>

```
ok, err = pipeline:close()
```

`close()` is idempotent. It closes descriptors, attempts `SIGTERM`, uses `SIGKILL` when necessary, and reaps direct children. After closure, reads and writes return `nil`, `"closed"`, and `is_running()` returns `false`. A fully collected status remains available from `wait()`.

The same cleanup is used by `__gc` and `__close`. Recommended use:

```
local pipeline <close> = assert(babet.spawnPipeline({
  { "producer" },
  { "consumer" },
}))
```

GC is a safety net, not a synchronization mechanism. Use `wait()`, `terminate()`, or `close()` explicitly when cleanup timing matters.

Launch errors and reuse

Validation, pipe, fork, `chdir`, `exec`, and `launch_timeout` failures return `nil`, `err`. When a stage fails during launch, every already-created stage is terminated and reaped; Lua never receives a partially valid object.

Non-destructive `"timeout"` and `"interrupted"` errors allow another call. `"closed"` means the relevant stream or object can no longer perform that operation. Other system errors are returned as text and should be handled as such.

FS — files, directories, paths, searching, copying, permissions, and checksums

The functions in this chapter are exposed directly on the global `babet` table. There is no `babet.fs` subtable: this flat namespace is historical and is now part of the stable API.

The FS module covers:

- checking whether a path exists and what type it has;
- creating and removing files and directories;
- changing the current working directory;
- lexical path manipulation;
- listing, iterating, and recursively searching a tree;
- copying and moving directory trees;
- symbolic links;
- Unix permissions, owners, and groups;
- CRC-32, MD5, SHA, and BLAKE2 checksums.

Module contents

- [General conventions](#)
- [API overview](#)
- [Existence, type, and size](#)
 - [fileExists](#)
 - [isFile / isfile](#)
 - [isDir / isdir](#)
 - [fileSize](#)
- [Current working directory](#)
 - [currentDir](#)
 - [chdir](#)
- [Lexical path manipulation](#)
 - [getBasename](#)
 - [getFilename](#)
 - [getExtension](#)

- getPath
 - joinPath
- Create, remove, rename, and link
 - touch
 - mkdir
 - remove
 - rmdir
 - rmdirAll
 - rename
 - link
- List, iterate, and search
 - listFiles
 - createFileIterator
 - find
- Copy and move
 - copy
 - copyTree
 - moveTree
- Unix permissions and attributes
 - getMode / setMode
 - getAttributes / setAttributes
 - symlinkAttr / symlinkattr
- File checksums
- In-memory CRC-32
- Error contract
- Design decisions and limitations

General conventions

Paths

Paths are Lua strings and may be relative or absolute.

Babet does not automatically perform:

- ~ expansion;
- variable expansion such as \$HOME;
- glob expansion such as *.lua;
- general . and .. normalization in lexical path helpers.

For example:

```
local home = assert(babet.env("HOME"))
local config = babet.joinPath(home, ".config", "my-app")
```

A NUL byte in a path is always rejected. Depending on the function, a bad signature raises a Lua error, while a filesystem failure is generally returned as (nil, err).

Symbolic links

There is no single symlink rule that applies to every function:

- isFile, isDir, fileSize, getMode, getAttributes, and checksum functions follow a valid link to its target;
- remove removes the link itself, including a dangling link;
- rmdir and rmdirAll reject a symlink root;
- symlinkAttr changes the owner and group of the link itself;
- listFiles, createFileIterator, and find do not recurse into a symlink to a directory;
- copyTree and moveTree reject a symlink source or destination root, but handle links found inside the tree.

Each function documents its exact behavior below.

Result order

`listFiles`, `createFileIterator`, and `find` preserve the order supplied by the filesystem. That order is not sorted and must not be treated as stable.

Sort explicitly when order matters:

```
local files = assert(babet.listFiles("documents", true))
table.sort(files)
```

Return values

The most common convention is:

```
local value, err = function_name(...)
if not value then
    print(err)
end
```

A successful action generally returns `(true, nil)`. A few pure functions and iterator methods use a different shape; those exceptions are called out in their sections.

API overview

Check and inspect

Function	Successful result
<code>babet.fileExists(path)</code>	<code>(boolean, nil)</code> — true only for a regular file
<code>babet.isFile(path)</code>	<code>(boolean, nil)</code> — same functional predicate as <code>fileExists</code>
<code>babet.isDir(path)</code>	<code>(boolean, nil)</code> — true for a directory
<code>babet.fileSize(path)</code>	<code>(integer, nil)</code> — regular-file size in bytes
<code>babet.currentDir()</code>	<code>(string, nil)</code> — current working directory
<code>babet.getMode(path)</code>	<code>(integer, nil)</code> — Unix mode <code>0000..0777</code>
<code>babet.getAttributes(path)</code>	<code>(table, nil)</code> — {mode, owner, group}

Manipulate paths

Function	Successful result
<code>babet.getBasename(path)</code>	<code>(string, nil)</code>
<code>babet.getFilename(path)</code>	<code>(string, nil)</code>
<code>babet.getExtension(path)</code>	<code>(string, nil)</code>
<code>babet.getPath(path)</code>	<code>(string, nil)</code>
<code>babet.joinPath(a, b, ...)</code>	string
<code>babet.joinPath({a, b, ...})</code>	string

Act on the filesystem

Function	Successful result
<code>babet.touch(path)</code>	<code>(true, nil)</code>
<code>babet.mkdir(path)</code>	<code>(true, nil)</code> — recursive and idempotent

Function	Successful result
<code>babet.remove(path)</code>	<code>(true, nil)</code> — regular file or symlink
<code>babet.rmdir(path)</code>	<code>(true, nil)</code> — real, empty directory
<code>babet.rmdirAll(path)</code>	<code>(true, nil)</code> — real directory, recursive removal
<code>babet.rename(source, destination)</code>	<code>(true, nil)</code>
<code>babet.link(target, linkpath)</code>	<code>(true, nil)</code> — creates a symlink
<code>babet.chdir(path)</code>	<code>(true, nil)</code>

List and search

Function	Successful result
<code>babet.listFiles(path, recursive?)</code>	<code>(table, nil)</code>
<code>babet.createFileIterator(path, recursive?)</code>	<code>(iterator, nil)</code>
<code>babet.find(path, opts?)</code>	<code>(table, nil)</code>

Copy and move

Function	Successful result
<code>babet.copy(source, destination)</code>	<code>(true, nil)</code> — one file
<code>babet.copyTree(source, destination, continue_on_error?)</code>	<code>(true, nil)</code> — one tree
<code>babet.moveTree(source, destination)</code>	<code>(true, nil)</code> — one tree

Change attributes

Function	Successful result
<code>babet.setMode(path, mode)</code>	<code>(true, nil)</code>
<code>babet.setAttributes(path, uid, gid, mode?)</code>	<code>(true, nil)</code>
<code>babet.symlinkAttr(path, uid, gid)</code>	<code>(true, nil)</code>

Existence, type, and size

`babet.fileExists(path)`

Checks whether `path` designates a **regular file**.

```
local exists, err = babet.fileExists("report.txt")
```

Results:

- regular file: `(true, nil)`;
- directory, dangling link, or missing path: `(false, nil)`;
- valid link to a regular file: `(true, nil)`;
- genuine inspection error: `(nil, err)`.

Despite its historical name, `fileExists` is not true for every kind of path. Use `isDir` for a directory.

```
local is_file = assert(babet.fileExists("archive.tar"))
local is_dir  = assert(babet.isDir("backups"))
```

babet.isFile(path) and babet.isfile(path)

isFile is the canonical name. isfile is a deprecated compatibility alias.

It tests the same kind of path as fileExists: a regular file, following a valid symlink if present.

```
local regular, err = babet.isFile("program.lua")
if err then
    print("Cannot inspect path:", err)
elseif regular then
    print("This is a regular file")
end
```

A missing path returns (false, nil), not an error.

babet.isDir(path) and babet.isdir(path)

isDir is the canonical name. isdir is a deprecated alias.

```
local directory, err = babet.isDir("assets")
```

Results:

- real directory: (true, nil);
- valid link to a directory: (true, nil);
- file, dangling link, or missing path: (false, nil);
- access or inspection error: (nil, err).

babet.fileSize(path)

Returns the size of a regular file in bytes.

```
local size, err = babet.fileSize("video.mp4")
if not size then
    print("Cannot read size:", err)
else
    print(size, "bytes")
end
```

The function follows a valid symlink to a file. Directories and other file types are rejected.

```
local size, err = babet.fileSize("documents")
-- size == nil; err says the path is not a regular file
```

Current working directory

babet.currentDir()

Returns the process current working directory.

```
local cwd, err = babet.currentDir()
assert(cwd, err)
print(cwd)
```

The result is the path supplied by the filesystem and is normally absolute.

babet.chdir(path)

Changes the current working directory of the entire process.

```
local previous = assert(babet.currentDir())
assert(babet.chdir("/tmp"))
```

```
print(assert(babet.currentDir()))
assert(babet.chdir(previous))
```

The path is resolved to its canonical form: `.` and `..` components and symlinks are resolved before changing directory.

The working directory is shared by all threads. Consequently, `chdir` is permanently rejected after the first `babet.workers.spawn()`, even after that worker has completed.

```
assert(babet.chdir("/srv/my-app")) -- do this before any spawn
local worker = assert(babet.workers.spawn("return true"))

local ok, err = babet.chdir("/tmp")
-- ok == nil; err explains that chdir is forbidden after workers.spawn
```

Lexical path manipulation

These helpers only process strings. They do not check whether the path exists.

babet.getBaseName(path)

Returns the last component, including its extension.

```
local name = assert(babet.getBaseName("/var/log/app.log"))
print(name) -- app.log
```

A path with no usable last component, such as `/`, returns `(nil, err)`.

babet.getFilename(path)

Returns the last component without its final extension.

```
print(assert(babet.getFilename("report.tar.gz"))) -- report.tar
print(assert(babet.getFilename("README")))        -- README
```

babet.getExtension(path)

Returns the final extension, including the dot.

```
print(assert(babet.getExtension("report.tar.gz"))) -- .gz
```

A name without an extension returns `(nil, err)`, not an empty string.

```
local ext, err = babet.getExtension("README")
-- ext == nil
```

babet.getPath(path)

Returns the lexical parent directory.

```
print(assert(babet.getPath("/var/log/app.log"))) -- /var/log
```

A simple name with no parent component, such as `app.log`, returns `(nil, err)` rather than `."`.

babet.joinPath(...)

Two forms are accepted:

```
local path = babet.joinPath("var", "lib", "my-app", "data.db")
local path = babet.joinPath({ "var", "lib", "my-app", "data.db" })
```

At least two non-empty segments are required. On success, `joinPath` returns **one value**, the joined string. On failure it returns `(nil, err)`.

The function only adjusts the `/` separator at segment boundaries:

```
print(babet.joinPath("/var/", "/log", "app.log"))
-- /var/log/app.log
```

It does not normalize `.` or `..`, and a later absolute-looking segment does not reset the path:

```
print(babet.joinPath("base", "..", "other")) -- base/../other
print(babet.joinPath("base", "/other"))      -- base/other
```

Do not mix the table form with separate arguments. When the first argument is a table, that table is the complete segment list.

Create, remove, rename, and link

`babet.touch(path)`

Creates an empty file when the path does not exist, or updates the last-write time of an existing path.

```
assert(babet.touch("cache/ready.flag"))
```

The parent directory must already exist; `touch` does not create it.

```
local ok, err = babet.touch("missing/nested/file.txt")
-- ok == nil if missing/nested does not exist
```

Create parents first when needed:

```
assert(babet.mkdir("cache/images"))
assert(babet.touch("cache/images/index.dat"))
```

For an existing path, the function updates its timestamp; like the Unix `touch` command, it can therefore also touch a directory. A valid final symlink is followed to its target.

`touch` never opens an existing path with a truncating mode. The parent and existing target are pinned before the operation, so a file that appears during a concurrent create race is reused without losing its contents. A dangling final symlink is refused: Babet does not create the missing target through a mutable symlink path.

`babet.mkdir(path)`

Creates a directory with `mkdir -p` semantics:

- creation is **always recursive**;
- missing parents are created;
- calling it again on an existing directory succeeds;
- a file or another non-directory blocking the path returns `(nil, err)`.

```
assert(babet.mkdir("data/cache/images"))
-- creates data, data/cache, and data/cache/images as needed
assert(babet.mkdir("data/cache/images"))
-- also succeeds when the directory already exists
```

There is **no option to disable recursion**. To require an already-existing parent, check it before calling `mkdir`:

```
local parent = "data/cache"
assert(babet.isDir(parent), "parent must already exist")
assert(babet.mkdir(babet.joinPath(parent, "images")))
```

New directories use mode 0777 filtered through the process umask. No source directory mode is involved.

babet.remove(path)

Removes only:

- a regular file;
- a symbolic link, valid or dangling.

```
assert(babet.remove("temp.txt"))
-- the link is removed, not its target
assert(babet.link("important-file.txt", "shortcut"))
assert(babet.remove("shortcut"))
```

Directories, FIFOs, sockets, devices, and other special types are rejected.

```
local ok, err = babet.remove("my-directory")
-- ok == nil; use rmdir or rmdirAll
```

babet.rmdir(path)

Removes a **real empty directory**.

```
assert(babet.mkdir("empty-cache"))
assert(babet.rmdir("empty-cache"))
```

It rejects:

- a non-empty directory;
- a file;
- a symlink, even when it points to a directory;
- a missing path.

```
local ok, err = babet.rmdir("non-empty-directory")
```

babet.rmdirAll(path)

Recursively removes a **real directory root** and all of its contents.

```
assert(babet.rmdirAll("temporary-build"))
```

The root must exist and be a real directory. A file or a symlink to a directory is rejected.

Symlinks *inside* the removed directory are deleted as directory entries; their external targets are not traversed.

```
local is_dir = babet.isDir("temporary-build")
if is_dir then
    assert(babet.rmdirAll("temporary-build"))
end
```

babet.rename(source, destination)

Renames or moves an entry on the same filesystem.

```
assert(babet.rename("old.txt", "new.txt"))
```

Files, directories, and symlinks are accepted, including dangling links. The destination parent directory is not created.

On Linux, the function follows rename(2) semantics: an existing destination may be replaced when the entry types are compatible. Crossing filesystems fails; rename does not perform a copy-and-delete fallback.

```
local ok, err = babet.rename("/tmp/a.txt", "/other-mount/a.txt")
-- may fail with cross-device link
```

Use `moveTree` to move a directory tree with a cross-filesystem fallback.

`babet.link(target, linkpath)`

Always creates a **symbolic link**, never a hard link.

The target is stored exactly as supplied:

```
assert(babet.link("../shared/config.toml", "app/config.toml"))
```

A relative target stays relative, and a missing target is allowed:

```
assert(babet.link("future-file.txt", "dangling-link"))
```

Unlike `touch` and `copy`, `link` recursively creates explicit parent directories of `linkpath`:

```
assert(babet.link("../../data.db", "a/b/c/database"))
-- creates a/, a/b/, and a/b/c/ as needed
```

The link path must not already be occupied.

List, iterate, and search

The three APIs do not return paths in the same form:

Function	Returned path form
<code>listFiles(root, ...)</code>	relative to root
<code>createFileIterator(root, ...)</code>	as produced from root: prefixed by the supplied root path
<code>find(root, ...)</code>	as produced from root: prefixed by the supplied root path

`babet.listFiles(path, recursive?)`

Returns an array containing regular files only. Directories are never included.

Non-recursive listing — the default

```
local files, err = babet.listFiles("assets")
assert(files, err)

for _, relative_path in ipairs(files) do
    print(relative_path)
end
```

Only files directly contained in `assets` are returned.

Recursive listing

```
local files, err = babet.listFiles("assets", true)
assert(files, err)

table.sort(files)
for _, relative_path in ipairs(files) do
    print(relative_path)
end
```

Paths are relative to the requested root:

```
logo.png
icons/open.png
icons/close.png
```

not:

```
assets/logo.png
assets/icons/open.png
```

Symlinks

- a valid symlink to a regular file is included under the link name;
- a dangling link is skipped;
- a symlink to a directory is neither included as a file nor traversed;
- directory-symlink loops therefore cannot cause infinite recursion.

```
local files = assert(babet.listFiles("collection", true))
-- collection/external -> /other/directory is not followed
```

The second argument is optional and defaults to false. When present, it must be a strict Lua boolean; numbers and strings are not coerced. The function accepts exactly one or two arguments.

babet.createFileIterator(path, recursive?)

Creates a userdata with two methods:

- `iterator:next()` — next path, or nil at end;
- `iterator:close()` — explicitly releases the iterator.

Non-recursive

```
local iterator, err = babet.createFileIterator("assets")
assert(iterator, err)
```

```
while true do
    local path = iterator:next()
    if path == nil then
        break
    end
    print(path) -- assets/logo.png, etc.
end
```

```
iterator:close()
```

Recursive

```
local iterator = assert(babet.createFileIterator("assets", true))
while true do
    local path = iterator:next()
    if not path then break end
    print(path)
end
iterator:close()
```

Despite its name, the current implementation builds the complete list during `createFileIterator`. Access and traversal failures are therefore returned at creation time, before the first `next()` call.

Symlink behavior:

- valid link to a regular file: included under the link path;
- dangling link, loop, or inaccessible target: skipped;
- link to a directory: not traversed;
- genuine entry-inspection or traversal-increment error: creation fails with `(nil, err)`.

Calling `next()` after `close()` raises a Lua error. Omitting `close()` is not fatal: the garbage collector eventually releases the object. The optional recursive argument must be a strict Lua boolean, and the constructor accepts exactly one or two arguments.

`babet.find(path, opts?)`

Recursively searches the entries of a directory and returns an array of paths.

`path` must be a strict Lua string. `opts` must be a table or `nil`, and the function accepts exactly one or two arguments. Option fields are also strict: depths are Lua integers, and every `name/path/regex/glob` field is a Lua string.

These calls are equivalent:

```
local entries = assert(babet.find("src"))
local entries = assert(babet.find("src", nil))
```

Without options, both files **and** directories are included. The root itself is not included.

Options

Option	Type	Default	Effect
<code>type</code>	string	no filter	"f" for files, "d" for directories
<code>glob</code>	string	absent	bounded glob on the complete basename, case-sensitive
<code>iglob</code>	string	absent	bounded glob on the complete basename, ASCII case-insensitive
<code>path_glob</code>	string	absent	bounded glob on the complete returned path, case-sensitive
<code>path_iglob</code>	string	absent	bounded glob on the complete returned path, ASCII case-insensitive
<code>name</code>	string	absent	RE2 regex on the complete basename, case-sensitive
<code>iname</code>	string	absent	RE2 regex on the complete basename, case-insensitive
<code>path</code>	string	absent	RE2 regex searched within the full path
<code>mindepth</code>	integer	0	minimum included depth
<code>maxdepth</code>	integer	practically unlimited	maximum included depth

All supplied options are combined with logical **AND**. An empty pattern acts like an absent filter, matching the historical regex behavior.

Safe glob filters Use the glob fields for ordinary filename filtering, especially when a pattern may come from an untrusted user:

```
local lua_files = assert(babet.find("src", {
    type = "f",
    glob = "*.lua",
}))

local images = assert(babet.find("assets", {
    type = "f",
    iglob = "*.jpg",
}))
```

The glob language is deliberately small:

- *: zero or more bytes except /;
- **: zero or more bytes, including /;
- ?: exactly one byte except /;
- \x: literal byte x.

Matching is anchored to the complete basename or path. Brackets, braces, and shell extglob syntax have no special meaning and are matched literally. Each pattern is limited to 4096 bytes. The matcher uses no recursive backtracking; its work is bounded polynomially by the pattern and candidate lengths.

path_glob and path_iglob match the complete returned path with / as the separator. Use ** when a wildcard must cross directories:

```
local tests = assert(babet.find(".", {
    type = "f",
    path_glob = "**/tests/**/*.lua",
}))
```

The insensitive variants fold ASCII letters only. UTF-8 bytes outside ASCII are compared exactly, which keeps behavior independent of the process locale.

Bounded RE2 regular expressions The historical name, iname, and path fields now use the statically linked RE2 engine. name and iname match the complete basename:

```
local images = assert(babet.find("assets", {
    type = "f",
    iname = ".*\\.(png|jpg|jpeg)$",
}))
```

path performs a partial regex search within the complete path:

```
local tests = assert(babet.find(".", {
    type = "f",
    path = "/tests/",
}))
```

Each regex is limited to 4096 bytes and compiled with a 1 MiB RE2 memory budget. Matching is linear-time; when the DFA cache reaches its budget, RE2 falls back to its bounded NFA instead of backtracking exponentially. Babet uses RE2's Latin-1 mode so every Linux pathname byte remains matchable, including names that are not valid UTF-8. iname disables case sensitivity through RE2; no process locale is consulted.

RE2 deliberately rejects constructs that require backtracking or non-regular state, notably back-references and look-around assertions. Such a pattern makes the whole call fail with (nil, err). Use the safe glob fields for simple filename filters and RE2 when grouping, alternation, character classes, or bounded repetitions are genuinely useful.

Limit depth Depth 0 is the root's immediate children:

```
local direct_children = assert(babet.find("src", {
    maxdepth = 0,
}))
```

Children of a subdirectory are at depth 1:

```
local one_level_below = assert(babet.find("src", {
    mindepth = 1,
    maxdepth = 1,
}))
```

mindepth filters results only; shallower directories are still traversed to reach requested depths.

Symlinks and errors Traversal does not recurse into directory symlinks. Type checks do follow a valid target, however: a symlink to a file may match type = "f", and a symlink to a directory may appear with type = "d" without its contents being traversed.

An invalid or oversized regex, invalid or oversized glob, or traversal error fails the entire call with (nil, err). There is no "continue after errors" mode for find.

Copy and move

babet.copy(source, destination)

Copies one file using std::filesystem::copy_file and overwrites an existing file destination.

```
assert(babet.copy("config/default.toml", "config/local.toml"))
```

The destination parent directory must already exist:

```
assert(babet.mkdir("backup/config"))
assert(babet.copy("config/app.toml", "backup/config/app.toml"))
```

Important behavior:

- a valid source symlink is followed and produces a new regular file;
- a valid destination symlink is followed: the link target is overwritten;
- source and destination directories are rejected;
- ordinary source permissions are copied according to filesystem semantics; owner, group, and timestamps are not guaranteed;
- this operation does not use copyTree's hardened destination confinement.

Use copyTree for a tree or when destination symlink redirection must be prevented.

babet.copyTree(source, destination, continue_on_error?)

Recursively copies a source directory to a destination directory.

```
assert(babet.copyTree("site", "backup/site"))
```

Default value of continue_on_error The optional third argument defaults to **true**. When present it must be a strict Lua boolean; the function accepts exactly two or three arguments.

For an error limited to one entry:

1. a warning is written to stderr;
2. copying continues with other entries when possible;
3. the final call returns (nil, "completed with warnings ...").

A partial success is therefore never reported as a full success.

```

local ok, err = babet.copyTree("source", "destination")
if not ok then
    print(err) -- may report warnings after a partial copy
end

```

Strict mode Pass false explicitly to stop at the first error:

```

local ok, err = babet.copyTree("source", "destination", false)
assert(ok, err)

```

Pass true explicitly when you want the continuation choice to be visible:

```

local ok, err = babet.copyTree("source", "destination", true)

```

Destination and merging

- the destination is recursively created when absent;
- existing destinations are merged;
- existing regular files are replaced;
- a pre-existing destination symlink, either as an intermediate component or final file, is rejected;
- the destination cannot be inside the source;
- source and destination roots must not be symlinks.

Metadata When a new file is created:

- ordinary source rwx bits are preserved;
- setuid, setgid, and sticky bits are removed;
- the new inode's owner and group are determined by the system for the caller;
- timestamps are not preserved.

Created directories use 0777 filtered through the umask. Source directory mode, owner, group, and timestamps are not reproduced.

Internal symlinks Links inside the source tree are recreated:

- relative targets stay unchanged;
- dangling links stay dangling;
- an absolute target inside the source is rewritten to its corresponding destination path;
- an absolute external target stays unchanged.

Directory symlinks are never traversed as directories.

babet.moveTree(source, destination)

Recursively moves a directory tree.

```

assert(babet.moveTree("staging/site", "public/site"))

```

Fast path When the destination does not exist, the move remains on one filesystem, and no internal absolute link needs rewriting, Babet renames the whole tree. This is fast and preserves inodes and their metadata.

Merge or cross-filesystem fallback When the destination exists, the move crosses a filesystem, or an internal link needs rewriting, Babet uses an entry-by-entry fallback:

- scan the complete source before the first modification;
- create destination directories;
- create destination symlinks;
- move other entries;

- remove the residual source tree.

In this fallback, a file copied to a new inode preserves ordinary rwx bits but not special bits, owner, group, or timestamps. Created directories follow the umask.

Safety checks

- source must be a real directory, not a symlink;
- destination root must not be a symlink;
- destination cannot be inside source;
- a pre-existing symlink in a merge destination is rejected;
- internal links use the same rewriting rules as copyTree.

moveTree reduces partial-state risk by scanning first and preparing symlinks before moving files. It is not a general transaction, however: an error after several file moves may leave some entries in the source and others in the destination. Such an error requires checking both trees manually.

Unix permissions and attributes

Modes are integers. Lua has no 0o755 literal; use an octal string where the API accepts it, or `tonumber("755", 8)`.

```
local mode_755 = tonumber("755", 8)
```

babet.getMode(path) and babet.setMode(path, mode)

getMode returns ordinary permissions and special bits in the range 0000..07777.

```
local mode, err = babet.getMode("run.sh")
assert(mode, err)
print(string.format("%04o", mode))
```

setMode accepts two forms:

```
assert(babet.setMode("run.sh", "755"))
assert(babet.setMode("run.sh", tonumber("755", 8)))
```

A string is interpreted in base 8. A number is taken as the direct integer value. Non-integers, negative values, and values above 07777 are rejected.

Special bits may be requested explicitly:

```
assert(babet.setMode("tool", "4755"))
local mode = assert(babet.getMode("tool"))
assert(mode == tonumber("4755", 8))
```

Both functions follow a symlink to its target.

babet.getAttributes(path)

Returns exactly these three fields:

```
local attrs, err = babet.getAttributes("file.txt")
assert(attrs, err)

print(attrs.mode) -- integer 0000..07777
print(attrs.owner) -- UID
print(attrs.group) -- GID
```

The table contains no size, mtime, or file type.

The function follows a symlink to its target.

babet.setAttributes(path, uid, gid, mode?)

Changes owner and group, then optionally the mode.

Without a mode:

```
local attrs = assert(babet.getAttributes("file.txt"))
assert(babet.setAttributes(
  "file.txt",
  attrs.owner,
  attrs.group
))
```

With a mode:

```
assert(babet.setAttributes(
  "file.txt",
  1000,
  1000,
  tonumber("640", 8)
))
```

Unlike `setMode`, the fourth argument must be an **integer**; a string such as "640" is not accepted.

Validation before mutation:

- UID and GID must be non-negative integers in the `uid_t` and `gid_t` range;
- mode must be between 0 and 0777.

POSIX provides no atomic operation combining `chown` and `chmod`. Babet resolves the path once, pins that target, and performs the initial metadata read, `chown`, optional `chmod`, and any rollback on the same inode. Replacing the pathname during the call therefore cannot redirect the later phase to a new target. If `chmod` fails after a successful `chown`, Babet attempts to restore the original owner, group, and mode; an incomplete rollback is reported in the error.

The call follows a final symlink, but the target selected when the operation starts remains pinned even if the link is changed concurrently. Required privileges depend on the system and process identity; changing owner normally requires root privileges.

babet.symlinkAttr(path, uid, gid) and babet.symlinkattr(...)

`symlinkAttr` is the canonical name. `symlinkattr` is a deprecated alias.

The function uses `lchown` and therefore does not follow a symbolic link:

```
assert(babet.symlinkAttr("shortcut", 1000, 1000))
```

It changes the UID and GID of the link itself, not of its target. It does not change mode.

UID and GID are validated as in `setAttributes`.

File checksums

All functions below stream the file and return lowercase hexadecimal text:

Function	Algorithm	Hex length	Recommended use
<code>babet.crc32sum(path)</code>	CRC-32 IEEE	8	accidental-error detection only
<code>babet.md5sum(path)</code>	MD5	32	legacy compatibility, not security
<code>babet.sha1sum(path)</code>	SHA-1	40	legacy compatibility, not security

Function	Algorithm	Hex length	Recommended use
<code>babet.sha256sum(path)</code>	SHA-256	64	common cryptographic integrity
<code>babet.sha384sum(path)</code>	SHA-384	96	cryptographic integrity
<code>babet.sha512sum(path)</code>	SHA-512	128	cryptographic integrity
<code>babet.sha3_256sum(path)</code>	SHA3-256	64	cryptographic integrity
<code>babet.sha3_384sum(path)</code>	SHA3-384	96	cryptographic integrity
<code>babet.sha3_512sum(path)</code>	SHA3-512	128	cryptographic integrity
<code>babet.blake2b512sum(path)</code>	BLAKE2b-512	128	cryptographic integrity
<code>babet.blake2s256sum(path)</code>	BLAKE2s-256	64	cryptographic integrity

Example:

```
local digest, err = babet.sha256sum("release.tar.gz")
assert(digest, err)
print(digest)
```

Check against an expected value:

```
local expected = "...
local actual = assert(babet.sha256sum("release.tar.gz"))
assert(actual == expected, "checksum mismatch")
```

Common contract:

- regular files only;
- a valid symlink to a regular file is followed;
- directories, FIFOs, sockets, devices, and non-regular pseudo-files are rejected;
- a read error returns (`nil`, `err`) and never a misleading digest of partial data;
- the complete content is not loaded into memory.

CRC-32, MD5, and SHA-1 must not authenticate data against an attacker. Prefer SHA-256, SHA-3, or BLAKE2 for new integrity checks.

In-memory CRC-32

`babet.crc32(data)`

Computes CRC-32 for a Lua string already in memory. Lua strings are binary-safe, so embedded NUL bytes are accepted.

```
print(babet.crc32("abc")) -- 352441c2
print(babet.crc32("")) -- 00000000

local bytes = "a\0b\0c"
local digest = babet.crc32(bytes)
```

On success this function returns one string, without a second `nil` value. A bad argument count or type raises a Lua error.

Error contract

Raised Lua errors

A bad signature or incompatible type generally raises a Lua error:

```
babet.fileSize({})      -- Lua error
babet.mkdir("a", true)  -- Lua error: mkdir accepts exactly one argument
```

Paths containing a NUL byte are rejected before any system call.

Returned errors

Filesystem failures are normally returned:

```
local ok, err = babet.remove("missing.txt")
if not ok then
    print(err)
end
```

The `fileExists`, `isFile`, and `isDir` predicates treat a simply missing path as the normal result (`false`, `nil`). A genuine inspection failure remains (`nil`, `err`).

Partial operations

- `copyTree` in continuation mode may copy some entries and then return a summary error reporting warnings;
- `moveTree` may leave state split between source and destination when a late error occurs in its entry-by-entry fallback;
- `setAttributes` attempts rollback if `chmod` fails after `chown`, and explicitly reports an incomplete rollback.

Design decisions and limitations

- **Historical flat namespace:** `babet.fileExists`, not `babet.fs.fileExists`.
- **Canonical names:** `isFile`, `isDir`, and `symlinkAttr`. The `isfile`, `isdir`, and `symlinkattr` forms are deprecated.
- **mkdir is always recursive:** no non-recursive API variant exists.
- **copy for one file, copyTree for a tree:** no implicit overload based on source type.
- **Glob is a find filter, not a standalone function:** use `glob`, `iglob`, `path_glob`, or `path_iglob` in `babet.find`.
- **No automatic sorting:** call `table.sort` when stable ordering matters.
- **No complete metadata preservation during copy:** regular file permission bits are handled, but owners, groups, timestamps, and directory modes are not preserved.
- **No asynchronous I/O:** use [workers](#) to parallelize independent work.

HTTP — synchronous web requests, headers, bodies, TLS, and limits

`babet.http` is a synchronous HTTP/HTTPS client built on `cpp-httplib` and `OpenSSL`. It is intended for APIs, webhooks, simple REST calls, bounded in-memory responses, and direct-to-file downloads.

It supports:

- HTTP and HTTPS;
- GET, HEAD, OPTIONS, POST, PUT, PATCH, and DELETE;
- encoded and normalised query parameters;
- request headers;

- text or binary request bodies;
- binary responses;
- repeated response headers;
- optional redirects;
- TLS verification by default;
- connection and global request timeouts;
- a maximum in-memory response size;
- streamed GET downloads committed atomically to a destination file.

It does not expose streaming uploads, response callbacks, persistent sessions, a cookie jar, multipart/form-data, form helpers, proxies, an HTTP server, WebSockets, HTTP/2, or HTTP/3.

Module contents

- [Core conventions](#)
- [API overview](#)
- [request\(opts\)](#)
- [get\(url, opts?\)](#)
- [download\(url, destination, opts?\)](#)
- [post call forms](#)
- [URLs, fragments, and query](#)
- [HTTP methods](#)
- [Request headers](#)
- [Request body and Content-Type](#)
- [Timeouts](#)
- [TLS and CA](#)
- [Redirects](#)
- [Maximum response size](#)
- [Download size and file safety](#)
- [Response tables](#)
- [Complete examples](#)
- [Error contract](#)
- [Security and limitations](#)

Core conventions

Synchronous client

The call blocks the current thread until response or error. Use independent [workers](#) for parallel requests.

HTTP status is not a transport error

A received 404 or 500 is a normal response table:

```
local response, err = babet.http.get(url, { timeout = 10 })
assert(response, err)

if response.status >= 200 and response.status < 300 then
  print(response.body)
else
  io.stderr:write("HTTP ", response.status, "\n", response.body)
end
```

DNS, TCP, TLS, timeout, and body-limit failures return (nil, err).

Binary bodies

Request bodies and `response.body` may contain NUL bytes:

```
local response = assert(babet.http.post(url, "AB\0CD", {
    timeout = 10,
}))
```

URLs, methods, header names, CA paths, and other C-facing strings reject NUL. URLs and headers also reject CR/LF to prevent HTTP-line injection.

Unknown option fields

Unknown fields are currently ignored. A typo such as `timeot = 5` does not activate a timeout and does not fail. Use the documented names exactly.

API overview

```
local response, err = babet.http.request(opts)
local response, err = babet.http.get(url, opts?)
local result, err = babet.http.download(url, destination, opts?)
local response, err = babet.http.post(url)
local response, err = babet.http.post(url, body)
local response, err = babet.http.post(url, opts)
local response, err = babet.http.post(url, body, opts)
local response, err = babet.http.post(url, nil, opts)
```

Field	Type	Default	Meaning
url	string	required by request	absolute http:// or https:// URL
method	string	"GET"	supported method, case-insensitive
headers	table	{}	string names; string/number values
body	binary string	absent	body for body-capable methods
query	table	absent	string keys; string/number values
timeout	finite number > 0	library defaults	seconds, connect plus global budget
verify	strict boolean	true	HTTPS certificate verification
ca_cert	string	OpenSSL trust store	CA file path
follow_redirects	strict boolean	false	follow redirects automatically
max_body_size	integer 1..2 GiB	64 MiB	in-memory cap for request/get/post
max_file_size	positive integer	8 GiB	received-file cap for download

`get` and `post` always overwrite `url` and `method` after shallow-copying options.

`babet.http.request(opts)`

General form for every method:

```

local response, err = babet.http.request({
    url = "https://api.example.com/items/42",
    method = "DELETE",
    headers = {
        ["Authorization"] = "Bearer " .. token,
    },
    timeout = 10,
})

```

The first argument must be a table. `opts.url` must be a strict string. Invalid option fields return `(nil, err)`; a wrong first-argument type raises.

babet.http.get(url, opts?)

The wrapper shallow-copies `opts`, then forces:

```

opts.url = url
opts.method = "GET"

local response, err = babet.http.get(
    "https://api.example.com/health",
    { timeout = 5 }
)

```

The second argument must be absent, `nil`, or a table. GET rejects `opts.body` instead of silently dropping it.

babet.http.download(url, destination, opts?)

Downloads a GET response directly to a file without accumulating the body in Lua or in a C++ response string:

```

local result, err = babet.http.download(
    "https://example.com/archive.tar.gz",
    "downloads/archive.tar.gz",
    {
        timeout = 120,
        follow_redirects = true,
        max_file_size = 4 * 1024 * 1024 * 1024,
    }
)
assert(result, err)
assert(result.saved, "HTTP " .. result.status)
print(result.path, result.bytes)

```

The call is synchronous, but received chunks are written progressively to a same-directory temporary file. The final destination is replaced with one renameat only after the complete final response has been received and the temporary file has been synced and closed.

The wrapper always forces GET. It accepts the common headers, query, timeout, verify, `ca_cert`, and `follow_redirects` options. `body` is rejected. `max_file_size` replaces `max_body_size` for this call:

- strict positive Lua integer;
- default: 8 GiB;
- configurable up to `math.maxinteger`;
- enforced on the actual bytes delivered to the file receiver.

Only a final 2xx response is committed. A received 1xx, unfollowed 3xx, 4xx, or 5xx response is not a transport failure: `download` returns a result table with `saved = false`, removes the temporary file, and leaves any existing destination unchanged.

The destination contract is intentionally strict:

- the parent directory must already exist;
- Babet does not create parent directories;
- ... path components are rejected;
- parent-directory symlink components are rejected;
- a symlink at the final destination is replaced as an inode; its target is never followed or modified;
- a temporary file is created in the same directory with `O_EXCL` and is removed on every failure;
- a pre-existing destination remains untouched on DNS, TCP, TLS, timeout, size-limit, HTTP non-2xx, or disk-write failure.

The atomic rename prevents readers from observing a partially downloaded final path. It does not provide resume support, progress callbacks, crash-proof parent-directory durability, or preservation of metadata from an older file. The new file is created with mode `0666` filtered by the process `umask`.

babet.http.post call forms

No explicit body:

```
babet.http.post(url)
babet.http.post(url, { timeout = 5 })
babet.http.post(url, nil, { timeout = 5 })
```

Positional body:

```
babet.http.post(url, "payload", { timeout = 5 })
```

Body in options:

```
babet.http.post(url, {
    body = "payload",
    timeout = 5,
})
```

A table in the second position is interpreted as `opts`. Any other non-string, non-nil second value raises. When both a positional body and `opts.body` exist, the positional body wins.

URLs, fragments, and query

Only absolute HTTP/HTTPS URLs are accepted. A non-empty host is required; bracketed IPv6 literals are supported:

```
local r, err = babet.http.get("http://[::1]:8080/", { timeout = 2 })
```

Fragments are removed and never sent.

Add query parameters with a table:

```
local response = assert(babet.http.get("https://example.com/search", {
    query = {
        q = "lua socket",
        page = 2,
    },
    timeout = 10,
}))
```

Keys must be strings; values may be strings or numbers. Babet protects query delimiters and non-ASCII bytes first, then `cpp-httplib` 0.45.0 normalises the query before it is sent. The effective wire format therefore follows these conventions:

- spaces become `+`;

- + becomes %2B;
- / and ? remain literal inside a value;
- non-ASCII UTF-8 bytes are percent-encoded as %HH;
- &, =, and % are percent-encoded when they belong to a key or value.

For example, `q = "lua socket"` becomes `q=lua+socket`, while `path = "a/b"` remains `path=a/b`.

When the URL already contains `?a=b`, new parameters are appended with `&`. The existing query is normalised as well: `%20` may become `+` and `%2F` may become `/`. The `#...` fragment is removed before transmission.

Lua table iteration order is not stable, so do not build canonical signatures by assuming the order produced from query. For canonical signing, construct an already ordered URL yourself and account for the normalisation described above.

NUL bytes in query keys/values are sent as `%00`; make sure the server accepts that semantic.

HTTP methods

Supported methods:

- GET;
- HEAD;
- OPTIONS;
- POST;
- PUT;
- PATCH;
- DELETE.

Method names are uppercased internally:

```
local response = assert(babet.http.request({
    url = "https://api.example.com/items/42",
    method = "patch",
    body = '{"enabled":true}',
    headers = { ["Content-Type"] = "application/json" },
    timeout = 10,
}))
```

GET, HEAD, and OPTIONS reject a body. POST, PUT, PATCH, and DELETE accept an absent, empty, or non-empty body.

There are no put, patch, or delete shortcuts; use request.

Request headers

```
headers = {
    ["Accept"] = "application/json",
    ["Authorization"] = "Bearer " .. token,
    ["X-Retry"] = 3,
}
```

Header names must be non-empty HTTP token strings: alphanumeric plus `!#$%&'*+-.^_`|~`. Spaces, tabs, colon, CR/LF, NUL, and non-string keys are rejected.

Values may be strings or numbers. Numbers are converted using Lua's textual conversion. NUL, CR/LF, booleans, tables, and other types are rejected.

A Lua table cannot represent repeated exact request-header names. The binding does not expose a multi-value request-header form. Combine values only when the specific header grammar permits it.

Names are sent with the provided case; HTTP matching is case-insensitive. Content-Type detection is case-insensitive.

Request body and Content-Type

Absent and explicitly empty bodies differ:

```
babet.http.post(url, { timeout = 5 }) -- no body field
babet.http.post(url, "", { timeout = 5 }) -- present empty body
```

When a body exists on POST/PUT/PATCH/DELETE and no Content-Type is supplied, Babet uses application/octet-stream. No default Content-Type is added when the body is absent.

JSON example:

```
local payload = assert(babet.json.encode({ name = "babet" }))
local response = assert(babet.http.post(url, payload, {
    headers = {
        ["Content-Type"] = "application/json",
        ["Accept"] = "application/json",
    },
    timeout = 10,
}))
```

There is no form helper. Encode URL-encoded form data yourself and set application/x-www-form-urlencoded. The query table changes the URL only; it does not build a form body.

No multipart builder or streaming upload is exposed. For large/complex uploads, use a specialised tool through `babet.exec` without unsafe shell concatenation.

opts.timeout

```
local response, err = babet.http.get(url, { timeout = 10 })
```

The value must be finite and strictly positive. Positive values below 1 ms are rounded up. Babet configures both a connection timeout and `cpp-httpplib`'s global maximum request timeout; the first limit reached wins.

Synchronous DNS may still happen outside the controllable library budget and can make wall-clock duration exceed the requested timeout.

Without an explicit timeout, `cpp-httpplib`/system defaults apply. Always set one for remote services.

HTTPS, verification, and CA

Verification defaults to true:

```
local response = assert(babet.http.get("https://example.com/", {
    timeout = 10,
}))
```

Use a private CA file:

```
local response = assert(babet.http.get("https://internal.example/", {
    ca_cert = "/etc/myapp/internal-ca.pem",
    timeout = 10,
}))
```

HTTP exposes `ca_cert`, not `ca_path`, and does not run the socket TLS module's manual distro-path probing. It relies on the embedded OpenSSL defaults, environment, and the supplied CA file.

```
local response = assert(babet.http.get("https://127.0.0.1:8443/", {
    verify = false,
    timeout = 3,
}))
```

Use `verify = false` only in controlled tests. HTTP has no separate hostname override; the URL host is used.

follow_redirects

Redirects are visible by default:

```
local r = assert(babet.http.get(url, { timeout = 10 }))
if r.status == 302 then
    print(r.headers.location)
end
```

Enable automatic following:

```
local r = assert(babet.http.get(url, {
    follow_redirects = true,
    timeout = 10,
}))
```

The result describes the final response. Redirect history and `max_redirects` are not exposed; cpp-httplib's internal limit applies.

For untrusted URLs, consider redirects to another domain, HTTPS-to-HTTP downgrade, internal/metadata addresses, and sensitive-header handling.

max_body_size

The entire response is buffered in memory:

```
local response, err = babet.http.get(url, {
    timeout = 30,
    max_body_size = 8 * 1024 * 1024,
})
```

- default: 64 MiB;
- minimum: 1 byte;
- maximum: 2 GiB;
- strict positive Lua integer.

When actual received chunks would exceed the cap, Babet aborts and returns:

```
nil, "http: response body exceeds max_body_size"
```

No partial body is exposed. The guard applies to actual received bytes, not only Content-Length, including chunked or misleading responses.

For large GET payloads, use `babet.http.download` instead of raising this in-memory limit.

max_file_size and file safety

`max_file_size` applies only to download. It limits received body bytes, not the value advertised by Content-Length. If the stream would exceed the cap, Babet aborts, removes the temporary file, preserves an existing destination, and returns:

```
nil, "http: response body exceeds max_file_size"
```

The default 8 GiB is a safety guard, not a recommendation to accept every file of that size. Set the smallest limit compatible with the expected artifact. Free-space exhaustion and filesystem errors are reported as (nil, err) with the same cleanup guarantee.

Response tables

In-memory response

```
{
  status = 200,
  body = "...",
  headers = {
    ["content-type"] = "application/json",
    ["set-cookie"] = "b=2",
  },
  headers_multi = {
    ["content-type"] = { "application/json" },
    ["set-cookie"] = { "a=1", "b=2" },
  },
}
```

- status: integer HTTP status;
- body: complete binary string, possibly empty; HEAD returns an empty body;
- headers: lowercase names, last occurrence wins;
- headers_multi: lowercase names, every occurrence in an array, even one.

Use headers_multi for repeated fields such as Set-Cookie:

```
for _, cookie in ipairs(response.headers_multi["set-cookie"] or {}) do
  print(cookie)
end
```

Download result

```
{
  status = 200,
  saved = true,
  bytes = 123456,
  path = "downloads/archive.tar.gz",
  headers = { ... },
  headers_multi = { ... },
}
```

- status, headers, and headers_multi have the same meaning as above;
- saved is true only when the final status is 2xx and the atomic commit succeeded;
- bytes is the number of received body bytes, including for a non-2xx result;
- path exists only when saved is true;
- there is deliberately no body field.

Complete examples

GET JSON

```
local response, err = babet.http.get("https://api.example.com/v1/status", {
  headers = { ["Accept"] = "application/json" },
  query = { verbose = 1 },
  timeout = 10,
  max_body_size = 1024 * 1024,
})
```

```

assert(response, err)
assert(response.status == 200, response.body)
local data = assert(babet.json.decode(response.body))

```

Authenticated JSON POST

```

local body = assert(babet.json.encode({ title = "hello" }))
local response, err = babet.http.post(
    "https://api.example.com/v1/items",
    body,
    {
        headers = {
            ["Authorization"] = "Bearer " .. token,
            ["Content-Type"] = "application/json",
        },
        timeout = 15,
        max_body_size = 2 * 1024 * 1024,
    }
)
assert(response, err)

```

Binary PUT

```

local file = assert(io.open("image.bin", "rb"))
local payload = file:read("a")
file:close()

local response = assert(babet.http.request({
    url = "https://upload.example.com/blob/42",
    method = "PUT",
    body = payload,
    headers = { ["Content-Type"] = "application/octet-stream" },
    timeout = 30,
    max_body_size = 1024 * 1024,
}))

```

This loads the file into memory and is not suitable for very large uploads.

Download a release artifact

```

assert(babet.mkdir("downloads"))
local result, err = babet.http.download(
    "https://example.com/releases/tool.tar.gz",
    "downloads/tool.tar.gz",
    {
        timeout = 120,
        follow_redirects = true,
        max_file_size = 512 * 1024 * 1024,
    }
)
assert(result, err)
if not result.saved then
    error("unexpected HTTP status " .. result.status)
end
print("saved", result.bytes, "bytes to", result.path)

```

Separate network and HTTP failures

```
local response, err = babet.http.get(url, { timeout = 5 })
if not response then
    io.stderr:write("transport failure: ", err, "\n")
elseif response.status == 404 then
    print("not found")
elseif response.status >= 400 then
    io.stderr:write("HTTP ", response.status, "\n")
else
    print(response.body)
end
```

Parallel requests with WORKERS

```
local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = assert(babet.workers.spawn([[
        local r, err = babet.http.get(worker.args.url, {
            timeout = 10,
            max_body_size = 1024 * 1024,
        }])
        if not r then error(err) end
        return { status = r.status, size = #r.body }
    ]], { url = url })))
end

for i, job in ipairs(jobs) do
    local ok_result, value = job:join()
    if ok_result then
        print(urls[i], value.status, value.size)
    else
        io.stderr:write(urls[i], ": ", value, "\n")
    end
end
```

Error contract

Wrong positional types raise:

```
babet.http.request("not a table")
babet.http.get(42)
babet.http.get(url, "not a table")
babet.http.post(url, 42)
babet.http.download(url, 42)
babet.http.download(url, "file", "not a table")
```

Invalid values and runtime failures return (nil, err): missing/bad URL, unsupported method, forbidden body, invalid options/query/headers, DNS/TCP/TLS, timeout, response/file limit, destination-path or filesystem failure, and converted internal exceptions.

Any received HTTP status, including 3xx/4xx/5xx, returns a normal response. For download, only 2xx is saved; all other statuses return saved = false.

Security and limitations

- Validate untrusted URLs against SSRF to loopback, LAN, cloud metadata, and administrative services.

- Treat redirects as new destinations and protect sensitive headers.
- Keep TLS verification enabled; use private CA trust instead of disabling it.
- Set the smallest practical `max_body_size` or `max_file_size`.
- `request`, `get`, and `post` keep response bodies in memory. `download` streams only GET responses to a file; uploads are still in memory.
- Atomic replacement protects the final path from partial content, but callers remain responsible for validating expected status, size, and cryptographic checksum.
- Every call constructs a client; no exposed cookie jar or connection pool.
- No form/multipart helper, proxy configuration, HTTP/2/3, WebSocket, or server.
- DNS is synchronous and may exceed the requested timeout.

babet.inotify — filesystem watching

`babet.inotify` provides a small interface to Linux `inotify(7)`. Notifications come from the kernel, so watched directories do not need to be scanned periodically.

The module is **Linux-specific** and watching is not recursive.

Module contents

- [API](#)
- [Creating a watcher](#)
- [Adding a watch](#)
 - [onlydir](#) option
- [Reading events](#)
 - [Queue overflow](#)
 - [Signal interruption](#)
- [Removing and closing](#)
- [Example](#)
- [Error contract](#)
- [Limits](#)

API

Function	Result
<code>babet.inotify.new()</code>	<code>watcher</code> <code>(nil, err)</code>
<code>w:add(path, events [, opts])</code>	<code>integer wd</code> <code>(nil, err)</code>
<code>w:read([timeout])</code>	<code>event array</code> <code>(nil, "timeout")</code> <code>(nil, "interrupted")</code> <code>(nil, err)</code>
<code>w:remove(wd)</code>	<code>(true, nil)</code> <code>(nil, err)</code>
<code>w:close()</code>	<code>(true, nil)</code>

Signatures are strict: extra arguments raise a Lua error. `close()` is idempotent, and garbage collection also closes a forgotten watcher.

Creating a watcher

```
local watcher, err = babet.inotify.new()
if not watcher then
    error(err)
end
```

Each call creates an independent instance. The `inotify` descriptor uses `IN_NONBLOCK` and `IN_CLOEXEC`: `read()` controls waiting itself, and the descriptor is not inherited by programs launched through `exec`.

Adding a watch

```
local wd, err = watcher:add(path, events [, opts])
```

path must be a string without an embedded NUL byte. It may refer to an existing directory or file.

events is a strict, non-empty 1..N array with no holes or extra keys. Every element must be one of these names:

Name	Requested event
"access"	file read or accessed
"modify"	content modified
"attrib"	attributes or metadata modified
"close_write"	write-open descriptor closed
"close_nowrite"	non-write descriptor closed
"close"	combination of close_write and close_nowrite
"open"	file opened
"moved_from"	entry moved out of the watched directory
"moved_to"	entry moved into the watched directory
"move"	combination of moved_from and moved_to
"create"	entry created in the watched directory
"delete"	entry deleted from the watched directory
"delete_self"	watched target deleted
"move_self"	watched target moved

onlydir option

```
local wd = assert(watcher:add(path, { "create" }, {  
    onlydir = true,  
}))
```

When present, onlydir must be a boolean:

- true adds Linux's IN_ONLYDIR mask and therefore rejects a target that is not a directory;
- false keeps the normal behavior;
- other fields in opts are currently ignored.

Adding a watch again for the same target follows native inotify semantics: the existing mask is replaced because IN_MASK_ADD is not used.

Reading events

```
local events, err = watcher:read([timeout])
```

The timeout is in seconds and may be fractional:

- omitted or nil: wait indefinitely;
- 0: non-blocking read of events already available;
- positive value: bounded wait.

A negative, NaN, infinite, or excessively large timeout returns (nil, err). A value of the wrong type raises a Lua error.

A successful read returns an array of tables:

```
{  
  {  
    wd = 1,  
    name = "photo.jpg",  
    events = { create = true, close_write = true },  
    is_dir = false,  
  },  
  ...  
}
```

```

        cookie = 0,
    },
}

```

- wd is the watch descriptor returned by add;
- name is the affected entry name, or "" for an event on the watched target itself;
- events contains a true boolean for every received bit;
- is_dir reflects IN_ISDIR;
- cookie pairs moved_from with moved_to; outside moves it is usually 0.

Besides events requestable through add, the kernel may produce:

- events.ignored when a watch is removed, automatically or through remove;
- events.unmount when the watched filesystem is unmounted.

One call may return several events because the module reads and decodes a whole batch from the kernel queue.

Queue overflow

If the inotify queue overflows, events have been lost. The module surfaces this explicitly:

```

{
    wd = -1,
    name = "",
    events = { overflow = true },
    is_dir = false,
    cookie = 0,
}

```

The program must then rescan watched paths to rebuild reliable state.

Signal interruption

If a signal handled by `babet.signal` arrives while waiting, its Lua callback runs and `read()` then returns `(nil, "interrupted")`.

Removing and closing

```

assert(watcher:remove(wd))
assert(watcher:close())

```

After `remove(wd)`, the kernel normally queues an ignored event. Removing an invalid descriptor returns `(nil, err)`.

After `close`, `add`, `read`, and `remove` return an error saying the watcher is closed. `close` may be called more than once.

Example

```

local watcher = assert(babet.inotify.new())
local wd = assert(watcher:add("/srv/incoming", {
    "close_write",
    "moved_to",
}), {
    onlydir = true,
}))

while true do
    local events, err = watcher:read()

```

```

    if not events then
        if err == "interrupted" then
            break
        end
        error(err)
    end

    for _, event in ipairs(events) do
        if event.events.overflow then
            rescan_directory()
        elseif event.wd == wd then
            process_entry(event.name)
        end
    end
end

watcher:close()

```

Error contract

- wrong argument count or type: Lua error;
- invalid value, system error, or closed watcher: (nil, err);
- no event before the deadline: (nil, "timeout");
- handled signal during read: (nil, "interrupted");
- successful remove and close: (true, nil).

System errors are prefixed with "inotify: " and retain the description provided by the operating system.

Limits

- Linux only;
- no recursive watching;
- no IN_DONT_FOLLOW, IN_ONESHOT, or IN_MASK_ADD;
- one instance may hold several watches, and several independent instances may coexist in one process.

babet.json — JSON encoding and decoding

The `babet.json` module is backed by **nlohmann/json 3.11.3**. It converts Lua values to JSON text and back, with two sentinels that represent JSON `null` and the empty array `[]` without ambiguity.

Module contents

- [API](#)
- [Type mapping](#)
 - [Lua to JSON](#)
 - [JSON to Lua](#)
- [null](#)
- [Empty object, empty array, and `as\_array`](#)
- [encode options](#)
- [Strings, UTF-8, and NUL bytes](#)
- [Numbers](#)
- [Object order and duplicate keys](#)
- [Error contract](#)
- [Limits](#)

API

Element	Contract
<code>babet.json.encode(value [, opts])</code>	Returns text, nil or nil, err
<code>babet.json.decode(text)</code>	Returns value, nil or nil, err
<code>babet.json.null</code>	Sentinel representing JSON null
<code>babet.json.empty_array</code>	Constant sentinel that encodes as []
<code>babet.json.as_array(t)</code>	Tags t as a JSON array and returns the same table

`encode` accepts exactly one or two arguments. `decode` requires exactly **one Lua string**: a number is not implicitly converted to text. `as_array` requires exactly one table.

Type mapping

Lua to JSON

Lua	JSON
nil passed directly to encode	null
<code>babet.json.null</code>	null
<code>babet.json.empty_array</code>	[]
boolean	boolean
Lua integer	JSON integer number
finite Lua float	JSON floating-point number
valid UTF-8 string	JSON string
untagged empty table	empty object {}
integer keys exactly 1..n	JSON array
string keys only	JSON object

Inside a Lua table, assigning nil removes the key before encoding ever starts. Use `babet.json.null` to retain a key whose JSON value must explicitly be null.

A table mixing string and integer keys, a sparse array, or a table with an unrepresentable key (0, a negative integer, a float, a boolean, a table, and so on) cannot be converted: `encode` returns nil, err.

JSON to Lua

JSON	Lua
null	<code>babet.json.null</code>
boolean	boolean
signed integer in the Lua range	Lua integer
unsigned integer above <code>math.maxinteger</code>	Lua float
JSON floating-point number	Lua float
string	Lua string
array	sequential table 1..n
object	string-keyed table

A JSON integer too large for `lua_Integer` is converted to a float instead of silently wrapping to a negative value. As with any IEEE 754 conversion, precision may be lost.

nil

Lua cannot retain a nil value in a table. The shared `babet.json.null` sentinel fills that gap:

```
local J = babet.json

local value = assert(J.decode([[{"answer":null}]])
assert(value.answer == J.null)

local text = assert(J.encode({ answer = J.null }))
-- text represents {"answer":null}
```

Compare the sentinel by identity with `==`. It is implemented as a special table, but must not be used as a data table.

Empty object, empty array, and as_array

An empty Lua table is ambiguous. Babet chooses a JSON object by default:

```
local J = babet.json

assert(J.encode({}) == "{}")
assert(J.encode(J.empty_array) == "[]")
assert(J.encode(J.as_array({})) == "[]")
```

`empty_array` is a shared sentinel for values that are not meant to be modified. `as_array(t)` is preferable for a dynamically built array: it tags the table with an internal metatable, leaves it mutable, and returns that exact same table.

```
local tags = {}
-- table.insert(tags, ...) may never be called

local text = assert(J.encode({ tags = J.as_array(tags) }))
-- {"tags":[]}
```

Important details:

- `as_array` is idempotent;
- the tag **replaces any existing metatable** on the table;
- the shape is validated only when encode runs: a tagged table with string keys or holes returns `nil, err`;
- `as_array(J.null)` and `as_array(J.empty_array)` raise a Lua error;
- only a decoded **empty** JSON array receives the tag automatically. A decoded non-empty array that is later emptied becomes an ordinary empty table and therefore re-encodes as `{}`; call `as_array(t)` to retain `[]` in that case;
- ordinary writes to the sentinels are rejected. As with every Lua table, `rawset` bypasses metamethods: do not mutate the sentinels.

encode options

The optional table has one interpreted option:

```
local compact = assert(babet.json.encode({ a = 1 }))
local pretty  = assert(babet.json.encode({ a = 1 }, { indent = 4 }))
```

- `indent = 0..256` enables formatted output with line breaks;
- `indent = 0` adds no indentation spaces but keeps formatted-output line breaks;
- omitted, nil, or a negative integer: compact output;
- a non-integer value or a value above 256 raises a Lua error;
- other option fields are ignored. In particular, the previously and incorrectly documented `pretty` option does not exist.

Strings, UTF-8, and NUL bytes

JSON strings must contain valid UTF-8. Unicode characters are preserved in output; Babet does not force them into `\uXXXX` escapes.

Lua strings may contain NUL bytes. They remain binary-safe on the Lua side and are escaped according to JSON rules during encoding. The same applies to object keys:

```
local J = babet.json
local original = "a\0b"
local text = assert(J.encode({ [original] = original }))
local back = assert(J.decode(text))
assert(back[original] == original)
```

A string or key containing invalid UTF-8 makes `encode` return `nil, err`. JSON text containing an invalid UTF-8 string is likewise rejected by `decode`.

Numbers

Babet preserves the Lua numeric subtype when JSON permits it: `42` decodes as a Lua integer, whereas `42.0` and `1e3` decode as floats. When encoding, a Lua float such as `3.0` remains a JSON floating-point number.

`NaN`, `+Inf`, and `-Inf` are not part of the JSON standard and return `nil, err`.

Object order and duplicate keys

JSON object member order has no semantic meaning and must not be treated as an API contract. The current backend generally emits members in lexicographic key order, but callers should compare decoded data rather than raw text whenever an external format does not prescribe ordering.

When decoding a JSON object that repeats the same key, the last value replaces earlier ones.

Error contract

Conversion and parsing failures are returned and do not raise a Lua exception:

```
local value, err = babet.json.decode("{ invalid")
if not value then
    print(err) -- prefixed with "json:"
end
```

This includes, among other cases:

- invalid JSON, trailing data after the document, or invalid UTF-8;
- mixed tables, sparse arrays, or unrepresentable keys;
- a function, userdata, or thread that cannot be encoded;
- `NaN` or infinity;
- a cycle or excessive nesting depth.

API misuse does raise a Lua error:

- wrong argument count;
- a non-string argument to `decode`;
- an `encode` second argument that is neither a table nor `nil`;
- a non-integer `opts.indent` or one above 256;
- an invalid argument to `as_array`.

Parsing errors produced by `nlohmann/json` include the failure position, usually as a line and column.

Limits

Conversion depth is capped at **1000 levels** in both directions. This also acts as the guard against cyclic Lua tables. The module builds the whole document in memory and does not provide a streaming API.

logging — leveled logging

logging is a pure-Lua module bundled with Babet. Load it with `require("logging")`; it is not a member of the babet table.

```
local log = require("logging")
```

It exposes five levels:

Level	Value
log.TRACE	10
log.DEBUG	20
log.INFO	30
log.WARN	40
log.ERROR	50

There is no fatal level.

Module contents

- [Emitting a message](#)
 - [No-propagation guarantee](#)
- [Output format](#)
- [Threshold](#)
- [Destination](#)
- [ANSI colors](#)
- [Module state](#)
- [Error contract](#)
- [Limitations](#)

Emitting a message

```
log.trace(...)
log.debug(...)
log.info(...)
log.warn(...)
log.error(...)
```

Each function accepts zero or more values and returns no values. A message is emitted when the function's level is greater than or equal to the current threshold.

The initial threshold is `log.INFO`, so trace and debug are filtered by default.

Arguments are converted with `tostring` and joined with one space:

```
log.info("user=", 42, "active=", true)
-- ... [INFO ] user= 42 active= true
```

Calling a log function with no arguments emits a line with an empty message. Embedded NUL bytes and newlines are neither removed nor escaped. A multiline message therefore produces several physical lines, but only the start of the call has the logger prefix.

No-propagation guarantee

The five emission functions never raise. The whole pipeline is protected:

- argument conversion through `tostring`;
- timestamp creation through `os.date`;
- the sink's write call.

If any of these operations fails, the message is silently lost and the script continues. Arguments of a filtered message are not passed to `tostring`.

Output format

Without colors, each line uses this format:

YYYY-MM-DD HH:MM:SS [LEVEL] message\n

Example:

2026-07-13 21:45:02 [WARN] attempt 3 of 5

The timestamp uses the machine's **local time**. It includes neither a time zone nor milliseconds. Labels occupy five characters: [TRACE], [DEBUG], [INFO], [WARN], [ERROR].

Threshold

```
log.set_level(level)
local level = log.get_level()
```

`set_level` requires exactly one argument and returns no values. `get_level` accepts no arguments and returns the current threshold.

The level may be:

- one of "trace", "debug", "info", "warn", "error", case-insensitively;
- any finite number, allowing an intermediate threshold such as 25.5.

```
log.set_level("DEBUG")
log.set_level(log.WARN)
log.set_level(25.5)
```

Numeric strings are not coerced and whitespace is not trimmed: "30" and " info " are rejected. NaN, +inf, and -inf are rejected as well.

Destination

```
log.set_output(out)
local out = log.get_output()
```

The initial destination is `io.stderr`.

`set_output` requires exactly one table or userdata exposing a callable write method. A method supplied through `__index` is accepted. The method is then called as `out:write(line)`.

```
local file, err = io.open("app.log", "a")
if not file then
    log.error("cannot open log file:", err)
else
    log.set_output(file)
end
```

The module:

- performs no write during `set_output`;

- never closes the destination;
- never calls flush;
- ignores the value returned by write;
- swallows write errors during emission.

A closed file still exposes a write method and can therefore be installed; subsequent writes will fail silently. Sink lifetime management belongs to the script.

get_output accepts no arguments and returns the exact installed object.

ANSI colors

```
log.set_color(true)
local enabled = log.get_color()
```

Colors are disabled by default. set_color requires exactly one boolean and returns no values. get_color accepts no arguments and returns a boolean.

When colors are enabled:

- trace: ANSI dim;
- debug: cyan;
- info: no color;
- warn: yellow;
- error: red.

The ANSI reset is written before the \n. The module does not detect whether the sink is a terminal; enabling colors is always explicit.

Module state

require("logging") returns the same table until package.loaded.logging is removed. The threshold, sink, and color setting are therefore shared by every user of the module within **one Lua state**.

Each Babet worker owns a separate Lua state, so its logging settings are independent from the main thread and from other workers.

The initial values are:

```
log.set_level(log.INFO)
log.set_output(io.stderr)
log.set_color(false)
```

The module also exposes:

```
log._VERSION      -- "babet logging 1.1.0"
log._DESCRIPTION  -- textual description
```

Error contract

Setters and getters raise a Lua error on wrong arity or an invalid value. They do not return (nil, err).

trace, debug, info, warn, and error never raise and return no values.

Limitations

The module does not provide:

- named or hierarchical loggers;
- per-module thresholds;
- multiple destinations;

- custom or JSON formatting;
- UTC timestamps or sub-second precision;
- file rotation;
- buffered or asynchronous writes;
- protection against a recursive sink that calls the logger again;
- automatic terminal detection.

Use an external tool such as `logrotate` for rotation. For more advanced needs, wrap the module or replace its functions in the script.

SIGNAL — graceful shutdown, reloads, and POSIX signals

`babet.signal` lets the main script install a Lua callback for a small set of POSIX signals, ignore a signal, or restore its operating-system default.

The module covers:

- graceful shutdown on `TERM` and `INT`;
- configuration reload requests on `HUP`;
- two application-defined signals, `USR1` and `USR2`;
- explicitly ignoring or restoring a signal;
- interruption of selected blocking Babet calls;
- deferred, safe callback dispatch in the main Lua thread.

It does not send signals, expose a pending-signal queue, configure POSIX masks, or allow handlers to be installed from a worker.

Module contents

- [Essential conventions](#)
- [API overview](#)
- [Supported signals](#)
- [handle\(name, fn\)](#) — install or replace
- [handle\(name, nil\)](#) — remove a callback
- [ignore\(name\)](#) — ignore
- [default\(name\)](#) — restore the OS default
- [Callback execution](#)
 - [Main thread and safe context](#)
 - [Dispatch order](#)
 - [Occurrence coalescing](#)
 - [Callback errors](#)
- [Interaction with blocking calls](#)
- [Interaction with `debug.sethook`](#)
- [Complete examples](#)
 - [Graceful shutdown on `TERM` and `INT`](#)
 - [Reload configuration with `HUP`](#)
 - [Interrupt a socket wait](#)
 - [Temporarily ignore `PIPE`](#)
 - [Observe callback errors](#)
- [Error contract](#)
- [Workers, concurrency, and limitations](#)
- [Features not provided](#)

Essential conventions

Success returns one value

All three functions return only the boolean `true` on success. They do not return a second `nil`.

```
local ok = babet.signal.ignore("PIPE")
assert(ok == true)
```

A rare sigaction(2) failure instead returns two values:

```
local ok, err = babet.signal.ignore("PIPE")
if not ok then
    io.stderr:write(err, "\n")
end
```

Names are strict

A signal name must be an actual Lua string, with no NUL byte, in the exact case shown by this documentation.

```
assert(babet.signal.handle("TERM", function() end))
```

These forms are invalid:

```
babet.signal.handle("term", function() end) -- wrong case
babet.signal.handle(15, function() end)      -- number, not string
babet.signal.handle("TERM\0X", function() end)
```

A callback receives no arguments

The callback is invoked with no parameters. Install a distinct function for each signal when code needs to know which signal arrived.

```
babet.signal.handle("USR1", function()
    print("USR1")
end)
```

API overview

```
local ok, err = babet.signal.handle(name, callback_or_nil)
local ok, err = babet.signal.ignore(name)
local ok, err = babet.signal.default(name)
```

Function	Success	Rare system failure	Invalid call
handle(name, fn)	true	(nil, err)	raises a Lua error
handle(name, nil)	true	(nil, err)	raises a Lua error
ignore(name)	true	(nil, err)	raises a Lua error
default(name)	true	(nil, err)	raises a Lua error

handle(name) without a second argument is intentionally rejected. Write handle(name, nil) explicitly to remove a callback.

Supported signals

Babet name	POSIX signal	Common use	Usual OS default
"TERM"	SIGTERM	graceful shutdown request, notably from systemd	terminate the process
"INT"	SIGINT	keyboard interruption, usually Ctrl-C	terminate the process

Babet name	POSIX signal	Common use	Usual OS default
"HUP"	SIGHUP	configuration reload request	often terminate the process
"USR1"	SIGUSR1	free application event	terminate the process
"USR2"	SIGUSR2	second free application event	terminate the process
"PIPE"	SIGPIPE	writing to a closed pipe or socket	terminate the process

KILL and STOP cannot be exposed: POSIX does not allow them to be caught or ignored.

Dangerous signals or signals reserved for other internal mechanisms — such as SEGV, BUS, FPE, ILL, CHLD, and ALRM — are not in the allowlist.

handle(name, fn) — install or replace

```
local ok, err = babet.signal.handle(name, fn)
```

- name is one of the six supported names;
- fn must be a Lua function;
- the previous POSIX disposition is replaced;
- an existing Lua callback for the same signal is replaced;
- the callback is retained in the Lua registry until removed.

```
local stopping = false
```

```
assert(babet.signal.handle("TERM", function()
    stopping = true
end))
```

Replacing a callback:

```
assert(babet.signal.handle("USR1", function()
    print("first version")
end))
```

```
assert(babet.signal.handle("USR1", function()
    print("new version")
end))
```

After the second call, only the new function can run.

handle(name, nil) — remove a callback

```
local ok, err = babet.signal.handle(name, nil)
```

This form:

1. removes the associated Lua callback;
2. restores SIG_DFL, the operating-system default.

```
assert(babet.signal.handle("HUP", reload_config))
```

```
-- Later: explicit removal.
```

```
assert(babet.signal.handle("HUP", nil))
```

handle("TERM") without a second argument is **not** shorthand: it raises an error, preventing accidental removal caused by a missing argument.

ignore(name) — ignore

```
local ok, err = babet.signal.ignore(name)
```

ignore installs SIG_IGN and removes any previously associated Lua callback. The kernel then ignores that signal.

```
assert(babet.signal.ignore("PIPE"))
```

This is useful when an application prefers normal write errors over process termination caused by SIGPIPE.

The ignore disposition remains active until a later handle, default, or handle(name, nil) call for the same signal.

default(name) — restore the OS default

```
local ok, err = babet.signal.default(name)
```

default installs SIG_DFL and removes any Lua callback. For TERM, INT, HUP, USR1, USR2, and usually PIPE, the next matching signal terminates the process according to operating-system rules.

```
assert(babet.signal.ignore("PIPE"))
-- ... section where SIGPIPE should be ignored ...
assert(babet.signal.default("PIPE"))
```

handle(name, nil) and default(name) have the same system effect. default is more explicit when no callback is involved.

Callback execution

The real POSIX handler never invokes Lua. It only sets a sig_atomic_t flag, which is safe in asynchronous signal context.

The Lua callback runs later from a safe point:

1. a supported signal arrives;
2. the C handler marks that signal pending;
3. the main thread notices the flag from the instruction hook or when an interrupted blocking call returns;
4. Babet clears the flag;
5. Babet invokes the Lua callback with zero arguments.

Main thread and safe context

The callback always runs in the main Lua state, never in the POSIX handler and never in a worker. It can therefore use ordinary Lua and Babet APIs.

```
babet.signal.handle("USR1", function()
  -- Runs in normal Lua context.
  local now = babet.time.now()
  print("USR1 at", now)
end)
```

That freedom does not mean a callback should be long. A short function that sets a flag or schedules an action remains easier to reason about.

Dispatch order

When several distinct signals are pending at the same dispatch point, Babet processes them in the fixed order of its allowlist:

1. TERM;
2. INT;
3. HUP;
4. USR1;
5. USR2;
6. PIPE.

This is neither callback registration order nor a guarantee of exact kernel arrival order.

For example, if USR1 and then TERM arrive before the next dispatch, the TERM callback runs before the USR1 callback.

Occurrence coalescing

Babet stores one flag per signal type, not a counter or queue. Multiple identical occurrences received before the next dispatch are therefore coalesced into one callback invocation.

USR1, USR1, USR1 before dispatch -> one USR1 callback invocation

A new identical signal received **while** its callback runs can set the flag again and be processed during a later dispatch.

Do not use this module as a reliable event counter. Use a queue, pipe, socket, or another message mechanism when every business event must be retained.

Callback errors

Babet invokes callbacks through `lua_pcall`. An uncaught error is removed from the stack and silently ignored; it is not propagated to the code that was running.

```
babet.signal.handle("USR1", function()
    error("invisible to the caller")
end)
```

Protect the body explicitly to log failures:

```
babet.signal.handle("USR1", function()
    local ok, err = pcall(function()
        refresh_metrics()
    end)
    if not ok then
        io.stderr:write("USR1: ", tostring(err), "\n")
    end
end)
```

Interaction with blocking calls

Babet installs handlers without `SA_RESTART`. Selected internal blocking calls can therefore be interrupted by a handled signal. When the main thread sees a pending Babet signal, it dispatches callbacks and generally returns a typed "interrupted" error.

Integrated call families currently include, among others:

- `babet.sleep(...)`;
- `Inotify watcher:read(...)`;
- TCP connection, accept, and receive waits;
- TLS handshakes and waits.

Example:

```
local data, err = peer:recv(4096, 30)
if not data then
```

```

    if err == "interrupted" then
        -- A callback has already run: re-check application state.
    elseif err == "timeout" then
        -- No bytes arrived before the deadline.
    else
        -- Close or network error.
    end
end
end

```

Not every Babet function is interruptible through this mechanism. `babet.exec`, for example, applies its own timeout and does not expose an "interrupted" result for handled signals.

A syscall can also receive `EINTR` while no Babet-managed signal is pending. In that case internal loops resume waiting instead of reporting a false "interrupted".

Interaction with `debug.sethook`

Outside integrated blocking functions, Babet uses a Lua count hook, triggered about every 10,000 instructions, to inspect pending flags.

Lua supports only one active hook per state. Consequently:

- the first `signal.handle(...)` replaces an existing user hook;
- a later `debug.sethook(...)` replaces Babet's hook;
- after that replacement, callbacks are no longer dispatched by instruction count, though they may still be dispatched when integrated blocking calls return;
- removing every callback with `handle(name, nil)`, `ignore(name)`, or `default(name)` does not uninstall Babet's hook and does not restore a user hook that was previously replaced. The hook remains active, but does nothing while no Babet callback is registered.

Avoid combining `babet.signal` and a custom debug hook in the same Lua state unless this interaction is deliberate and understood.

Complete examples

Graceful shutdown on `TERM` and `INT`

```

local running = true

local function request_stop()
    running = false
end

assert(babet.signal.handle("TERM", request_stop))
assert(babet.signal.handle("INT", request_stop))

while running do
    do_one_iteration()

    local slept, err = babet.sleep(250, "ms")
    if not slept and err ~= "interrupted" then
        io.stderr:write(tostring(err), "\n")
        break
    end
    -- On "interrupted", the callback has already run and the loop
    -- re-checks running.
end

close_resources()

```

For a network loop, use the socket's direct "interrupted" result instead of pcall.

Reload configuration with HUP

It is often better for the callback to set a flag only. Full loading then happens in the normal loop.

```
local running = true
local reload_requested = false

assert(babet.signal.handle("TERM", function()
    running = false
end))

assert(babet.signal.handle("HUP", function()
    reload_requested = true
end))

while running do
    if reload_requested then
        reload_requested = false
        local new_config, err = load_config()
        if new_config then
            config = new_config
        else
            io.stderr:write("reload: ", err, "\n")
        end
    end

    do_one_iteration(config)
end
```

Interrupt a socket wait

```
local running = true

assert(babet.signal.handle("TERM", function()
    running = false
end))

while running do
    local client, err = server:accept(60)
    if client then
        serve(client)
    elseif err == "interrupted" then
        -- The TERM callback may have set running to false.
    elseif err ~= "timeout" then
        io.stderr:write("accept: ", err, "\n")
    end
end

server:close()
```

Temporarily ignore PIPE

```
assert(babet.signal.ignore("PIPE"))

local sent, err = socket:send(payload)
```

```

if not sent then
    io.stderr:write("send: ", err, "\n")
end

```

```

assert(babet.signal.default("PIPE"))

```

Warning: restoring default SIGPIPE behavior means a later write to a closed pipe may terminate the process.

Observe callback errors

```

local function safe_handler(label, fn)
    return function()
        local ok, err = xpcall(fn, debug.traceback)
        if not ok then
            io.stderr:write(label, ": ", tostring(err), "\n")
        end
    end
end

assert(babet.signal.handle("USR2", safe_handler("USR2", function()
    rotate_logs()
end)))

```

Error contract

Raised Lua errors

These situations immediately raise a Lua error:

- call from a worker;
- missing, non-string, or NUL-containing name;
- unsupported name or wrong case;
- handle without a second argument;
- handler other than a function or nil.

```

local ok, err = pcall(function()
    babet.signal.handle("TERM")
end)
assert(not ok)

```

An unsupported name also raises; it does not return (nil, err).

```

local ok, err = pcall(function()
    babet.signal.ignore("QUIT")
end)

```

Returned system error

If sigaction(2) fails, the function returns:

```

nil, "signal: sigaction failed: ..."

```

This is rare with the six allowed signals, but callers may retain the (ok, err) convention when they need to handle it.

Workers, concurrency, and limitations

Signal dispositions are process-wide, not thread-local. Babet therefore enforces these rules:

- handle, ignore, and default are restricted to the main thread;
- using them from a worker raises an explicit error;
- workers block all six supported signals so they remain owned by the main thread;
- workers cannot observe or consume the parent's pending flags.

Control a worker through message queues, not `babet.signal`:

```
local job = assert(babet.workers.spawn([[
    while true do
        local ok, message = worker.recv()
        if not ok then
            return "closed"
        end
        if message == "stop" then
            return "stopped"
        end
    end
]))

-- The main handler converts the signal into an application message.
babet.signal.handle("TERM", function()
    job:send("stop", 0)
end)
```

Keep this callback short. A full queue can make the non-blocking send fail; a robust application also keeps a fallback flag.

Features not provided

The module currently does not provide:

- `kill(pid, name)` to send a signal;
- `list()` to retrieve supported names;
- `is_pending()` or an occurrence counter;
- `sigprocmask` or per-thread masks;
- `signalfd`;
- signals beyond the six listed names;
- a debug hook that can be chained with the user's hook.

An external program can send a signal from Lua:

```
local result, err = babet.exec("kill", {
    "-TERM",
    tostring(pid),
})
assert(result, err)
assert(result.code == 0, result.stderr)
```

Reception remains subject to ordinary POSIX rules and the dispatch model described on this page.

SOCKET — TCP clients and servers, binary streams, lines, and timeouts

`babet.socket` provides synchronous TCP sockets for custom protocols: IRC, plain SMTP before STARTTLS, line-oriented services, binary streams, and small internal daemons.

It covers:

- connecting to a hostname or IP address;

- listening on one interface or all interfaces;
- accepting clients;
- sending a complete binary Lua string;
- receiving chunks, lines, or everything until remote EOF;
- per-socket and per-call deadlines;
- local and peer addresses;
- explicit close and garbage-collector cleanup;
- interruption by signals handled through Babet.

It does not expose UDP, Unix-domain sockets, asynchronous I/O, `select/poll` loops, or an application protocol. Use [HTTP](#) for web requests and [TLS](#) for direct TLS or STARTTLS.

Module contents

- [Core conventions](#)
- [API overview](#)
- [Connecting a client](#)
- [Creating a server](#)
- [Accepting a client](#)
- [Default timeout](#)
- [Sending data](#)
- [Receiving at most N bytes](#)
- [Receiving one line](#)
- [Receiving until EOF](#)
- [Mixing receive methods](#)
- [Local and peer addresses](#)
- [Closing and GC](#)
- [Signals and interruption](#)
- [Complete examples](#)
- [Error contract](#)
- [Security and limitations](#)

Core conventions

Synchronous TCP

Every call runs in the current thread. Without a timeout, `connect`, `accept`, `send`, and `receive` methods can wait indefinitely. Use [workers](#) for independent concurrent connections; Babet does not expose a Lua event loop.

Binary-safe data

Payload strings may contain NUL bytes:

```
assert(sock:send("AB\0CD") == 5)
local data = assert(sock:recv(5, 2))
assert(#data == 5 and data:byte(3) == 0)
```

Hostnames are passed to C/POSIX APIs and may not contain NUL.

Timeout rules

Timeouts are seconds and may be fractional:

- omitted or `nil`: use the socket default;
- `0`: wait forever;
- positive finite number: one deadline for the whole call;
- positive values below 1 ms: rounded up to 1 ms;

- negative, NaN, infinity, or excessively large: error.

The deadline is not restarted for every byte or internal retry.

Return values

Constructors and reads return a value or `(nil, err)`. `set_timeout`, `starttls`, and `close` return exactly `(true, nil)` on success.

Wrong positional types raise a Lua error. Invalid values and runtime failures normally return `(nil, err)`.

API overview

```
local sock, err = babet.socket.connect(host, port, timeout?)
local server, err = babet.socket.listen(host, port, backlog?)

local count, err = sock:send(data)
local data, err = sock:recv(count, timeout?)
local line, err, partial = sock:recv_line(timeout?)
local data, err = sock:recv_all(timeout?, max_bytes?)
local ok, err = sock:set_timeout(seconds)
local addr, err = sock:peer()
local addr, err = sock:sockname()
local ok, err = sock:close()

local client, err = server:accept(timeout?)
```

Argument	Type	Default	Meaning
client host	strict non-empty string	required	DNS name or IP address
listen host	strict string	required	interface; "" means all interfaces
port	strict Lua integer 0..65535	required	TCP port
connect timeout	finite number >= 0	0	connection deadline in seconds
backlog	strict Lua integer 1..INT_MAX	16	requested listen queue size

Numeric strings such as "443" and floats such as 443.0 are rejected for integer fields.

A listening socket rejects stream operations and `peer`; a connected socket rejects `accept`.

`babet.socket.connect(host, port, timeout?)`

```
local sock, err = babet.socket.connect("example.net", 9000, 5)
assert(sock, err)
```

Babet calls `getaddrinfo()` and tries returned IPv4/IPv6 addresses in order. All connection attempts share the same deadline. Synchronous DNS resolution itself is outside that deadline and may add delay.

Use a numeric address to avoid DNS:

```
local sock = assert(babet.socket.connect("127.0.0.1", 9000, 2))
```

The constructor timeout only covers connection establishment. The new socket's default I/O timeout is still zero:

```
local sock = assert(babet.socket.connect(host, port, 3))
assert(sock:set_timeout(10))
```

Typical results:

- timeout: (nil, "timeout");
- handled signal: (nil, "interrupted") after callback dispatch;
- DNS/TCP failure: (nil, "socket: ...");
- empty host or out-of-range port: (nil, err);
- wrong positional type: Lua error.

babet.socket.listen(host, port, backlog?)

Creates a TCP listening socket with SO_REUSEADDR.

Loopback only:

```
local server = assert(babet.socket.listen("127.0.0.1", 9000))
```

All interfaces:

```
local server = assert(babet.socket.listen("", 9000))
```

An empty host uses AI_PASSIVE. Use "0.0.0.0" or ":::" to request a specific address family. Listening on all interfaces exposes the service to the network; add authentication and firewall rules.

Ask the kernel for a free test port:

```
local server = assert(babet.socket.listen("127.0.0.1", 0))
local addr = assert(server:sockname())
print(addr.host, addr.port)
```

Backlog is a kernel request, not a maximum number of active clients:

```
local server = assert(babet.socket.listen("127.0.0.1", 9000, 128))
```

server:accept(timeout?)

```
local client, err = server:accept(10)
```

The positional timeout overrides server:set_timeout() for this call only. The accepted socket has a default I/O timeout of zero; it does not inherit the server timeout:

```
local client = assert(server:accept(10))
assert(client:set_timeout(30))
```

Accepted descriptors are close-on-exec.

sock:set_timeout(seconds)

Sets the default for send, recv, recv_line, recv_all, and accept:

```
assert(sock:set_timeout(3.5))
assert(sock:set_timeout(0)) -- infinite again
```

It does not apply to a completed constructor and does not control STARTTLS; starttls uses opts.timeout.

Per-call overrides:

```
assert(sock:set_timeout(10))
local a, err = sock:recv(4096, 0.2) -- 200 ms
local b, err = sock:recv(4096)      -- 10 s
local c, err = sock:recv(4096, 0)   -- infinite
```

sock:send(data)

Sends the whole string or fails:

```
local count, err = sock:send("PING\r\n")
assert(count, err)
assert(count == 6)
```

Babel loops over partial OS/OpenSSL writes, so a successful count equals #data. Empty and binary strings are accepted. send has no positional timeout and uses the socket default.

If a transport error happens after some bytes were written, no partial count is returned. Protocols needing safe retries must use application-level IDs or acknowledgements.

sock:recv(count, timeout?)

Reads **at most** count bytes and may return fewer bytes while the connection remains open:

```
local chunk, err = sock:recv(4096, 5)
```

count must be a strict Lua integer from 1 through 16 MiB. This is not “read exactly N”. A helper can accumulate:

```
local function recv_exact(sock, wanted, timeout)
    local chunks, total = {}, 0
    while total < wanted do
        local chunk, err = sock:recv(wanted - total, timeout)
        if not chunk then return nil, err end
        chunks[#chunks + 1] = chunk
        total = total + #chunk
    end
    return table.concat(chunks)
end
```

Here the timeout is per recv call. Compute a monotonic application deadline when one budget must cover the whole helper.

EOF before data returns (nil, "closed"); expiration returns (nil, "timeout"); a handled signal returns (nil, "interrupted").

sock:recv_line(timeout?)

Reads through LF and returns the line without LF. A CR immediately before LF is also stripped:

```
hello\n    -> "hello"
hello\r\n -> "hello"
```

The line limit is 8 MiB. Oversized lines return a line too long error and the protocol should generally be considered desynchronised.

EOF before LF returns three values:

```
local line, err, partial = sock:recv_line(5)
if not line and err == "closed" then
    print(partial) -- always a string, possibly empty
end
```

On timeout/interruption, bytes already consumed are retained internally and are delivered first by the next `recv`, `recv_line`, or `recv_all`.

sock:recv_all(timeout?, max_bytes?)

Reads until remote EOF; EOF is the successful terminator:

```
local body, err = sock:recv_all(10)
```

Use this only when connection close frames the message. A persistent peer that keeps the connection open makes this call wait until timeout.

Memory limit:

- default: 64 MiB;
- strict positive integer;
- maximum: 2 GiB.

```
local body, err = sock:recv_all(10, 4 * 1024 * 1024)
```

Limit, timeout, and interruption failures expose no partial result, but already consumed bytes remain buffered. A later call can resume with a larger limit or new deadline:

```
local body, err = sock:recv_all(2, 1024)
if not body and err:find("max_bytes", 1, true) then
    body = assert(sock:recv_all(2, 4096))
end
```

Mixing recv, recv_line, and recv_all

All three methods share one pending-byte buffer. Switching methods after a timeout does not lose or reorder the TCP stream:

```
-- Peer sent "abc" without LF.
local line, err = sock:recv_line(0.1)
assert(line == nil and err == "timeout")
local prefix = assert(sock:recv(3, 1))
assert(prefix == "abc")
```

Still prefer one framing strategy per protocol phase. Mixing helpers is useful for a line header followed by a fixed-size body, but the application must know the exact boundary.

Pending plaintext must be consumed before `starttls`; Babel rejects the upgrade otherwise.

sock:peer() and sock:sockname()

Both return numeric address tables:

```
{ host = "127.0.0.1", port = 9000 }
```

- `peer()` returns the remote endpoint of a connected socket;
- `sockname()` returns the local endpoint and also works on listeners.

No reverse DNS is performed. `peer()` is invalid on a listening socket.

sock:close() and garbage collection

`close()` is idempotent and returns `(true, nil)`:

```
assert(sock:close())
assert(sock:close())
```

Other methods then report a closed socket. The userdata owns its FD and SSL object, so GC eventually cleans forgotten sockets, but GC timing is not a lifecycle guarantee. Close explicitly.

Created and accepted sockets are close-on-exec, so `babet.exec` children do not silently inherit them.

Signals and interruption

A wait interrupted by a signal registered through `babet.signal` may return `(nil, "interrupted")` after the Lua callback is dispatched. Pending bytes from line/all reads are retained.

```
local stopping = false
babet.signal.handle("TERM", function() stopping = true end)
while not stopping do
    local client, err = server:accept(1)
    if client then
        client:close()
    elseif err ~= "timeout" and err ~= "interrupted" then
        error(err)
    end
end
```

Complete examples

Line-oriented echo server

```
local S = babet.socket
local server = assert(S.listen("127.0.0.1", 9000, 64))
assert(server:set_timeout(1))

while true do
    local client, err = server:accept()
    if client then
        assert(client:set_timeout(30))
        local line, read_err = client:recv_line()
        if line then
            assert(client:send("echo: " .. line .. "\n"))
        elseif read_err ~= "closed" then
            io.stderr:write(read_err, "\n")
        end
        client:close()
    elseif err ~= "timeout" then
        error(err)
    end
end
```

Length-prefixed binary client

```
local sock = assert(babet.socket.connect("127.0.0.1", 9000, 3))
assert(sock:set_timeout(5))
assert(sock:send(string.pack(">I4", 5) .. "hello"))
local raw_len = assert(recv_exact(sock, 4, 5))
local len = string.unpack(">I4", raw_len)
local payload = assert(recv_exact(sock, len, 5))
print(payload)
sock:close()
```

Response framed by EOF

```
local sock = assert(babet.socket.connect("127.0.0.1", 9001, 3))
assert(sock:send("request\n"))
local response = assert(sock:recv_all(10, 8 * 1024 * 1024))
print(#response)
```

Error contract

Wrong positional types raise:

```
babet.socket.connect("localhost", "9000")
sock:send(42)
sock:recv(4096.0)
sock:set_timeout("5")
```

Invalid ranges and runtime failures return `(nil, err)`: invalid ports, backlog, timeouts, receive sizes, incompatible methods, closed sockets, DNS/TCP errors, and the stable typed states "timeout", "closed", and "interrupted".

Only compare complete strings for stable typed errors. System messages may vary across libc and kernel versions.

Security and limitations

- Raw TCP has no encryption or peer authentication.
- Set deadlines for peer-controlled input.
- Bind loopback unless remote access is required.
- TCP is a byte stream; send boundaries are not preserved.
- DNS is synchronous and outside the controlled connection deadline.
- No half-close, configurable keepalive, UDP, Unix sockets, or Lua event loop.
- One userdata should not be used concurrently; socket userdata cannot cross a WORKERS message boundary.

babet.sqlite — embedded SQL database

babet.sqlite embeds SQLite 3.53.1 and exposes a compact but complete Lua API: connections, direct SQL execution, lazy iterators, reusable prepared statements, managed transactions, and explicit BLOB binding.

Module contents

- [API](#)
 - [Opening and closing](#)
 - [Direct execution](#)
 - [Reusable prepared statements](#)
 - [Managed transactions](#)
 - [SQL parameters](#)
 - [Explicit BLOB values](#)
 - [SQL text and NUL bytes](#)
- [Row reading and type mapping](#)
- [Lifetime](#)
- [Error contract](#)
- [Complete examples](#)
- [Functions not exposed](#)

API

Opening and closing

Function	Returns
<code>babot.sqlite.open(path, opts?)</code>	db userdata (nil, err)
<code>babot.sqlite.blob(data)</code>	opaque BLOB wrapper
<code>db:close()</code>	(true, nil) — idempotent
<code>db:in_transaction()</code>	boolean (nil, err)

`open` opens or creates a read/write database. The special `":memory:"` path creates a temporary in-memory database. The path must be a string without a NUL byte.

Optional `opts` fields:

Field	Type	Default
<code>wal</code>	boolean — requests PRAGMA journal_mode=WAL	false
<code>busy_timeout</code>	integer from 0 to 3600000 ms	0

`wal = true` requests WAL mode, but SQLite may keep another journal mode when WAL is not applicable, notably for `":memory:"`.

`busy_timeout` asks SQLite to retry for the requested duration when the database is locked. Zero reports SQLITE_BUSY immediately.

`db:in_transaction()` returns true whenever the connection is inside a transaction, whether it was opened by `db:transaction()` or by manual SQL.

Direct execution

Function	Returns
<code>db:exec(sql)</code>	(true, nil) (nil, err) — multiple statements allowed
<code>db:exec(sql, params)</code>	(true, nil) (nil, err) — one statement
<code>db:query(sql, params?)</code>	callable temporary stmt (nil, err)
<code>stmt:close()</code>	(true, nil) — idempotent

db:exec Without `params`, `exec` accepts multiple semicolon-separated statements. They are prepared and executed in order. Execution stops on the first error; earlier statements remain committed unless they were inside an explicit transaction.

With `params`, exactly one statement is accepted. Trailing whitespace, semicolons, and SQL comments do not count as another statement.

A SELECT passed to `exec` is stepped to completion and its rows are ignored. Empty or comment-only SQL is a successful no-op with `exec(sql)` or `exec(sql, {})`.

db:query `query` prepares one statement immediately and returns a **single-use callable iterator**. Execution starts on the first iterator call:

```
for row in db:query("SELECT id, name FROM users ORDER BY id") do
  print(row.id, row.name)
end
```

Each row is a table keyed by column name. The iterator returns `nil` when it is exhausted and automatically finalizes its statement. `stmt:close()` releases it earlier, and garbage collection finalizes an iterator abandoned after `break`.

`query` also accepts DDL and DML. Such a statement runs on the first call and then ends without yielding a row.

Multiple statements are rejected. Empty or comment-only SQL returns an already exhausted iterator.

Reusable prepared statements

Function	Returns
<code>db:prepare(sql)</code>	<code>prepared</code> <code>(nil, err)</code>
<code>prepared:exec(params?)</code>	<code>(true, nil)</code> <code>(nil, err)</code>
<code>prepared:query(params?)</code>	the same prepared userdata <code>(nil, err)</code>
<code>prepared:reset()</code>	<code>(true, nil)</code> <code>(nil, err)</code>
<code>prepared:close()</code>	<code>(true, nil)</code> — idempotent
<code>prepared:finalize()</code>	alias of <code>close()</code>

`db:prepare()` accepts exactly **one non-empty SQL statement**. Preparation is immediate, so an unknown table or column is reported before the first run.

A statement can then be reused as many times as needed:

```
local insert = assert(db:prepare(
    "INSERT INTO files(path, digest) VALUES(?, ?)"
))

for _, file in ipairs(files) do
    assert(insert:exec({ file.path, file.digest }))
end

assert(insert:finalize())
```

`prepared:exec()` resets the statement before binding, steps it to `SQLITE_DONE`, then clears its bindings. A `SELECT` is fully consumed and its rows are ignored.

`prepared:query()` resets and binds the statement, then returns **the same userdata**, whose metatable makes it callable:

```
local select_by_age = assert(db:prepare([
    SELECT id, name FROM users
    WHERE age >= ?
    ORDER BY id
]))

for row in select_by_age:query({ 18 }) do
    print(row.id, row.name)
end

for row in select_by_age:query({ 65 }) do
    print("senior", row.name)
end
```

Natural exhaustion automatically resets the statement and clears all bindings. After breaking early, either call `prepared:reset()`, start another `prepared:query(...)`, or call `prepared:exec(...)`. The latter two operations also reset the previous partial iteration and discard its remaining rows. A

prepared statement can have only one active iteration: reusing the same userdata from a nested loop therefore resets the outer loop.

Calling `prepared()` directly without an active `prepared:query()` simply returns `nil`; it never executes SQL with cleared bindings.

`prepared:reset()` aborts the current run and clears bindings. `close()` and `finalize()` permanently finalize the statement. Calling a closed prepared userdata directly returns `nil`; execution methods return `(nil, "sqlite: statement closed")`.

Managed transactions

```
local ok, result = db:transaction(function(tx)
    assert(tx:exec(
        "INSERT INTO accounts(name, balance) VALUES(?, ?)",
        { "alice", 100 }
    ))

    assert(tx:exec(
        "INSERT INTO audit(message) VALUES(?)",
        { "account created" }
    ))

    return "created"
end, "immediate")
```

```
assert(ok, result)
print(result) -- "created"
```

Signature:

```
ok, ...callback_results = db:transaction(callback [, mode])
-- or
nil, err = db:transaction(callback [, mode])
```

The callback receives the database connection as its only argument. `mode` is one of:

- "deferred" — default;
- "immediate";
- "exclusive".

When the callback returns normally, Babet commits and returns `true` followed by every callback result. A normal `nil` or `false` result **does not request a rollback**. Only a Lua error triggers automatic rollback.

SQLite operations usually return `(nil, err)`, so use `assert` inside the callback to convert operational failures into Lua errors:

```
local ok, err = db:transaction(function(tx)
    assert(tx:exec("INSERT INTO unique_names VALUES(?)", { "alice" }))
    assert(tx:exec("INSERT INTO unique_names VALUES(?)", { "alice" }))
end)
```

-- The second INSERT fails, assert raises, and both writes are rolled back.

Callback errors are caught: Babet attempts `ROLLBACK` and returns `(nil, "sqlite: transaction callback failed: ...")` rather than rethrowing. A failed `COMMIT` also returns `(nil, err)` after a best-effort rollback.

Intentional limitations:

- nested `db:transaction()` helpers are rejected;

- the helper is rejected while a manual SQL transaction is already active;
- `db:close()` is rejected during the callback;
- the callback must not issue its own `BEGIN`, `COMMIT`, or `ROLLBACK`.

Savepoints remain available through raw SQL.

SQL parameters

The following forms are supported by direct and prepared execution:

Placeholder	Lua value
<code>?</code>	<code>params[1]</code> , <code>params[2]</code> , and so on in <code>?</code> order
<code>:name</code> , <code>@name</code> , <code>\$name</code>	<code>params.name</code> with the prefix removed

Positional and named parameters may be mixed:

```
assert(db:exec(
    "INSERT INTO events VALUES (?, :kind, ?)",
    { 42, 1700000000, kind = "start" }
))
```

The table must match all placeholders exactly. Missing or extra values, sparse numeric indices, non-integer numeric indices, and unsupported value types raise a Lua error. Explicit `nil` as the `params` argument is the same as omitting it.

Numbered `?NNN` placeholders are outside the public contract.

Accepted bind values:

Lua type	SQLite value
boolean	INTEGER 0 or 1
integer	INTEGER
non-integer number	REAL
string	TEXT, including embedded NUL bytes
<code>babet.sqlite.blob(data)</code>	binary-safe BLOB

Explicit BLOB values

A normal Lua string is **always** bound with `sqlite3_bind_text`, even for a column with BLOB affinity. `babet.sqlite.blob(data)` explicitly marks the same bytes as a BLOB:

```
local insert = assert(db:prepare(
    "INSERT INTO assets(name, payload) VALUES(?, ?)"
))

assert(insert:exec({
    "icon",
    babet.sqlite.blob("\x89PNG\r\n\x1a\n...")
}))
```

The constructor accepts exactly one Lua string and returns an opaque userdata. Embedded NUL bytes are preserved. `babet.sqlite.blob("")` creates a real zero-byte BLOB rather than `NULL`.

The wrapper is only used for binding. Reading a BLOB column still returns a binary-safe Lua string.

There is no explicit `NULL` sentinel yet; use the SQL literal `NULL` where needed.

SQL text and NUL bytes

SQL text must be a string without a NUL byte. SQLite would treat a NUL as the end of the SQL string and could execute only a prefix, so Babet rejects it before preparation. SQL text is also limited to INT_MAX bytes to match `sqlite3_prepare_v2`.

This restriction applies only to **SQL text**. Bound strings and BLOB values are binary-safe.

Row reading and type mapping

SQLite type	Lua type
INTEGER	integer
REAL	number (float)
TEXT	string
BLOB	binary-safe string, including an empty BLOB
NULL	absent table key (<code>row.col == nil</code> and absent from pairs)

When multiple columns use the same name, the last column wins. Use aliases to preserve every value.

SQLite `length()` counts characters for TEXT and may stop at an embedded NUL. For diagnostic byte counts, use for example `length(CAST(col AS BLOB))`.

Lifetime

`db:close()` is idempotent. After closing, new connection operations return `(nil, "sqlite: connection closed")`.

An iterator or prepared statement created **before** `db:close()` remains usable. Babet uses `sqlite3_close_v2`, so SQLite keeps a zombie connection until the last active statement is finalized.

```
local stmt = assert(db:prepare("SELECT id FROM users ORDER BY id"))
assert(db:close())
```

```
for row in stmt:query() do
    print(row.id)
end
```

```
stmt:finalize()
```

Temporary `db:query` iterators and prepared statements both finalize their handles from `__gc`. Explicit close/finalize is still preferable because it releases locks and resources immediately.

Error contract

Errors fall into two groups:

- operational failures (open, preparation, step, transaction, closed handle) generally return `(nil, "sqlite: <description>")`;
- wrong types, invalid arity, and invalid parameter tables raise a Lua error.

Errors raised while calling a query iterator (`db:query` or `prepared:query`) are Lua errors because the iterator protocol cannot also return a separate `(nil, err)` result.

When no params table is provided, placeholders are detected before execution. Babet never lets SQLite silently bind them as NULL.

`db:transaction()` is the controlled exception: it catches callback Lua errors, rolls back, and converts them to `(nil, err)`.

Complete examples

Prepared inserts and BLOB values

```
local db = assert(babet.sqlite.open("state.db", {
    wal = true,
    busy_timeout = 2000,
}))

assert(db:exec([[
    CREATE TABLE IF NOT EXISTS files (
        path TEXT PRIMARY KEY,
        digest BLOB NOT NULL
    )
]]))

local insert = assert(db:prepare([[
    INSERT INTO files(path, digest)
    VALUES(?, ?)
    ON CONFLICT(path) DO UPDATE SET digest = excluded.digest
]]))

for _, file in ipairs(files) do
    assert(insert:exec({
        file.path,
        babet.sqlite.blob(file.digest),
    }))
end

insert:finalize()
```

Reusable query and managed transaction

```
local by_prefix = assert(db:prepare([[
    SELECT path, digest
    FROM files
    WHERE path LIKE ?
    ORDER BY path
]]))

local ok, count = db:transaction(function(tx)
    local n = 0
    for row in by_prefix:query({ "images/%" }) do
        assert(tx:exec(
            "INSERT INTO audit(path) VALUES(?)",
            { row.path }
        ))
        n = n + 1
    end
    return n
end, "immediate")

assert(ok, count)
print(count, "rows audited")

by_prefix:finalize()
assert(db:close())
```

Functions not exposed

The following are not implemented:

- an explicit `sqlite.NULL` bind sentinel;
- `db:last_insert_rowid()` and `db:changes()`;
- `opts.readonly` and `opts.foreign_keys`;
- a nested savepoint helper;
- the `sqlite3_blob_open` streaming BLOB API;
- the `sqlite3_backup_init` backup API.

`last_insert_rowid`, `changes`, foreign keys, savepoints, and `VACUUM INTO` remain available through raw SQL. FTS5 and R-Tree are not enabled in the current embedded build.

babet strings — splitting strings

Babet exposes one string-manipulation function: `babet.split`. It is registered directly on the `babet` table, without a sub-namespace, to preserve the historical API.

Module contents

- [API](#)
- [Separator mode](#)
 - [Limiting the number of cuts](#)
- [Byte mode](#)
- [Binary strings and NUL bytes](#)
- [Empty subject](#)
- [Error contract](#)
- [Limits and design choices](#)

API

```
local parts = babet.split(s [, sep [, max_splits]])
```

The function requires **one to three arguments** and returns exactly one value: a dense table indexed from 1 to `n`.

Argument	Type and behavior
<code>s</code>	Required Lua string. Numbers are not converted implicitly. Embedded NUL bytes are preserved.
<code>sep</code>	Optional Lua string. Omitted or empty: byte mode. Otherwise it must contain exactly one byte , used literally.
<code>max_splits</code>	Optional Lua integer greater than or equal to -1. -1 means unlimited and 0 prevents every cut. Used only when <code>sep</code> contains one byte.

`nil` is not a placeholder for an optional argument: `babet.split("abc", nil)` raises an error. To supply `max_splits` in byte mode, pass `""` explicitly as the second argument.

Separator mode

When `sep` contains exactly one byte, every occurrence of that byte produces a cut. The separator is **literal**: it is neither a Lua pattern nor a regular expression.

```
babet.split("a,b,c", ",")
-- { "a", "b", "c" }
```

```
babet.split("a.b.c", ".")
-- { "a", "b", "c" } -- the dot is not a wildcard
```

Empty fields are preserved at the beginning, between adjacent separators, and at the end:

```
babet.split(",a", ",")
-- { "", "a" }
```

```
babet.split("a,,b", ",")
-- { "a", "", "b" }
```

```
babet.split("a,", ",")
-- { "a", "" }
```

If the separator does not occur, the result contains the whole subject:

```
babet.split("hello", ",")
-- { "hello" }
```

Limiting the number of cuts

`max_splits` counts **cuts**, not produced elements. The uncut remainder is stored verbatim in the last element.

```
babet.split("a,b,c,d", ",", 2)
-- { "a", "b", "c,d" }
```

```
babet.split("a,b,c", ",", 0)
-- { "a,b,c" }
```

```
babet.split("a,b,c", ",", -1)
-- { "a", "b", "c" }
```

Any value greater than the number of available separators has the same effect as `-1`.

Byte mode

If `sep` is omitted or is `"`, the subject is split byte by byte. There is therefore **no default space separator**.

```
babet.split("ab c")
-- { "a", "b", " ", "c" }
```

```
babet.split("abc", "")
-- { "a", "b", "c" }
```

When supplied in this mode, `max_splits` is still validated but is not used:

```
babet.split("abc", "", 0)
-- { "a", "b", "c" }
```

This mode works on **bytes**, not Unicode code points. A multi-byte UTF-8 character is therefore returned as several one-byte strings:

```
local bytes = babet.split("é")
-- #bytes == 2 in UTF-8
-- table.concat(bytes) == "é"
```

Likewise, a multi-byte UTF-8 separator such as "é" is rejected because sep must contain zero or one byte. Use Lua code (`string.find`, `string.gmatch`, and so on) or a dedicated function to split on a multi-byte string.

Binary strings and NUL bytes

The subject, separator, and output elements are handled with their exact Lua length. A NUL byte never truncates the data:

```
local parts = babet.split("a\0b,c", ",")
-- { "a\0b", "c" }
```

```
local parts2 = babet.split("a\0b", "\0")
-- { "a", "b" }
```

UTF-8 content also stays intact when split with an ASCII byte:

```
babet.split("été:ok", ":")
-- { "été", "ok" }
```

Empty subject

Two different historical behaviors are preserved:

```
babet.split("", ",")
-- { "" }
```

```
babet.split("")
-- {}
```

```
babet.split("", "")
-- {}
```

In separator mode, finding zero separators means returning one element with the whole subject, even when it is empty. In byte mode, a zero-byte subject produces no elements.

Error contract

Every failure is a raised Lua error; `split` has no `(nil, err)` return path.

The function raises in particular when:

- fewer than one or more than three arguments are supplied;
- `s` is not an actual Lua string;
- `sep` is present but is not an actual Lua string;
- `sep` contains more than one byte;
- `max_splits` is not a Lua integer;
- `max_splits` is less than -1.

Implicit number-to-string conversions are not accepted:

```
babet.split(123, ",")          -- error
babet.split("123", 2)          -- error
babet.split("a,b", ",", "1")  -- error
```

Limits and design choices

- This is not a CSV parser: quoting, escaping, and records are not handled.
- Multi-byte separators and Lua patterns are not supported.
- Empty fields are intentionally preserved.

- No join counterpart is exposed because Lua's standard `table.concat` already provides it.

SYS - process, host, environment, executables, version, and Lua memory

The functions in this chapter are exposed directly on the global `babet` table. There is no `babet.sys` subtable: this flat namespace is historical and is part of the stable API.

SYS covers:

- the identity of the Babet process;
- the hostname and kernel information;
- reading and changing environment variables;
- locating an executable program;
- runtime version constants;
- memory used by the current Lua state.

The current working directory is also process-global state, but `currentDir` and `chdir` are documented under [FS - Current working directory](#), alongside path operations.

Module contents

- [General conventions](#)
- [API overview](#)
- [Version constants](#)
- [Identify the process and host](#)
 - [pid](#)
 - [hostname](#)
 - [uname](#)
- [Locate an executable](#)
 - [PATH lookup](#)
 - [Direct path](#)
 - [Missing or non-executable program](#)
- [Environment variables](#)
 - [env](#)
 - [setenv](#)
 - [Empty value versus missing variable](#)
 - [Interaction with workers](#)
- [Lua VM memory](#)
 - [getMemoryUsage](#)
 - [getDetailedMemoryUsage](#)
 - [What is and is not measured](#)
- [Error contract](#)
- [Design decisions and limits](#)

General conventions

Flat functions

All functions are called directly from `babet`:

```
local pid = babet.pid()
local home = babet.env("HOME")
```

The following forms do not exist:

```
-- Incorrect: there is no babet.sys subtable
-- babet.sys.pid()
-- babet.sys.env("HOME")
```

Strict strings

env, setenv, and which require actual Lua strings. Numbers are not converted automatically.

```
local ok, err = pcall(function()
    return babet.env(42)
end)
```

```
-- ok == false; err describes an invalid call
```

An embedded NUL byte is always rejected. This prevents a Lua string from being silently truncated when passed to a C API.

Process-global state

The environment and current working directory belong to the **whole process**, not to a single script or worker.

- setenv changes the environment seen by the runtime and future programs launched through babet.exec;
- chdir changes how all relative paths are resolved;
- both mutations are permanently forbidden after the first babet.workers.spawn().

This restriction prevents data races between threads.

API overview

Function or constant	Result
babet.VERSION	runtime major.minor.patch string
babet.VERSION_MAJOR	integer major component
babet.VERSION_MINOR	integer minor component
babet.VERSION_PATCH	integer patch component
babet.pid()	process PID as an integer
babet.hostname()	hostname, or (nil, err)
babet.uname()	system table, or (nil, err)
babet.which(name)	executable path, or (nil, err)
babet.env(name)	variable value, or nil when missing
babet.setenv(name, value)	(true, nil) or (nil, err)
babet.getMemoryUsage()	current Lua-state memory in bytes
babet.getDetailedMemoryUsage()	two currently identical integers

Version constants

The four constants describe the Babet binary running the script.

```
print(babet.VERSION)      -- for example "2.6.0"
print(babet.VERSION_MAJOR) -- for example 2
print(babet.VERSION_MINOR) -- for example 5
print(babet.VERSION_PATCH) -- for example 0
```

babet.VERSION matches the output of:

```
babet --version
```

The integer components let code compare versions without parsing a string.

```

local function version_at_least(major, minor, patch)
    local current = {
        babet.VERSION_MAJOR,
        babet.VERSION_MINOR,
        babet.VERSION_PATCH,
    }
    local wanted = { major, minor, patch }

    for i = 1, 3 do
        if current[i] ~= wanted[i] then
            return current[i] > wanted[i]
        end
    end
    return true
end

assert(version_at_least(2, 6, 0), "Babet >= 2.6.0 is required")

```

The following consistency is covered by tests:

```

assert(babet.VERSION == string.format(
    "%d.%d.%d",
    babet.VERSION_MAJOR,
    babet.VERSION_MINOR,
    babet.VERSION_PATCH
))

```

Identify the process and host

babet.pid()

Returns the Babet process ID as a strictly positive integer. The call cannot fail on a supported POSIX system.

```

local pid = babet.pid()
print("Babet PID:", pid)

```

Workers are threads in the same process. They therefore see the same PID as the main thread; `pid()` cannot identify an individual worker.

```

local main_pid = babet.pid()
local worker = assert(babet.workers.spawn([
    return babet.pid()
]))

```

```

local joined, worker_pid = worker:join()
assert(joined and worker_pid == main_pid)

```

babet.hostname()

Returns the hostname configured for the machine.

```

local host, err = babet.hostname()
assert(host, err)
print("Host:", host)

```

Signature:

```

local host, err = babet.hostname()

```

Results:

- success: host is a non-empty string and err is nil;
- system failure: (nil, "hostname: ...").

The hostname is not necessarily a fully qualified DNS name. It may be a short locally configured name.

babet.uname()

Returns the five main POSIX uname(2) fields in a table.

```
local info, err = babet.uname()
assert(info, err)
```

```
print("System :", info.sysname)
print("Node   :", info.nodename)
print("Kernel :", info.release)
print("Version:", info.version)
print("Machine:", info.machine)
```

Returned table:

```
{
  sysname = "Linux",
  nodename = "workstation",
  release = "6.12.0-arch1-1",
  version = "#1 SMP PREEMPT_DYNAMIC ...",
  machine = "x86_64",
}
```

Field	Meaning
sysname	operating system or kernel name
nodename	network node name, often close to the hostname
release	kernel release string
version	detailed kernel build string
machine	hardware architecture reported by the kernel

If uname(2) fails, the function returns (nil, "uname: ..."). This is very rare, but it is part of the real contract.

Locate an executable

Signature:

```
local path, err = babet.which(name)
```

name must be a string without an embedded NUL byte.

PATH lookup

When name contains no /, Babet scans the PATH environment variable from left to right and returns the first candidate that is:

- a regular file;
- executable by the current process.

```
local shell, err = babet.which("sh")
assert(shell, err)
print(shell) -- usually /usr/bin/sh or /bin/sh
```

The returned path is normalized to an absolute path whenever the system allows it.

To perform a simple availability check:

```
if babet.which("ffmpeg") then
  print("ffmpeg is available")
end
```

PATH is read at call time. A change made with setenv before workers is therefore visible to later calls.

```
local old_path = babet.env("PATH")
assert(babet.setenv("PATH", "/opt/my-app/bin:/usr/bin"))
local tool = babet.which("my-tool")
```

An empty PATH component, as in :/usr/bin or /bin::usr/bin, represents the current working directory, following the historical Unix convention.

Direct path

When the argument contains /, PATH is not consulted. The path is tested directly.

```
local sh, err = babet.which("/bin/sh")
assert(sh, err)
```

A relative path containing / is accepted too:

```
local tool, err = babet.which("./bin/my-tool")
assert(tool, err)
```

A valid symlink to an executable regular file is accepted because inspection follows the target.

Missing or non-executable program

When no valid executable is found:

```
local path, err = babet.which("missing-program")
-- path == nil
-- err == "which: 'missing-program' not found in PATH"
```

A file that exists but is not executable is treated as not found:

```
local path, err = babet.which("./script-without-x-bit")
assert(path == nil)
print(err)
```

The following are rejected too:

- directories;
- broken symlinks;
- non-regular files;
- files the current process cannot execute.

Environment variables

`babet.env(name)`

Reads a process environment variable.

```
local home = babet.env("HOME")
if home then
  print("HOME:", home)
end
```

Signature:

```
local value = babet.env(name)
```

Results:

- defined variable: its value as a string;
- missing variable: nil only, with no error message.

This enables the common idiom:

```
local port = babet.env("APP_PORT") or "8080"
```

name must be a string. The following forms raise a Lua error:

```
-- babet.env()           -- missing argument
-- babet.env(42)         -- no automatic conversion
-- babet.env({})         -- wrong type
-- babet.env("A\0B")     -- embedded NUL
```

babet.setenv(name, value)

Creates an environment variable or replaces its existing value.

```
local ok, err = babet.setenv("APP_MODE", "production")
assert(ok, err)
assert(babet.env("APP_MODE") == "production")
```

A later call overwrites the value:

```
assert(babet.setenv("APP_MODE", "test"))
assert(babet.env("APP_MODE") == "test")
```

name and value must both be actual Lua strings.

Caller errors, which raise a Lua error:

```
-- babet.setenv()
-- babet.setenv("APP_MODE")
-- babet.setenv(42, "test")
-- babet.setenv("APP_MODE", 42)
```

Runtime validation errors return (nil, err):

```
local ok, err = babet.setenv("BAD=NAME", "x")
-- ok == nil; err reports an invalid name
```

An empty name, a name containing =, or an embedded NUL byte is rejected. A NUL byte in the value is rejected as well.

Empty value versus missing variable

An empty value is still defined:

```
assert(babet.setenv("APP_OPTION", ""))
assert(babet.env("APP_OPTION") == "")
```

It is therefore different from a missing variable:

```
local value = babet.env("MISSING_VARIABLE")
assert(value == nil)
```

Babet does not currently expose unsetenv. A variable cannot be removed with setenv(name, nil): nil is a wrong type and the call raises a Lua error.

Interaction with workers

setenv is allowed only before the first workers.spawn().

```
assert(babet.setenv("APP_MODE", "production"))
```

```
local worker = assert(babet.workers.spawn([[
    return babet.env("APP_MODE")
]]))
```

```
local joined, value = worker:join()
assert(joined and value == "production")
```

After the first spawn, even when the worker has already finished:

```
local ok, err = babet.setenv("APP_MODE", "test")
-- ok == nil
-- err explains that the shared environment can no longer be changed
```

The restriction lasts for the whole process, including when the first spawn failed. Prepare PATH, application variables, and the current directory before starting workers.

Lua VM memory

Both functions first perform a full Lua garbage-collection cycle. The result therefore reflects memory still in use after collection, and the call may cause a pause proportional to the size of the Lua state.

babet.getMemoryUsage()

Returns the number of bytes currently accounted for by the memory manager of the current Lua state.

```
local bytes = babet.getMemoryUsage()
print(string.format("Lua uses %d bytes", bytes))
```

The result is a positive integer.

Example before/after comparison:

```
local before = babet.getMemoryUsage()
```

```
local values = {}
for i = 1, 10000 do
    values[i] = string.rep("x", 100)
end
```

```
local after = babet.getMemoryUsage()
print("Lua change:", after - before, "bytes")
```

This is not a precise profiling tool: allocator behavior, string interning, and GC optimizations may change the result.

babet.getDetailedMemoryUsage()

Returns two integers:

```
local used, total = babet.getDetailedMemoryUsage()
```

In the current version, both values are **strictly identical**:

```
assert(used == total)
```

The second return value is kept only for API stability. Lua does not expose a separate reliable measurement of fragmentation or reserved-but-unused memory.

New code that needs one measurement should prefer:

```
local bytes = babet.getMemoryUsage()
```

What is and is not measured

The measurement covers memory managed by **the Lua state performing the call**.

It does not represent:

- total process RSS;
- the native stack;
- C++ runtime allocations;
- OpenSSL, SQLite, HTTP, or socket buffers;
- executable code and shared libraries;
- Lua states belonging to other workers.

Inside a worker, the function measures only that worker's isolated Lua state. To measure the whole process, use a system facility such as /proc, ps, smem, Valgrind, or an appropriate profiler.

Error contract

Situation	Behavior
missing argument or wrong type	Lua error, recoverable with pcall
NUL in env, setenv, or which	Lua error for env, (nil, err) for setenv and which
missing variable in env	nil only
executable not found	(nil, err)
hostname or uname system error	(nil, err)
invalid setenv or mutation after a worker	(nil, err)
pid	always an integer
memory functions	always one or two integers after a full GC

Handling a runtime failure:

```
local tool, err = babet.which("optional-tool")
if not tool then
    print("Feature disabled:", err)
end
```

Handling an invalid call:

```
local ok, err = pcall(function()
    return babet.env(42)
end)

if not ok then
    print("Programming error:", err)
end
```

Design decisions and limits

- Historical functions remain directly under babet to avoid breaking existing scripts.
- which returns the first executable found, not every candidate.

- which observes the process PATH and current-process permissions; its result may differ between users or environments.
- `env` distinguishes a missing variable (`nil`) from a variable defined as an empty string (`""`).
- `setenv` always overwrites an existing value.
- No `unsetenv` function or complete environment dump is exposed.
- `pid` is the process PID shared by all workers.
- Memory helpers deliberately run a full GC and do not measure the whole process.
- For system accounts, see [User - system users](#).

babet tables — merging and copying Lua tables

Two historical helpers live directly on the `babet` table:

- `babet.mergeTables(...)` builds a new table from several sources;
- `babet.deepCopyTable(t)` recursively copies **values** that are tables.

They operate on entries actually stored in Lua tables. The `__pairs` and `__index` metamethods therefore do not provide virtual fields to merge or copy.

Module contents

- [API](#)
- `babet.mergeTables(t1, t2, ...)`
 - Positive integer keys: concatenation
 - Every other key: last writer wins
 - Shallow merge
- `babet.deepCopyTable(t)`
 - Values and shared graph structure
 - Keys are not copied
 - Metatables and raw traversal
 - Maximum depth
- [Error contract](#)
- [Limits and design choices](#)
- [Not in v1](#)

API

Function	Returns
<code>babet.mergeTables(t1, t2, ...)</code>	one new table
<code>babet.deepCopyTable(t)</code>	one new table

Both functions return exactly one value on success. They never return `(nil, err)`: call errors and depth errors are raised.

babet.mergeTables(t1, t2, ...)

At least two arguments are required and every argument must be a table. The API sets no additional arity limit beyond Lua's general limits.

Sources are not mutated. The result is a plain table without a metatable, even when source tables have one.

Positive integer keys: concatenation

Every key represented by Lua as an integer ≥ 1 is treated as a list position. This includes a key written as `2.0`, which Lua canonicalizes to an integer.

Within each source, such keys are visited in ascending numeric order, then their values are appended to the result. Holes and original indices are compacted:

```
local r = babet.mergeTables(
  { [4] = "d", [2] = "b" },
  { [7] = "g", [1] = "a" }
)

-- r == { "b", "d", "a", "g" }
```

The same list position appearing in several sources is not overwritten: every value is appended.

Every other key: last writer wins

String, boolean, table, function, thread, userdata and light userdata keys, along with floating, zero and negative numeric keys, keep their original identity. A later source overwrites an earlier value for the same key:

```
local key = {}
local r = babet.mergeTables(
  { mode = "safe", [key] = 1, [0] = "a" },
  { mode = "fast", [key] = 2, [0] = "b" }
)

-- r.mode == "fast"
-- r[key] == 2
-- r[0] == "b"
```

Shallow merge

Values are never copied. A subtable placed in the result is the same table as in the source:

```
local nested = { enabled = false }
local r = babet.mergeTables({ nested = nested }, {})

r.nested.enabled = true
-- nested.enabled == true
```

To obtain an independent graph for table values afterwards, use:

```
local isolated = babet.deepCopyTable(
  babet.mergeTables(defaults, overrides)
)
```

`babet.deepCopyTable(t)`

The function requires exactly one table. It creates a new table for the root and for every **value** that is itself a table.

Values and shared graph structure

Cycles reached through values are supported and graph identity is preserved:

```
local shared = { value = 42 }
local source = { a = shared, b = shared }
```

```
source.self = source
```

```
local copy = babet.deepCopyTable(source)
```

```
-- copy ~= source  
-- copy.a ~= shared  
-- copy.a == copy.b  
-- copy.self == copy
```

Non-table values — numbers, strings, booleans, functions, threads, userdata and light userdata — are reused as-is.

Keys are not copied

Every key keeps its original value and identity. In particular, a table used as a key remains the source table:

```
local key = { id = 1 }  
local source = {  
    [key] = "value",  
    key_as_value = key,  
}  
local copy = babet.deepCopyTable(source)  
  
-- copy[key] == "value"           -- original key retained  
-- copy.key_as_value ~= key      -- table value copied  
-- copy[copy.key_as_value] == nil
```

The function is therefore a deep copy of **table values**, not a structural copy of table keys.

Metatables and raw traversal

The actual metatable of every copied table is attached to its copy **by reference**. It is not duplicated. Mutating that metatable therefore affects both source and copy.

Only actually stored pairs are traversed. A field returned through `__index` or invented by `__pairs` is not materialized in the copy. Once the shared metatable is attached, its behavior naturally remains active on the copy.

Maximum depth

The root is at depth 0. Up to **75 descents into new table values** are accepted, for at most 76 tables along one root-inclusive path. A 76th descent into a previously unseen table raises:

Table is too deep to copy (max depth 75 exceeded)

A reference to an already copied table does not create a new level, so a cycle may close exactly at the boundary.

Error contract

Errors are raised through the Lua API for:

- fewer than two arguments to `mergeTables`;
- any non-table argument to `mergeTables`;
- an argument count other than one for `deepCopyTable`;
- a non-table argument to `deepCopyTable`;
- exceeding the maximum depth.

```
local ok, err = pcall(babet.deepCopyTable, 42)
-- ok == false; err contains "Argument must be a table"
```

Limits and design choices

- mergeTables is not a deep merge: a later subtable replaces the earlier reference wholesale.
- Source metatables are ignored by mergeTables; its result has none.
- deepCopyTable shares metatables and table keys with the source.
- A weak metatable (__mode) remains weak on the copy because it is shared.
- Map-key order is undefined. Only the positive-integer ordering of mergeTables is guaranteed.
- The ordered list-key selection in mergeTables favors safety across Lua errors; its cost is quadratic per source in the number of positive integer keys. This helper is therefore intended mainly for moderately sized tables.

Not in v1

Features such as deepMergeTables or deep structural comparison could be added separately. They are not part of the current API.

babet time - clocks, sleep, dates and durations

Babet exposes two sub-second clocks, interruptible sleep, and small utilities for ISO 8601 timestamps and compact durations.

The historical functions remain available at the root:

- `babet.monotonic()`;
- `babet.now()`;
- `babet.sleep(amount [, unit])`.

The same functions are available under `babet.time`, together with four additional utilities: `iso`, `parse_iso`, `parse_duration`, and `format_duration`.

Module contents

- [API](#)
- `monotonic()` and `now()`
- `sleep(amount [, unit])`
- `babet.time.iso([ts])`
- `babet.time.parse_iso(text)`
- `babet.time.parse_duration(text)`
- `babet.time.format_duration(seconds)`
- [Error contract summary](#)
- [Limits and design choices](#)

API

Function	Result
<code>babet.monotonic()</code>	number, or exceptionally (nil, err) if <code>clock_gettime</code> fails
<code>babet.now()</code>	number, or exceptionally (nil, err) if <code>clock_gettime</code> fails
<code>babet.sleep(amount [, unit])</code>	(true, nil) or (nil, err)
<code>babet.time.iso([ts])</code>	UTC string
<code>babet.time.parse_iso(text)</code>	(integer, nil) or (nil, "parse_iso: ...")

Function	Result
<code>babet.time.parse_duration(text)</code>	(integer, nil) or (nil, "parse_duration: ...")
<code>babet.time.format_duration(seconds)</code>	compact string, for example "1d2h3m4s"

The `babet.time.now`, `babet.time.monotonic`, and `babet.time.sleep` aliases use exactly the same bindings and contract as their root names.

monotonic() and now()

```
local started = babet.monotonic()
do_something()
local elapsed = babet.monotonic() - started
print(string.format("elapsed: %.3f s", elapsed))

local timestamp = babet.now()
print(timestamp)
```

`monotonic()` uses `CLOCK_MONOTONIC`. Its value starts at an arbitrary origin, usually related to system boot. It is intended for measuring durations because it does not suffer wall-clock jumps.

`now()` uses `CLOCK_REALTIME`. It returns POSIX time in seconds since 1970-01-01 UTC, including a fractional part. It is appropriate for timestamps, but not for measuring durations because the wall clock may be adjusted.

Both functions:

- accept no arguments;
- return one number in normal operation;
- may theoretically return (nil, err) if the `clock_gettime` system call fails.

sleep(amount [, unit])

```
assert(babet.sleep(250, "ms"))
assert(babet.time.sleep(0.5))      -- 0.5 second
assert(babet.sleep(1000, "us"))
```

`amount` must be an actual Lua number, finite and non-negative. Floating-point values are accepted. A numeric string such as "100" is rejected.

`unit` must be an actual Lua string:

Unit	Meaning
"s"	seconds, the default
"ms"	milliseconds
"us"	microseconds

A zero duration succeeds immediately. An unknown textual unit keeps the historical behavior:

```
local ok, err = babet.sleep(1, "minutes")
-- ok == nil
-- err == "Invalid time unit"
```

If a signal managed by `babet.signal` arrives during the wait, its Lua callback is dispatched and `sleep` then returns (nil, "interrupted"). An unmanaged signal that causes `EINTR` does not shorten the wait: Babet resumes with the remaining time.

Any other `nanosleep` failure returns (nil, "sleep: <description>").

babet.time.iso([ts])

iso formats a Unix second in UTC:

```
print(babet.time.iso(0))  
-- 1970-01-01T00:00:00Z  
  
print(babet.time.iso())  
-- current time, at whole-second precision
```

Without an argument, iso() uses the current time. With an argument:

- ts must be an actual Lua number;
- an integer or finite float is accepted;
- a fractional value is rounded toward negative infinity;
- the value must fit in a signed 64-bit integer.

The rounding matters before the Unix epoch:

```
babet.time.iso(0.9)    -- 1970-01-01T00:00:00Z  
babet.time.iso(-0.5)   -- 1969-12-31T23:59:59Z
```

For ordinary years, the result has the form YYYY-MM-DDTHH:MM:SSZ. Extreme valid int64 timestamps may produce a signed year wider than four digits. That extended result is not necessarily accepted by parse_iso, whose grammar requires an exactly four-digit year.

iso applies neither locale nor local timezone and does not retain fractional seconds.

babet.time.parse_iso(text)

The parser intentionally accepts a small, deterministic subset:

```
YYYY-MM-DDTHH:MM:SSZ  
YYYY-MM-DD HH:MM:SSZ  
YYYY-MM-DDTHH:MM:SS.fraction+HH:MM  
YYYY-MM-DDTHH:MM:SS.fraction-HH:MM
```

Exact rules:

- text must be an actual Lua string;
- the year contains exactly four digits and ranges from 0001 to 9999;
- the Gregorian calendar is validated, including leap years;
- the separator is uppercase T or one space;
- hours 00..23, minutes 00..59, seconds 00..59;
- leap seconds written as :60 are rejected;
- a fraction is optional, but the dot must be followed by at least one digit; all fraction digits are ignored, without rounding;
- a timezone is required: uppercase Z, +HH:MM, or -HH:MM;
- accepted offsets range from 00:00 through 23:59;
- +0200, +02, lowercase z, leading or trailing whitespace, and trailing data are rejected;
- an embedded NUL byte is never truncated: it makes the string invalid.

Example:

```
local ts = assert(babet.time.parse_iso(  
    "2026-06-17T10:00:00+02:00")  
  
assert(ts == babet.time.parse_iso("2026-06-17T08:00:00Z"))
```

A syntactically or calendrically invalid string returns (nil, "parse_iso: ..."). A wrong type or argument count raises a Lua error.

babet.time.parse_duration(text)

The format is a whitespace-free sequence of number + unit chunks:

```
assert(babet.time.parse_duration("45s") == 45)
assert(babet.time.parse_duration("1h30m") == 5400)
assert(babet.time.parse_duration("2d12h") == 216000)
```

Available units:

Unit	Seconds
d	86400
h	3600
m	60
s	1

Each unit may occur at most once and units must be in strict order $d > h > m > s$. Signs, decimal values, whitespace, and the units *w*, *ms*, *us*, or *ns* are rejected. An embedded NUL byte is not truncated and also produces a parse error.

Components are not normalized: "1h90m" is valid and equals 9000 seconds. Leading zeroes and ordered zero-valued chunks are also accepted, for example "0001s" and "0d0h0m0s".

The total must remain between 0 and `math.maxinteger` on Babet's 64-bit configuration. Overflow returns a parse error.

An invalid string returns `(nil, "parse_duration: ...")`. A wrong type or argument count raises a Lua error.

babet.time.format_duration(seconds)

```
babet.time.format_duration(0)      -- "0s"
babet.time.format_duration(90)     -- "1m30s"
babet.time.format_duration(90061)  -- "1d1h1m1s"
```

`seconds` must be an actual Lua number with a non-negative integer value. A floating-point value such as 3.0 is accepted; 3.7, "3", NaN, infinity, and negative values raise a Lua error.

The output is Babet's canonical representation:

- units in d, h, m, s order;
- zero components omitted;
- zero represented as "0s".

For every valid integer $n \geq 0$:

```
assert(babet.time.parse_duration(
    babet.time.format_duration(n)) == n)
```

The textual reverse is not guaranteed because the parser also accepts non-normalized forms such as "90m" or "0h".

Error contract summary

Situation	Behavior
Wrong type or wrong argument count	Lua error
NaN, infinity, negative duration, or out-of-range numeric value	Lua error
Unknown textual unit in sleep	<code>(nil, "Invalid time unit")</code>

Situation	Behavior
Managed signal during sleep	(nil, "interrupted") after callback dispatch
Invalid ISO or duration text	(nil, "parse_iso: ...") or (nil, "parse_duration: ...")
Clock or sleep system failure	(nil, err)

Limits and design choices

- Use `monotonic()` for durations and `now()` for timestamps.
- `iso` is a fixed UTC formatter, not a general replacement for `os.date`.
- No named timezone, daylight-saving rule, locale, or calendar arithmetic is provided.
- ISO fractions are discarded and are not emitted by `iso`.
- `parse_duration` works with whole seconds only.
- `CLOCK_BOOTTIME`, negative durations, and week/month/year units are not exposed.

TLS — direct encryption, STARTTLS, certificates, and SNI

Babet TLS uses the same userdata as `babet.socket`. After the handshake, `send`, `recv`, `recv_line`, `recv_all`, `set_timeout`, `peer`, `sockname`, and `close` behave like their plain-TCP versions.

Two modes are available:

- `babet.socket.connect_tls()` starts TLS on the first application byte;
- `sock:starttls()` upgrades an existing TCP connection after a plaintext protocol negotiation.

The implementation provides TLS 1.2 minimum, certificate verification by default, hostname checks, SNI independent from verification, system and per-call CA trust, shared TCP+handshake deadlines, and fail-closed STARTTLS. It does not expose TLS servers, client certificates, ALPN, public-key pinning, or peer-certificate inspection.

Module contents

- [Core conventions](#)
- [API overview](#)
- [TLS options](#)
- [Direct TLS connection](#)
- [STARTTLS upgrade](#)
- [Verification, hostname, and SNI](#)
- [Certificate authorities](#)
- [TLS versions](#)
- [Timeouts](#)
- [Socket state after failure](#)
- [Complete examples](#)
- [Error contract](#)
- [Security and limitations](#)

Core conventions

Verification is on by default

```
local sock = assert(babet.socket.connect_tls("example.com", 443, {
    timeout = 10,
}))
```

`verify = false` explicitly disables both chain and hostname verification. It may be useful in controlled tests but encryption without authentication does not prevent an active man-in-the-middle attack.

Same stream API after handshake

```
local sock = assert(babet.socket.connect_tls("irc.example.net", 6697, {
    timeout = 10,
}))
assert(sock:set_timeout(120))
assert(sock:send("PING :hello\r\n"))
local line = assert(sock:recv_line())
```

Handshake timeout and later I/O timeout are separate.

SNI is not certificate verification

SNI selects a TLS virtual host. Verification checks trust and reference identity. Babet sends DNS SNI whenever a DNS hostname is available, including when `verify = false`.

API overview

```
local sock, err = babet.socket.connect_tls(host, port, opts?)
local ok, err = sock:starttls(opts?)
{
    verify = true,
    hostname = "example.com",
    ca_cert = "/path/ca-bundle.pem",
    ca_path = "/path/certs",
    min_version = "1.2",
    timeout = 10,
}
```

Field	Type	Default	Meaning
verify	strict boolean	true	chain and hostname verification
hostname	string	connect_tls host	reference name and DNS SNI
ca_cert	string	no custom file	extra PEM trust file
ca_path	string	no custom directory	OpenSSL hashed CA directory
min_version	"1.2" or "1.3"	"1.2"	minimum protocol version
timeout	finite number >= 0	0	handshake budget in seconds

Unknown fields are currently ignored; misspellings do not fail.

TLS options

`verify` must be an actual boolean. `hostname`, `ca_cert`, `ca_path`, and `min_version` must be strings without NUL. `timeout` must be finite and non-negative; a positive value below 1 ms is rounded up.

`hostname` has two roles:

1. OpenSSL reference identity when verification is enabled;

2. SNI value when it is a DNS name.

Connect to an IP while validating a DNS certificate:

```
local sock = assert(babet.socket.connect_tls("203.0.113.10", 443, {
    hostname = "api.example.com",
    timeout = 10,
}))
```

An IP literal is not sent as SNI. `ca_cert` and `ca_path` augment trust for that connection; they are not exclusive pinning.

`min_version = "1.3"` raises the minimum. It does not cap the maximum.

babet.socket.connect_tls(host, port, opts?)

Establishes TCP then performs a client handshake:

```
local sock, err = babet.socket.connect_tls("example.com", 443, {
    timeout = 10,
})
assert(sock, err)
```

Without `opts.hostname`, the host argument is used for verification and DNS SNI. For an IP connection to a DNS certificate, supply the name explicitly.

After synchronous DNS resolution, one deadline covers every TCP address attempt and the TLS handshake. The handshake does not receive a fresh budget. DNS itself remains outside this controlled deadline.

`opts.timeout` is not retained as the socket I/O default:

```
local sock = assert(babet.socket.connect_tls(host, port, { timeout = 10 }))
assert(sock:set_timeout(30))
```

sock:starttls(opts?)

Upgrades an existing connected plain TCP socket in place:

```
local ok, err = sock:starttls({
    hostname = "mail.example.com",
    timeout = 10,
})
```

On success it returns exactly `(true, nil)`.

The socket must be open, connected, not already TLS, and have no pending plaintext buffered by Babet. `starttls` does not remember the hostname passed to the earlier TCP constructor, so hostname is mandatory when `verify = true`.

Babet does not send a protocol STARTTLS command. The script must:

1. negotiate in plaintext;
2. request STARTTLS;
3. validate the positive protocol reply;
4. call `starttls`;
5. resume the protocol over TLS, often with a new greeting.

If a prior line/all read timed out after consuming plaintext, finish consuming or otherwise resolve that plaintext before upgrading. Babet rejects STARTTLS while the shared pending buffer is non-empty.

Verification, hostname, and SNI

Connection	verify	hostname	Identity check	SNI
DNS example.com	true	absent	example.com	example.com
IP	true	api.example.com	DNS name	DNS name
IP	true	absent	IP identity	none
DNS	false	absent	none	DNS host
IP	false	DNS name	none	DNS name
IP	false	absent	none	none

Providing hostname can therefore still be required by a virtual host even when verification is disabled.

Certificate authorities

The socket TLS context attempts OpenSSL defaults, OpenSSL environment variables, and several known Linux/BSD CA locations. Per-call `ca_cert` and `ca_path` are then added.

Custom trust is isolated per connection:

```
local a = assert(babet.socket.connect_tls("internal.example", 443, {
  ca_cert = "/tmp/test-root.pem",
  timeout = 5,
}))
a:close()
-- Later calls do not retain /tmp/test-root.pem.
```

`ca_path` must follow OpenSSL's hashed-directory conventions. A CA file trusts certificates issued by that CA; it is not a leaf-certificate/public-key pin. Babet does not expose the peer certificate for custom pinning.

TLS versions

Protocols older than TLS 1.2 are always rejected.

- default minimum: TLS 1.2;
- optional minimum: TLS 1.3;
- TLS 1.3 is negotiated automatically with the default when supported.

No maximum-version option is exposed.

Timeouts

For `connect_tls`, `opts.timeout` covers TCP plus handshake after DNS. For `starttls`, it covers only the upgrade handshake and does not inherit `sock:set_timeout()`.

After success, configure I/O separately:

```
assert(sock:set_timeout(30))
local line, err = sock:recv_line()
```

TLS sockets remain non-blocking internally and OpenSSL `WANT_READ/WANT_WRITE` is coordinated with one poll deadline.

Socket state after failure

`connect_tls` returns no socket and closes all intermediate resources.

Pre-handshake STARTTLS validation/configuration failures leave the plain TCP socket usable: invalid options, missing hostname, pending plaintext, context setup, or pre-handshake fcntl failure.

Once SSL_connect begins, any failure closes the socket: timeout, interruption, certificate failure, alert, protocol, or I/O error. A ClientHello may already have been sent and peer bytes consumed, so falling back to plaintext would be unsafe. This is intentionally fail-closed.

Complete examples

Raw HTTPS request

For normal HTTP use babette.org; this demonstrates stream use:

```
local sock = assert(babet.socket.connect_tls("example.com", 443, {
    timeout = 10,
}))
assert(sock:set_timeout(10))
assert(sock:send(
    "GET / HTTP/1.1\r\n" ..
    "Host: example.com\r\n" ..
    "Connection: close\r\n\r\n"
))
local response = assert(sock:recv_all(nil, 8 * 1024 * 1024))
print(response)
sock:close()
```

IP connection with DNS identity

```
local sock = assert(babet.socket.connect_tls("203.0.113.20", 443, {
    hostname = "api.example.com",
    ca_cert = "/etc/myapp/ca.pem",
    timeout = 5,
}))
```

Simplified SMTP STARTTLS

```
local sock = assert(babet.socket.connect("mail.example.com", 587, 5))
assert(sock:set_timeout(10))
assert(sock:recv_line():match("^220"))
assert(sock:send("EHLO client.example\r\n"))
-- Parse the full multiline EHLO response and require STARTTLS.
assert(sock:send("STARTTLS\r\n"))
assert(sock:recv_line():match("^220"))
assert(sock:starttls({
    hostname = "mail.example.com",
    timeout = 10,
}))
assert(sock:send("EHLO client.example\r\n"))
```

A complete SMTP client must correctly parse every multiline reply and prevent STARTTLS downgrade.

Controlled self-signed test

```
local sock = assert(babet.socket.connect_tls("127.0.0.1", 8443, {
    verify = false,
    hostname = "dev.local", -- SNI is still sent
```

```
    timeout = 3,  
  })
```

Do not use this trust policy in production.

Error contract

Wrong positional types raise; invalid values and runtime failures return `(nil, err)`: empty host, bad port/options, CA loading, DNS/TCP, timeout, interruption, handshake, certificate/hostname verification, and STARTTLS preconditions.

OpenSSL wording varies. Compare stable states such as "timeout" and "interrupted"; log full messages for diagnosis.

Security and limitations

- Keep `verify = true` in production.
- Validate against a trusted expected hostname, never one supplied by the peer.
- STARTTLS must be required by application policy to prevent stripping.
- No ALPN, client certificate, TLS server, peer-certificate access, custom cipher policy, or application OCSP/CRL policy.
- Protect and rotate private CA files.

babet.toml - TOML decoding

The `babet.toml` module uses **toml++ 3.4.0** and decodes documents that conform to **TOML 1.0.0**. Optional extensions outside the TOML 1.0 profile supported by this toml++ version are disabled.

The v1 API only decodes a Lua string. It does not read a file for you and it does not provide an encoder.

Module contents

- [API](#)
- [Type mapping](#)
 - [Integers and floats](#)
- [Strings, UTF-8, and keys](#)
- [Arrays and tables](#)
 - [Empty-container ambiguity](#)
- [Dates and times](#)
- [Ordering and discarded information](#)
- [Error contract](#)
- [Limits](#)
- [Not exposed in v1](#)

API

Function	Contract
<code>babet.toml.decode(text)</code>	Returns table, nil or nil, err

`decode` requires exactly **one argument**, and it must be an actual Lua string. A number is not implicitly converted to text, and extra arguments are rejected.

To read a file:

```

local file = assert(io.open("config.toml", "rb"))
local text = assert(file:read("*a"))
file:close()

```

```

local config, err = babet.toml.decode(text)
if not config then
    error(err)
end

```

A TOML document always has a table at its root. An empty string or a document containing comments only therefore returns an empty Lua table.

Type mapping

TOML	Lua
string	string
signed 64-bit integer	Lua integer
float	Lua number
boolean	boolean
array	1..n sequence table
table, inline table, or dotted table	table with string keys
array of tables	sequence table containing tables
local date/time and date-time	normalized string

TOML has no value equivalent to Lua `nil` or JSON `null`.

Integers and floats

TOML integers cover the full signed 64-bit range, from `math.mininteger` to `math.maxinteger`, and are preserved exactly. Decimal, hexadecimal, octal, and binary notation all produce a Lua integer. A value outside that range is a parse error.

Floats become Lua numbers. TOML also accepts these special values:

```

local values = assert(babet.toml.decode([[
positive = inf
negative = -inf
invalid = nan
]]))

assert(values.positive == math.huge)
assert(values.negative == -math.huge)
assert(values.invalid ~= values.invalid) -- NaN property

```

As in Lua, NaN compares equal to no value, including itself.

Strings, UTF-8, and keys

The TOML document must contain valid UTF-8. Basic, literal, and multiline strings are decoded before they are returned to Lua: TOML escape sequences become their corresponding characters.

Lua strings are binary-safe. A TOML sequence such as `\u0000` may therefore produce a NUL byte in a value **or in a quoted key**, without truncation:

```

local value = assert(babet.toml.decode([[
"a\u0000b" = "x\u0000y"
]]))

```

```
assert(value["a\0b"] == "x\0y")
```

A raw NUL byte injected into the TOML text is not treated as the end of a C string: the parser receives the whole buffer and rejects the document. The same applies to invalid UTF-8.

Bare, quoted, and dotted keys follow TOML rules:

```
local value = assert(babet.toml.decode([[
"a.b" = 1
site."google.com" = true
physical.color = "orange"
]]))

assert(value["a.b"] == 1)           -- literal dot in a quoted key
assert(value.site["google.com"] == true)
assert(value.physical.color == "orange")
```

An empty quoted key ("") is valid TOML and becomes the Lua key "".

Arrays and tables

TOML arrays become Lua sequences indexed from 1. TOML 1.0 permits heterogeneous arrays:

```
local value = assert(babet.toml.decode([[
items = [1, "two", true, { name = "three" }]
]]))

assert(value.items[1] == 1)
assert(value.items[2] == "two")
assert(value.items[3] == true)
assert(value.items[4].name == "three")
```

Sections, inline tables, and dotted keys all become ordinary Lua tables. The original syntax is not retained.

Empty-container ambiguity

An empty array [] and an empty table {} both become an empty Lua table:

```
local value = assert(babet.toml.decode([[
a = []
b = {}
]]))

assert(type(value.a) == "table" and next(value.a) == nil)
assert(type(value.b) == "table" and next(value.b) == nil)
```

No metatable or sentinel remembers the original TOML type. This information loss is usually harmless when reading configuration, but it prevents a faithful round trip.

Dates and times

TOML defines four temporal types. Since Lua has no native date type, Babet converts them to strings:

TOML type	Lua form
local date	YYYY-MM-DD
local time	HH:MM:SS[.fraction]

TOML type	Lua form
local date-time	YYYY-MM-DDTHH:MM:SS[.fraction]
offset date-time	YYYY-MM-DDTHH:MM:SS[.fraction]Z or +/-HH:MM

```
local value = assert(babet.toml.decode([[
date = 1979-05-27
time = 07:32:00.1234
local_dt = 1979-05-27T07:32:00
utc_dt = 1979-05-27T07:32:00Z
]]))
```

The temporal value is retained, but its original **spelling** may be normalized by toml++: canonical separator, fractional part, or UTC notation. Comments and the exact source representation are not available. After conversion, a TOML date is also indistinguishable from a TOML string containing the same text.

Ordering and discarded information

A TOML table has no guaranteed semantic order, and Lua table iteration must not be used as a proxy for source order. Decoding does not preserve:

- comments;
- lexical key order;
- whitespace and line breaks;
- the choice between sections, inline tables, and dotted keys;
- the original spelling of numbers and dates;
- the type of an empty container.

The module is intended to **read values**, not to edit and reproduce a TOML document byte-for-byte.

Error contract

A TOML syntax or value error returns:

```
local value, err = babet.toml.decode("answer = ")
assert(value == nil)
print(err) -- toml: ... (line ..., col ...)
```

The message is prefixed with toml: and includes the line and column reported by toml++.

The following are returned as (nil, err), among others:

- invalid syntax or a redefined key;
- integer outside the signed 64-bit range;
- invalid UTF-8 or a forbidden control character;
- an extension unavailable in the TOML 1.0 profile used by Babet.

API misuse raises a Lua error instead:

- missing argument;
- non-string argument;
- extra argument.

Limits

The whole document is parsed and converted in memory. There is no streaming API, decode_file, schema validation, or access to source positions for individual values. Recursive conversion protects the Lua stack; a depth that cannot be converted is returned as a toml: error.

Not exposed in v1

- `babet.toml.encode`;
- `babet.toml.decode_file`;
- parser options or extensions outside TOML 1.0;
- nodes retaining comments, order, and source positions;
- sentinels distinguishing empty arrays from empty tables;
- schema validation.

USER - look up system accounts by name or UID

The `babet.user` module queries the system user database through NSS. It can:

- look up an account by name;
- look up an account by UID;
- quickly test whether an account exists;
- retrieve the name, UID, primary GID, GECOS field, home directory, and shell.

It never parses `/etc/passwd` directly. It uses the same resolution mechanisms as `id` or `getent passwd`, so it can see accounts supplied by LDAP, SSSD, NIS, FreeIPA, or other sources configured in `/etc/nsswitch.conf`.

Module contents

- [General conventions](#)
- [API overview](#)
- [Returned user table](#)
- [Look up an account with `get`](#)
 - [By name](#)
 - [By UID](#)
 - [Missing account](#)
- [Test existence with `exists`](#)
 - [By name](#)
 - [By UID](#)
 - [When to prefer `get`](#)
- [Argument validation](#)
- [NSS, workers, and security](#)
- [Error contract](#)
- [Design decisions and limits](#)

General conventions

Dedicated subtable

Unlike the historical `SYS` or `FS` functions, user helpers live under `babet.user`:

```
local user = babet.user.get("root")
local exists = babet.user.exists("root")
```

Name or UID, without implicit conversion

The argument accepts exactly:

- a Lua string: lookup by **name**;
- a non-negative Lua integer in the `uid_t` range: lookup by **UID**.

A numeric string remains a name:

```
-- Looks for an account literally named "1000"
local by_name = babet.user.get("1000")

-- Looks up numeric UID 1000
local by_uid = babet.user.get(1000)
```

Floats, booleans, tables, nil, negative UIDs, and oversized UIDs raise a Lua error. No silent conversion or truncation is performed.

Fields are always present

When a user is found, the returned table always contains all six documented fields. Text fields may be empty strings, but never nil.

API overview

Function	Result
<code>babet.user.get(name_or_uid)</code>	table, (nil, "user not found"), or (nil, "user: ...")
<code>babet.user.exists(name_or_uid)</code>	strict boolean

Returned user table

Typical example:

```
{
  name  = "www-data",
  uid   = 33,
  gid   = 33,
  gecos = "www-data",
  home  = "/var/www",
  shell = "/usr/sbin/nologin",
}
```

Field	Type	Meaning
name	string	canonical account name returned by NSS
uid	integer	numeric user identifier
gid	integer	account's primary GID
gecos	string	raw descriptive field, often the full name
home	string	configured home directory
shell	string	configured login shell

gid is not a list of supplementary groups. The module does not perform group lookups and does not expose additional memberships.

The gecos field is returned unchanged. Its historical format can contain comma-separated values, but many systems simply store a free-form name.

```
local u = assert(babet.user.get("root"))
print("Name :", u.name)
print("UID  :", u.uid)
print("GID  :", u.gid)
print("GECOS:", u.gecos)
print("Home :", u.home)
print("Shell:", u.shell)
```

Look up an account with get

Signature:

```
local info, err = babet.user.get(name_or_uid)
```

Use get when you need account details or must distinguish a missing account from an NSS failure.

By name

```
local root, err = babet.user.get("root")
assert(root, err)
assert(root.name == "root")
assert(root.uid == 0)
```

Preparing a directory for a service account:

```
local account, err = babet.user.get("my-service")
assert(account, err)

assert(babet.mkdir("/var/lib/my-service"))
assert(babet.setAttributes(
    "/var/lib/my-service",
    account.uid,
    account.gid,
    tonumber("750", 8)
))
```

The name is passed to NSS unchanged. It must be a string without an embedded NUL byte. An empty string is a valid API input, but normally matches no account and returns `user not found`.

By UID

```
local root, err = babet.user.get(0)
assert(root, err)
assert(root.name == "root")
```

Displaying the account associated with a file owner:

```
local attrs, err = babet.getAttributes("report.txt")
assert(attrs, err)

local owner, user_err = babet.user.get(attrs.owner)
if owner then
    print("Owner:", owner.name)
else
    print("Unresolved owner UID:", attrs.owner, user_err)
end
```

The UID must be a non-negative integer compatible with the system `uid_t` type. On Linux the limit is usually $2^{32} - 1$, but the code uses the actual platform limit at compile time.

Missing account

A missing account is not a Lua error. The function returns:

```
local info, err = babet.user.get("missing-account")
-- info == nil
-- err == "user not found"
```

Typical handling:

```

local account, err = babet.user.get("my-service")
if not account then
    if err == "user not found" then
        print("The account must be created")
    else
        print("NSS lookup failed:", err)
    end
end

```

Avoid this when absence is expected:

```

-- Wrong for an optional account: assert raises immediately
-- local account = assert(babet.user.get("optional-account"))

```

Test existence with exists

Signature:

```

local present = babet.user.exists(name_or_uid)

```

The function always returns a boolean for a valid argument.

By name

```

if babet.user.exists("www-data") then
    print("The www-data account exists")
end

```

Simple precondition:

```

if not babet.user.exists("my-service") then
    error("The my-service account must exist before startup")
end

```

By UID

```

assert(babet.user.exists(0)) -- root on a normal Unix system

```

A missing UID returns false:

```

local present = babet.user.exists(2000000000)
print(present)

```

When to prefer get

exists deliberately converts an NSS error to false.

This makes it convenient for a simple branch, but it cannot distinguish:

- an account that truly does not exist;
- a temporarily unavailable LDAP directory;
- an I/O error;
- an out-of-memory condition in the resolver.

For an important administrative or security decision, use get:

```

local account, err = babet.user.get("my-service")
if account then
    print("Account available")
elseif err == "user not found" then
    print("Account missing")
else

```

```

    error("Unable to query NSS: " .. err)
end

```

Argument validation

The following cases raise a Lua error, recoverable with pcall:

```

local invalid_calls = {
    function() return babet.user.get() end,
    function() return babet.user.get(nil) end,
    function() return babet.user.get(true) end,
    function() return babet.user.get({}) end,
    function() return babet.user.get(1.5) end,
    function() return babet.user.get(-1) end,
    function() return babet.user.get(8589934592) end,
    function() return babet.user.get("root\0other") end,
}

for _, call in ipairs(invalid_calls) do
    local ok, err = pcall(call)
    assert(not ok)
    print(err)
end

```

exists applies exactly the same validation:

```

local ok = pcall(function()
    return babet.user.exists(-1)
end)
assert(not ok)

```

Rejecting NUL bytes prevents "root\0other" from being interpreted as "root" by getpwnam_r.

NSS, workers, and security

Account sources

The result depends on the machine's NSS configuration. According to /etc/nsswitch.conf, an account may come from:

- /etc/passwd;
- LDAP;
- SSSD;
- NIS;
- FreeIPA;
- systemd-userdb;
- another NSS module.

The same script may therefore see different accounts on different machines without any Babet change.

Calls from workers

The module uses the reentrant getpwnam_r and getpwuid_r functions. It can be used inside workers:

```

local worker = assert(babet.workers.spawn([
    local root, err = babet.user.get("root")
    if not root then
        error(err)
    end
])

```

```

    return root.uid
  end
end
end

```

```

local joined, uid = worker:join()
assert(joined and uid == 0)

```

Each call queries NSS. Babet does not maintain an application-level user cache.

Security-sensitive decisions

The home and shell fields are configuration data, not proof that a path exists or that a program is executable.

```

local u = assert(babet.user.get("my-service"))
local home_is_dir = assert(babet.isDir(u.home))
local shell_path = babet.which(u.shell)

```

Likewise, the presence of an account does not prove that it can log in, that a password is valid, or that it belongs to a particular supplementary group.

Error contract

Situation	get	exists
account found	user table	true
account missing	(nil, "user not found")	false
NSS error	(nil, "user: ...")	false
invalid argument	Lua error	Lua error

Generic handling:

```

local ok, info, err = pcall(function()
    return babet.user.get("my-service")
end)

if not ok then
    print("Invalid call:", info)
elseif not info then
    print("Lookup failed:", err)
else
    print("UID:", info.uid)
end

```

Design decisions and limits

- NSS is used instead of parsing /etc/passwd.
- Reentrant _r variants are used for worker compatibility.
- Text fields are always present and replaced with "" if NSS supplies a null pointer.
- The NSS buffer grows dynamically up to an internal 64 KiB limit; beyond that, get returns an NSS error.
- exists favors a simple boolean API and hides NSS errors; use get when a diagnosis is required.
- The module does not expose supplementary groups.
- It exposes neither /etc/shadow, passwords, nor account expiration data.
- It does not create, remove, or modify users.
- To create an account, a script can invoke a system utility through `babet.exec`, with appropriate privileges.

WORKERS — OS threads, isolated Lua states, and message queues

`babel.workers` executes Lua code in real POSIX threads. Each worker owns a separate `lua_State`, receives a serialized copy of its initial arguments, and communicates with the parent through two bounded message queues.

The module covers:

- starting a Lua chunk in an OS thread;
- passing an initial argument table;
- blocking or non-blocking retrieval of the first return value;
- a parent-to-worker inbox and a worker-to-parent outbox;
- timeouts and capacity-based backpressure;
- cooperative channel closing;
- loading Babet, bundled, and user modules in each state;
- Lua-memory isolation between threads.

It does not provide shared Lua memory, forced termination, a timeout for `join()`, a global pool, or direct worker-to-worker channels.

Module contents

- [Essential conventions](#)
- [API overview](#)
- [spawn\(code, args?, opts?\)](#)
 - [code](#)
 - [args and worker.args](#)
 - [Queue capacities](#)
 - [When setenv and chdir become locked](#)
- [Transferable values](#)
 - [Scalars and strings](#)
 - [List and object tables](#)
 - [Rejected values](#)
 - [Copies, identity, and depth](#)
- [Final worker result](#)
 - [join\(\) — wait and consume](#)
 - [poll\(\) — check and consume](#)
 - [Do not call join\(\) after completed poll\(\)](#)
- [Parent-to-worker messages](#)
- [Worker-to-parent messages](#)
- [worker.send and worker.recv](#)
- [Timeouts and failure reasons](#)
- [close\(\) and queue lifecycle](#)
- [Module loading and worker environment](#)
- [Complete examples](#)
 - [Simple calculation with join](#)
 - [Structured arguments](#)
 - [Non-blocking progress with poll](#)
 - [Persistent request/response worker](#)
 - [Sending and receiving nil](#)
 - [Close, then drain the outbox](#)
 - [Backpressure and timeout](#)
 - [Cooperative cancellation](#)
 - [Correct bounded pool](#)
- [Deadlocks to avoid](#)
- [Error contract](#)

- [Garbage collection and destruction](#)
- [Signals, environment, and concurrency](#)
- [Features not provided](#)

Essential conventions

Each worker owns a separate Lua state

A table, global, or module changed in the parent is not shared with a worker. Data crosses state boundaries only through serialization:

- args during spawn;
- `job:send` and `worker:recv`;
- `worker:send` and `job:recv`;
- the chunk's first return value.

There can therefore be no data race on Lua objects themselves. External side effects — files, sockets, databases, and processes — remain system resources and must still be coordinated by the application.

Results use a pcall-like convention

`join()` returns:

```
true, business_value
false, error_message
```

This lets a worker legitimately return nil:

```
local ok, value = job:join()
if ok then
    -- value may be nil.
else
    io.stderr:write(value, "\n")
end
```

Queue methods also return a boolean first, so a nil message is unambiguous.

The final result can be consumed once

As soon as `join()` retrieves the result, or `poll()` returns "done" or "error", that result is consumed. A later `join()` or `poll()` reports result already consumed.

API overview

Parent side

```
local job, err = babet.workers.spawn(code, args?, opts?)

local ok, value = job:join()
local state, value = job:poll()

local ok, err = job:send(value, timeout?)
local ok, value_or_err = job:recv(timeout?)
local ok, err = job:close()
```

API	Main result	Blocking
spawn	job or (nil, err)	creation only

API	Main result	Blocking
job:join()	(true, result) or (false, err)	yes, no timeout
job:poll()	("running", nil), ("done", result), or ("error", err)	no
job:send(v, t?)	(true, nil), (false, reason), or (nil, err)	depends on t
job:recv(t?)	(true, value) or (false, reason)	depends on t
job:close()	(true, nil)	no

Worker side

The chunk receives a global worker table:

```
worker.args
worker.send(value, timeout?)
worker.recv(timeout?)
```

API	Result
worker.args	copy of args, or nil
worker.send(v, t?)	(true, nil) or (false, reason_or_error)
worker.recv(t?)	(true, value) or (false, reason)

spawn(code, args?, opts?)

```
local job, err = babet.workers.spawn(code, args?, opts?)
```

On success, spawn returns only the job userdata. The thread may still be running, may already have completed, or may already have failed by the time the parent receives the userdata.

Chunk load or runtime errors occur **inside** the worker: spawn may succeed, then join() or poll() reports the error.

```
local job = assert(babet.workers.spawn("this is not valid Lua"))
local ok, err = job:join()
assert(not ok)
```

code

code is required and must be an actual Lua string. Numbers are not coerced.

```
local job = assert(babet.workers.spawn([
    return 6 * 7
]))
```

The source is loaded as a text chunk in a fresh state. It is not a closure that captures parent upvalues.

```
local factor = 3
```

```
-- factor does not exist in the worker: pass it through args.
```

```
local job = assert(babet.workers.spawn([
    return worker.args.value * worker.args.factor
]), {
    value = 14,
    factor = factor,
})
```

The binding does not truncate NUL bytes in code, but a NUL normally makes the source invalid for the Lua parser. Use ordinary Lua source text.

args and worker.args

args is optional. It must be a table or nil.

```
local job = assert(babet.workers.spawn([[
    return worker.args.left + worker.args.right
]], {
    left = 20,
    right = 22,
}))
```

Inside the worker:

- worker.args contains a deserialized copy of the table;
- without args, worker.args == nil;
- modifying worker.args never changes the parent's table;
- metatables and subtable identity do not cross the boundary.

A direct scalar is not accepted as the second argument:

```
babet.workers.spawn("return worker.args", 42) -- raises a Lua error
```

Wrap it in a table:

```
babet.workers.spawn("return worker.args.value", { value = 42 })
```

Queue capacities

opts can define:

```
{
    inbox_capacity = 64,
    outbox_capacity = 64,
}
```

- inbox_capacity: maximum queued parent-to-worker messages;
- outbox_capacity: maximum queued worker-to-parent messages;
- default: 64 messages for each queue;
- allowed range: integer from 1 to 1,000,000;
- the value must be an actual Lua integer; the string "64" and the floating-point value 64.0 are rejected;
- capacity counts **messages**, not bytes.

```
local job = assert(babet.workers.spawn(code, nil, {
    inbox_capacity = 8,
    outbox_capacity = 32,
}))
```

A larger capacity may consume more memory and does not replace a draining strategy. A small capacity applies backpressure earlier.

Unknown opts fields are currently ignored.

When setenv and chdir become locked

After validating types and capacities, the first spawn marks the process as having used workers. From then on:

- babet.setenv(...) is permanently rejected;
- babet.chdir(...) is permanently rejected;

- the restriction remains after `join()`;
- it remains even if `args` serialization, queue initialization, or `pthread_create` later fails.

A call rejected **before** this mark — for example `spawn(42)` or a non-integer capacity — does not lock the environment.

Configure the current directory and environment before the first worker.

Transferable values

The internal transport format is JSON. The same rules apply to:

- `args`;
- messages in either direction;
- the worker's first return value.

Scalars and strings

Lua value	Transferable	Note
<code>nil</code>	yes	becomes JSON null, then <code>nil</code> again
<code>boolean</code>	yes	identity preserved
<code>integer</code>	yes	normally restored as a Lua integer
<code>finite floating-point number</code>	yes	NaN, +Inf, and -Inf rejected
<code>text string</code>	yes	must pass the UTF-8 validator and contain no NUL

Unlike several filesystem and socket APIs, worker messages are **not binary-safe**. A string containing `\0` or bytes rejected by the UTF-8 validator cannot be transferred.

List and object tables

A table is transferable in one of two shapes.

Non-empty dense list

```
{ "a", "b", "c" }
```

All keys must be consecutive integers `1..n`, with no holes and no other key.

String-keyed object

```
{
  name = "alice",
  age = 30,
}
```

All keys must be strings accepted by the text validator.

An empty table is serialized as an **empty object** `{}`, not an empty array. There is no equivalent of `babel.json.empty_array` in the worker transport.

Sparse or mixed tables are rejected:

```
{ [1] = "a", [3] = "c" }      -- hole
{ [1] = "a", label = "x" }    -- list + map
{ [0] = "zero" }              -- numeric key outside 1..n
```

Rejected values

These values do not cross Lua states:

- functions and closures;
- userdata: socket, SQLite statement, watcher, job, file, and so on;
- Lua coroutines/threads;
- cyclic tables;
- sparse or mixed tables;
- non-string keys in object tables;
- NaN or infinite numbers;
- strings with NUL or bytes rejected as UTF-8;
- structures beyond the nesting limit.

Pass a path, URL, or serializable configuration instead of the system object itself. The worker then opens its own resource.

```
-- Wrong: socket is userdata.
babet.workers.spawn(code, { socket = peer })

-- Correct: pass connection parameters.
babet.workers.spawn(code, {
    host = "127.0.0.1",
    port = 9000,
})
```

Copies, identity, and depth

Transport copies values:

- no Lua reference is shared;
- metatables are lost;
- two references to the same subtable become two separate tables;
- cycles are rejected;
- maximum nesting is 32 levels according to the internal serialization counter.

```
local shared = { value = 1 }
local args = { a = shared, b = shared }

-- In the worker, worker.args.a and worker.args.b have equal content,
-- but are not the same table.
```

The limit protects the C++ and Lua stacks from pathological structures.

Final worker result

When the chunk completes normally, only its **first** return value is serialized.

```
return "first", "second", "third"
```

The parent receives only "first".

Return a table to transport several fields:

```
return {
    value = 42,
    elapsed = 0.12,
}
```

With no return, or return nil, the business result is nil.

An uncaught Lua error becomes the error state. The transferred text is a useful diagnostic, but Babet does not automatically add a traceback. A worker can use `xpcall(..., debug.traceback)` when it wants to build one.

join() — wait and consume

```
local ok, value = job:join()
```

join():

- waits without a timeout for the pthread to finish;
- does not close either queue before waiting;
- permanently consumes the result;
- returns (true, value) after normal completion;
- returns (false, err) for Lua errors, internal errors, or invalid results.

```
local ok, value = job:join()
```

```
if ok then
```

```
    print("result", value)
```

```
else
```

```
    io.stderr:write("worker: ", value, "\n")
```

```
end
```

Even when the result is nil, success remains identifiable through `ok == true`.

poll() — check and consume

```
local state, value = job:poll()
```

state	value	Consumed
"running"	nil	no
"done"	result, possibly nil	yes
"error"	message	yes

```
while true do
```

```
    local state, value = job:poll()
```

```
    if state == "running" then
```

```
        babet.sleep(10, "ms")
```

```
    elseif state == "done" then
```

```
        print("completed", value)
```

```
        break
```

```
    else
```

```
        io.stderr:write("worker: ", value, "\n")
```

```
        break
```

```
    end
```

```
end
```

Once the state is no longer running, `poll()` quickly joins the already completed pthread and consumes the result.

Do not call join() after completed poll()

This sequence is wrong:

```
local state = job:poll()
```

```
if state ~= "running" then
```

```
    local ok, result = job:join() -- result already consumed
```

```
end
```

Use both values from `poll()` directly:

```
local state, value = job:poll()
if state == "done" then
    use_result(value)
elseif state == "error" then
    report_error(value)
end
```

Alternatively, use `poll()` only while it returns running, and design another mechanism that tells code when to call `join()` without first consuming the result. In most event loops, polling through the final result is simpler.

Parent-to-worker messages

```
local ok, err = job:send(value, timeout?)
```

`job:send` serializes `value`, then appends it to the worker inbox in FIFO order.

Results:

- `(true, nil)`: message accepted;
- `(false, "full")`: full queue with timeout 0;
- `(false, "timeout")`: queue stayed full until the deadline;
- `(false, "closed")`: inbox closed;
- `(nil, err)`: non-transferable value or internal serialization failure.

```
local ok, err = job:send({ command = "scan", path = "/tmp" }, 1)
if not ok then
    io.stderr:write("send: ", err, "\n")
end
```

Success means the message entered the queue, not that the worker has already processed it.

Worker-to-parent messages

```
local ok, value_or_err = job:recv(timeout?)
```

`job:recv` removes the oldest message from the outbox.

Results:

- `(true, value)`: message received; value may be nil;
- `(false, "empty")`: empty queue with timeout 0;
- `(false, "timeout")`: no message before the deadline;
- `(false, "closed")`: queue closed and fully drained.

When a queue is closed but still contains messages, those messages are returned before "closed".

```
while true do
    local ok, message = job:recv(0)
    if ok then
        process(message)
    elseif message == "empty" then
        break
    elseif message == "closed" then
        break
    else
        error(message)
    end
end
```

worker.send and worker.recv

Inside the chunk:

```
local ok, message_or_err = worker.recv(timeout?)
local ok, err = worker.send(value, timeout?)
```

worker.recv reads the inbox filled by job:send. worker.send writes the outbox read by job:recv.

FIFO ordering, capacities, timeouts, and closing are symmetric.

Serialization-error convention differs slightly:

- job:send returns (nil, err);
- worker.send returns (false, err).

This keeps a boolean status convention inside worker code.

```
local job = assert(babet.workers.spawn([
  while true do
    local ok, message = worker.recv()
    if not ok then
      return "inbox closed"
    end

    local sent, err = worker.send({ echo = message })
    if not sent then
      return { send_error = err }
    end
  end
]))
```

Timeouts and failure reasons

All four message methods share the same contract:

Lua timeout	Behavior
omitted or nil	wait indefinitely
0	immediate, non-blocking attempt
$0 < t \leq 86400$	wait at most t seconds
negative, NaN, Inf, or > 86400	raises a Lua error
other type, including "1"	raises a Lua error

Internal resolution is one millisecond. A strictly positive duration is rounded up: 0.0005 second waits at least 1 ms and is not turned into non-blocking mode.

Queue reasons are:

Reason	Meaning
"full"	immediate send, queue full
"empty"	immediate receive, queue empty
"timeout"	positive deadline reached
"closed"	send to closed queue, or receive from closed and empty queue

Internal diagnostics such as "out of memory" or "internal mutex error" may exceptionally appear instead.

close() and queue lifecycle

```
local ok, err = job:close()
```

```
job:close():
```

- closes only the parent-to-worker **inbox**;
- is idempotent;
- returns (true, nil);
- wakes a waiting worker.recv(), which gets (false, "closed") once already queued messages are drained;
- makes future job:send() calls fail with (false, "closed");
- leaves the outbox open so the parent can still receive final worker messages.

It does not forcibly terminate the thread and does not consume its final result.

The worker automatically closes **both** queues when it completes, whether successfully or with an error. The parent can still drain messages already in the outbox before receiving "closed".

To cleanly finish a worker that waits for commands:

```
assert(job:close())
```

```
local ok, result = job:join()
```

The worker must treat worker.recv() == false, "closed" as an exit condition.

Module loading and worker environment

Each thread creates a fresh Lua state and:

- opens the standard Lua libraries;
- registers the complete babet namespace;
- exposes bundled modules through require;
- configures user-module loading like the parent project, in directory or embedded mode;
- exposes worker.args, worker.send, and worker.recv;
- does not copy the parent's global arg table: arg == nil in the worker.

```
local job = assert(babet.workers.spawn([[
    local inspect = require("inspect")
    local mymod = require("mymod")
    return inspect({ answer = mymod.answer() })
]]))
```

Lua states are isolated, but threads belong to the same process:

- same PID;
- same process-wide current directory;
- same process-wide environment;
- same filesystem and accessible external resources;
- Lua memory measured separately in each state.

Babet rejects cwd and environment mutations after the first spawn, stabilizing those two global states.

Complete examples

Simple calculation with join

```
local job, err = babet.workers.spawn([[
    return 21 * 2
]])
assert(job, err)
```

```

local ok, result = job:join()
assert(ok, result)
print(result) -- 42

```

Structured arguments

```

local job = assert(babet.workers.spawn([[
    local total = 0
    for _, value in ipairs(worker.args.values) do
        total = total + value
    end
    return {
        name = worker.args.name,
        total = total,
    }
]], {
    name = "batch A",
    values = { 10, 20, 12 },
}))

```

```

local ok, result = job:join()
assert(ok, result)
print(result.name, result.total)

```

Non-blocking progress with poll

```

local job = assert(babet.workers.spawn([[
    babet.sleep(200, "ms")
    return "ready"
]]))

```

```

while true do
    local state, value = job:poll()

    if state == "running" then
        update_ui()
        babet.sleep(10, "ms")
    elseif state == "done" then
        print(value)
        break
    else
        error(value)
    end
end

```

Do not call join() after the done or error branch: poll() has already consumed the result.

Persistent request/response worker

```

local job = assert(babet.workers.spawn([[
    while true do
        local ok, request = worker.recv()
        if not ok then
            return "parent closed inbox"
        end

        if request.op == "stop" then

```

```

        return "stopped"
    elseif request.op == "square" then
        local sent, err = worker.send({
            id = request.id,
            value = request.value * request.value,
        })
        if not sent then
            return { error = err }
        end
    end
end
end
]], nil, {
    inbox_capacity = 16,
    outbox_capacity = 16,
}))

for i = 1, 5 do
    assert(job:send({ id = i, op = "square", value = i }))
end

for _ = 1, 5 do
    local ok, response = job:recv(2)
    assert(ok, response)
    print(response.id, response.value)
end

assert(job:send({ op = "stop" }))
local ok, result = job:join()
assert(ok, result)
print(result)

```

Sending and receiving nil

The status boolean distinguishes a real nil message from an empty queue.

```

local job = assert(babet.workers.spawn([[
    local ok, value = worker.recv()
    assert(ok)
    assert(value == nil)

    assert(worker.send(nil))
    return "done"
]]))

assert(job:send(nil))

local got, value = job:recv()
assert(got == true)
assert(value == nil)

local ok, result = job:join()
assert(ok and result == "done")

```

Close, then drain the outbox

```

local job = assert(babet.workers.spawn([[
    assert(worker.send("ready"))

```

```

    local ok, reason = worker.recv()
    assert(not ok and reason == "closed")

    assert(worker.send("closing"))
    return "finished"
]))

local ok, first = job:recv(1)
assert(ok and first == "ready")

assert(job:close()) -- closes the inbox only

local got, last = job:recv(1)
assert(got and last == "closing")

local joined, result = job:join()
assert(joined and result == "finished")

```

Backpressure and timeout

```

local job = assert(babet.workers.spawn([
    babet.sleep(500, "ms")
    while true do
        local ok, message = worker.recv()
        if not ok then return "closed" end
        if message == "stop" then return "done" end
    end
]), nil, {
    inbox_capacity = 2,
}))

assert(job:send("one", 0))
assert(job:send("two", 0))

local ok, reason = job:send("three", 0)
assert(ok == false and reason == "full")

local waited, why = job:send("three", 0.1)
assert(waited == false and why == "timeout")

job:close()
job:join()

```

Cooperative cancellation

There is no `job:cancel()`. The parent sends a command and the worker checks it periodically.

```

local job = assert(babet.workers.spawn([
    local total = 0

    for i = 1, worker.args.limit do
        total = total + expensive_step(i)

        if i % 1000 == 0 then
            local ok, message = worker.recv(0)
            if ok and message == "stop" then

```

```

        return {
            cancelled = true,
            partial = total,
        }
    elseif not ok and message == "closed" then
        return {
            cancelled = true,
            partial = total,
        }
    end
end
end
end

```

```

    return { cancelled = false, total = total }
]], { limit = 1000000 }))

```

```

-- Later:
local sent, err = job:send("stop", 0.5)
if not sent then
    io.stderr:write("cancel: ", err, "\n")
end

```

```

local ok, result = job:join()
assert(ok, result)

```

Cancellation remains cooperative: a worker stuck in a non-interruptible call, or one that never checks its queue, does not stop because of this message.

Correct bounded pool

This loop bounds simultaneous pthreads and directly uses the `poll()` result, without calling `join()` afterward.

```

local function map_parallel(items, code, max_concurrent)
    max_concurrent = max_concurrent or 4

    local next_index = 1
    local active = {}
    local results = {}

    while next_index <= #items or #active > 0 do
        while next_index <= #items and #active < max_concurrent do
            local job, err = babet.workers.spawn(code, {
                item = items[next_index],
            })
            assert(job, err)

            active[#active + 1] = {
                index = next_index,
                job = job,
            }
            next_index = next_index + 1
        end

        for i = #active, 1, -1 do
            local entry = active[i]
            local state, value = entry.job:poll()

```

```

        if state == "done" then
            results[entry.index] = value
            table.remove(active, i)
        elseif state == "error" then
            results[entry.index] = { error = value }
            table.remove(active, i)
        end
    end

    if #active > 0 then
        babet.sleep(10, "ms")
    end

    return results
end

```

Every spawn still creates a new thread and Lua state. This pattern bounds concurrency; it does not reuse persistent workers.

Deadlocks to avoid

Worker waits for inbox while parent calls join()

```

-- Worker
local ok, message = worker.recv() -- waits indefinitely

-- Parent
job:join() -- waits for worker

```

Both sides wait. Close the inbox or send a command first:

```

job:close()
job:join()

```

Worker blocks on a full outbox

If the worker calls `worker.send()` with no timeout, the outbox is full, and the parent calls `join()` without draining it, neither side can progress.

Solutions:

- parent regularly calls `job:recv()`;
- worker uses a finite timeout;
- outbox capacity matches the protocol;
- message and join phases are clearly separated.

Forgetting that `poll()` consumes

Code that waits for `poll()` `~= "running"` and then calls `join()` does not deadlock, but loses the real result and gets already consumed. Use both values from `poll()`.

Potentially blocking GC

Dropping the last reference to an active worker can run `__gc`, which closes queues and then waits for the thread. Queue waits are released, but GC cannot interrupt infinite computation or an unbounded external syscall.

Error contract

Lua errors raised for invalid use

These include:

- missing or non-string code;
- args other than table or nil;
- opts other than table or nil;
- non-numeric, non-integer, or out-of-range capacities;
- non-numeric, negative, non-finite, or greater-than-86,400-second timeout;
- method call on a value that is not a job.

```
local ok, err = pcall(function()  
    babet.workers.spawn(42)  
end)  
assert(not ok)
```

Failures returned by spawn

spawn returns (nil, err) for runtime failures before or during creation:

- non-transferable args;
- JSON serialization failure;
- queue initialization failure;
- pthread_create failure.

```
local job, err = babet.workers.spawn("return 1", {  
    fn = function() end,  
})  
assert(job == nil)
```

Final worker errors

join() returns (false, err) and poll() returns ("error", err) for:

- invalid Lua syntax;
- uncaught Lua error;
- non-transferable return value;
- caught internal C++ exception;
- internal result deserialization corruption or failure.

The diagnostic does not automatically contain a complete traceback.

Message serialization

- job:send: (nil, err) when the value cannot be serialized;
- worker.send: (false, err) for the same case;
- recv: (false, err) on an internal deserialization anomaly.

Garbage collection and destruction

The userdata has a safety-net __gc. When collected:

1. inbox and outbox are closed;
2. queue waiters are awakened;
3. Babet calls pthread_join when needed;
4. pthread primitives and internal strings are destroyed.

This prevents freeing state still used by a thread. It can nevertheless block when the worker cannot terminate.

Do not use GC as the normal synchronization mechanism. Keep the job, finish the protocol, then call `join()` or consume the result through `poll()`.

`tostring(job)` returns an indication such as:

```
Worker(running)
Worker(done)
Worker(error)
```

This reflects instantaneous internal status, not whether the result has already been consumed.

Signals, environment, and concurrency

Signals

All six signals managed by `babet.signal` are blocked in worker pthreads. `babet.signal.handle`, `ignore`, and `default` raise inside a worker. The main thread remains the only owner of POSIX callbacks.

Environment and cwd

`setenv` and `chdir` change process-wide state that cannot safely be mutated while other threads use it. Babet therefore rejects both after the first spawn.

External resources

Two workers can open distinct files, SQLite databases, or sockets. When they target the same external resource, worker serialization provides no automatic lock.

For SQLite, each worker should open its own connection. `wal = true` and a suitable `busy_timeout` can improve concurrency, but do not replace sound transaction design.

Cost

Each spawn creates:

- a pthread;
- a complete Lua state;
- Babet modules;
- two queues and their buffers;
- JSON copies of transferred data.

Avoid a worker for tiny operations. Batch work or use a few persistent workers when the protocol allows it.

Features not provided

The module currently does not provide:

- `job:join(timeout)`;
- `job:cancel()` or forced termination;
- `worker.cancelled()`;
- `job:done()` — use `poll()`;
- `babet.workers.cpu_count()`;
- a reusable global pool;
- direct worker-to-worker channels;
- shared Lua memory;
- transfer of functions, userdata, or coroutines;
- a binary message format;
- automatic transfer of multiple return values.

Cooperative cancellation and a bounded pool can be built in Lua using the examples on this page. CPU count can be queried through an external program such as `babet.exec("nproc")`, after checking its result table.

Cookbook

Concrete recipes for common scripting tasks. Each recipe is short and self-contained ; copy, adapt, ship.

Watch a directory and email new files

A drop directory that mails out every file as it arrives. Uses `babet.inotify` for instant wakeup (no polling), and listens on `close_write` + `moved_to` so the file is guaranteed complete.

```
local w = assert(babet.inotify.new())
assert(w:add("/srv/incoming", { "close_write", "moved_to" }))

babet.signal.handle("TERM", function()
    w:close()
    os.exit(0)
end)

while true do
    local events, err = w:read()
    if not events then
        if err == "interrupted" then break end
        io.stderr:write("inotify: ", err, "\n"); break
    end
    for _, ev in ipairs(events) do
        if ev.events.overflow then
            -- queue overflowed: rescan the directory
            -- (events were lost)
        else
            local path = "/srv/incoming/" .. ev.name
            send_email(path)
            os.remove(path)
        end
    end
end
end
```

Ensure a service user exists, then drop to it

A typical install-time check before launching a daemon as a non-root user.

```
local function ensure_user(name)
    if babet.user.exists(name) then return true end
    local ok, err = babet.exec("useradd", {
        "--system",
        "--home-dir", "/var/lib/" .. name,
        "--create-home",
        "--shell", "/usr/sbin/nologin",
        name,
    })
    if not ok then
        return nil, "useradd failed: " .. tostring(err)
    end
end
```

```

    return true
end

assert(ensure_user("myapp"))
local u = assert(babet.user.get("myapp"))
babet.chdir(u.home)

```

Fetch many URLs in parallel

babet.workers runs Lua on multiple cores. Total wall-clock is close to the slowest single fetch, not the sum.

```

local urls = { "https://a.example/", "https://b.example/",
               "https://c.example/", "https://d.example/" }

local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = babet.workers.spawn([[
        local url = worker.args.url
        local r, err = babet.http.get(url, { timeout = 10 })
        if not r then return { error = err } end
        return { status = r.status, length = #r.body }
    ]], { url = url })
end

for i, job in ipairs(jobs) do
    local ok, result = job:join()
    print(i, ok, result and result.status or result.error)
end

```

Graceful shutdown of a long-running script

Catch SIGTERM / SIGINT so the script can close resources properly. Works with `systemctl stop` and Ctrl-C.

```

local log = require("logging")
log.set_level("info")

local running = true
babet.signal.handle("TERM", function()
    log.info("SIGTERM, shutting down")
    running = false
end)
babet.signal.handle("INT", function()
    log.info("Ctrl-C, shutting down")
    running = false
end)

while running do
    -- main loop ; blocking calls return (nil, "interrupted") when
    -- a handled signal arrives, so the loop exits quickly.
    local line, err = sock:recv_line()
    if not line then
        if err == "interrupted" then break end
        log.error("recv: ", err); break
    end
    process(line)
end

```

```
end
```

```
sock:close()
log.info("clean shutdown")
```

Read a TOML config, validate, persist to SQLite

```
-- Load TOML
local f = assert(io.open("config.toml", "r"))
local body = f:read("a"); f:close()
local cfg, perr = babet.toml.decode(body)
if not cfg then error("config.toml: " .. perr) end

-- Validate (cheap manual checks ; for a real schema, build one in Lua)
assert(type(cfg.server) == "table", "missing [server]")
assert(type(cfg.server.host) == "string", "missing server.host")
assert(math.type(cfg.server.port) == "integer", "server.port must be integer")

-- Open db, persist
local db = assert(babet.sqlite.open("state.db", { wal = true }))
db:exec([[CREATE TABLE IF NOT EXISTS config (k TEXT PRIMARY KEY, v TEXT)])
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",
        { "host", cfg.server.host })
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",
        { "port", tostring(cfg.server.port) })
db:close()
```

Cache with a human-readable TTL, expiry as ISO timestamps

Parse a TTL from config ("15m", "2h30m", "1d"), compute a deadline, and log it in a uniform UTC format that grep'ing across log files actually works on.

```
-- Config from TOML, CLI, env, anywhere.
local raw_ttl = "15m"

local ttl, err = babet.time.parse_duration(raw_ttl)
if not ttl then error("bad TTL '" .. raw_ttl .. "': " .. err) end

local deadline = babet.now() + ttl
print(string.format("[%s] cache entry created, expires at %s (TTL %s)",
    babet.time.iso(),
    babet.time.iso(deadline),
    babet.time.format_duration(ttl)))

-- ...later, somewhere else in the script:
if babet.now() >= deadline then
    print "[" .. babet.time.iso() .. "] cache miss: entry expired"
end
```

Useful properties of this pattern :

- `babet.time.iso()` is always UTC, always 20 characters, always the same shape. `grep` and `sort` work on the timestamps directly.
- `parse_duration("15m")` returns an integer, so `now() + ttl` is exact (no float drift over long durations).
- `format_duration(ttl)` round-trips : `parse_duration( format_duration(n)) == n` for every `n >= 0`. Safe to use as a canonical representation in logs or databases.

Pretty-print a Lua value

The bundled `inspect` module gives readable output for nested tables.

```
local inspect = require("inspect")
print(inspect({ a = 1, b = { c = 2, d = { 3, 4, 5 } } }))
-- { a = 1, b = { c = 2, d = { 3, 4, 5 } } }
```

Hello there

The minimal Babet script :

```
babet.helloThere()
```